

강화학습

LG Aimers



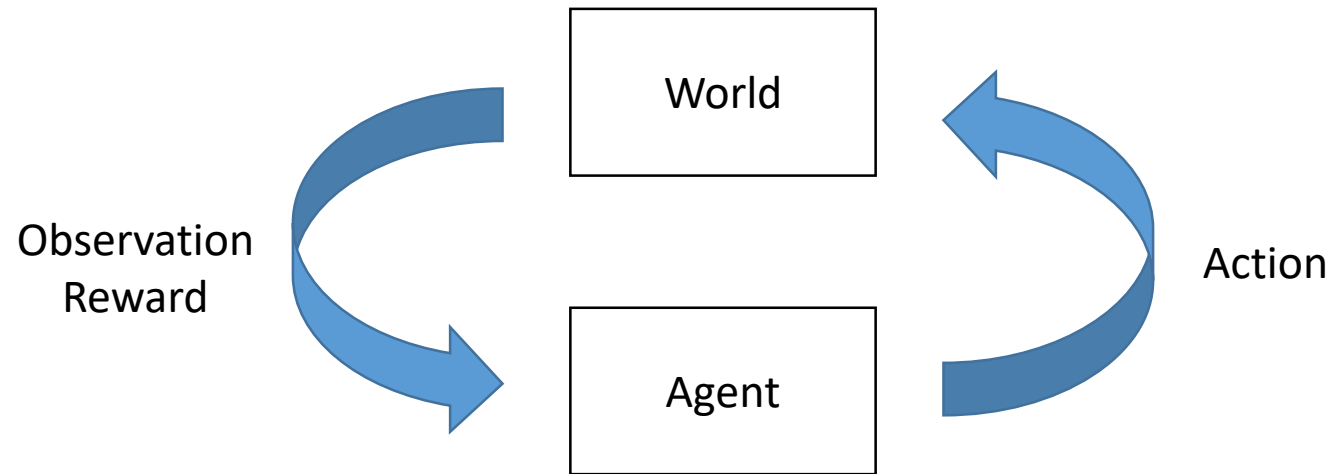
고려대학교 인공지능학과 이병준
Lecture 1: MDP and Planning

Learn to make good sequences of decisions

- Everything in our lives is sequential decision making
- We naturally learn to do it by trial-and-error



Sequential decision making



- Goal: Select actions to maximize total expected future reward
- May require balancing immediate & long term rewards

Markov Decision Processes (MDP)

- “Process”
 - (informal) Dealing with a system that changes over time
 - Here we deal with **discrete time**
- “Markov”
 - System state (full information about the current system) is observable
 - Given state, we do not have to see the **past history**
- “Decision”
 - We make decision (i.e. act) each time step
 - In order to maximize/minimize some criterion (reward/cost)

Markov Processes

- Markov Processes = (S, P)
 - State space and transition dynamics
- Markov Reward Processes = (S, P, R, γ)
 - Markov Processes + reward model + discount factor
- Markov Decision Processes = (S, A, P, R, γ)
 - Markov Reward Processes + Action space

Markov Processes (i.e. Markov Chain)

- Memoryless random process $M = (S, P)$
 - S is a (finite) set of states ($s \in S$)
 - $P: S \rightarrow \Delta(S)$ is dynamics/transition model that specifies $p(s_{t+1} = s' | s_t = s)$
- **Markov Property:** Future is independent of past given present
 - $p(s_{t+1} | s_t) = p(s_{t+1} | s_t, s_{t-1}, \dots, s_0)$
- If finite states (N), P is matrix
 - Elements are non-negative and rows sum to one

$$P = \begin{bmatrix} P(s_1 | s_1) & \cdots & P(s_N | s_1) \\ \vdots & \ddots & \vdots \\ P(s_1 | s_N) & \cdots & P(s_N | s_N) \end{bmatrix}$$

Markov Reward Processes (MRP)

- Markov Reward Process is a Markov Chain + rewards (no actions)
 - S is a (finite) set of states ($s \in S$)
 - $P: S \rightarrow \Delta(S)$ is dynamics/transition model that specifies $p(s_{t+1} = s' | s_t = s)$
 - $R: S \rightarrow \mathbb{R}$ is a reward function $R(s_t = s) = \mathbb{E}[r_t | s_t = s]$
 - Discount factor $\gamma \in [0,1]$
- **Markov Property:** Future is independent of past given present
 - $p(s_{t+1}, r_t | s_t) = p(s_{t+1}, r_t | s_t, r_{t-1}, s_{t-1}, r_{t-2}, \dots, r_0, s_0)$
- If finite states (N), R is a vector

Return & Value Function

- Definition of Horizon (H)
 - Number of time steps in each episode
 - Can be finite/infinite: finite-horizon MRP /infinite-horizon MRP
- Definition of Return, G_t
 - Discounted sum of rewards from time step t to horizon H
 - $G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots + \gamma^{H-1} r_{t+H-1}$
- Definition of State Value Function, $V(s): S \rightarrow \mathbb{R}$
 - Expected return from starting in state s
 - $V(s) = \mathbb{E}[G_t | s_t = s] = \mathbb{E}[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots + \gamma^{H-1} r_{t+H-1} | s_t = s]$

Discount factor

- Mathematically convenient (avoid infinite returns and values)
- Humans often act as if there's a discount factor < 1
- $\gamma = 0$: Only care about immediate reward
- $\gamma = 1$: Future reward is as beneficial as immediate reward
- If episode lengths are always finite ($H < \infty$), can use $\gamma = 1$
- Note that the maximum return in infinite horizon task is:
- $G_{max} = R_{max}(1 + \gamma + \gamma^2 + \dots) = \frac{R_{max}}{1-\gamma}$
 - $\frac{1}{1-\gamma}$ works as an “effective length” of horizon

Computing the Value of MRP

- Markov property provides structure
- MRP value function satisfies

$$V(s) = \underbrace{R(s)}_{\text{Immediate reward}} + \underbrace{\gamma \sum_{s' \in \mathcal{S}} P(s'|s) V(s')}_{\text{Discounted sum of future rewards}}$$

- This equation is called Bellman equation
 - It can be easily proved from the definition of Value function

Matrix Form of Bellman Equation for MRP

- For finite state MRP, we can express $V(s)$ using a matrix equation

$$\begin{bmatrix} V(s_1) \\ \vdots \\ V(s_N) \end{bmatrix} = \begin{bmatrix} R(s_1) \\ \vdots \\ R(s_N) \end{bmatrix} + \gamma \begin{bmatrix} P(s_1|s_1) & \cdots & P(s_N|s_1) \\ P(s_1|s_2) & \cdots & P(s_N|s_2) \\ \vdots & \ddots & \vdots \\ P(s_1|s_N) & \cdots & P(s_N|s_N) \end{bmatrix} \begin{bmatrix} V(s_1) \\ \vdots \\ V(s_N) \end{bmatrix}$$

- $V = R + \gamma PV$

Analytic Solution for Value of MRP

- For finite state MRP, we can express $V(s)$ using a matrix equation

$$\begin{bmatrix} V(s_1) \\ \vdots \\ V(s_N) \end{bmatrix} = \begin{bmatrix} R(s_1) \\ \vdots \\ R(s_N) \end{bmatrix} + \gamma \begin{bmatrix} P(s_1|s_1) & \cdots & P(s_N|s_1) \\ P(s_1|s_2) & \cdots & P(s_N|s_2) \\ \vdots & \ddots & \vdots \\ P(s_1|s_N) & \cdots & P(s_N|s_N) \end{bmatrix} \begin{bmatrix} V(s_1) \\ \vdots \\ V(s_N) \end{bmatrix}$$

- $V = R + \gamma PV$
- $V - \gamma PV = R$
- $(I - \gamma P)V = R$
- $V = (I - \gamma P)^{-1}R$
- Solving directly requires taking a matrix inverse $\sim O(N^3)$
- Note that $(I - \gamma P)$ is invertible

Iterative Algorithm for Computing Value of a MRP

- Dynamic programming (DP):
 - Instead of evaluation based on exponentially many state sequences, we save and update intermediate results: value functions

- Initialize $V_0(s) = 0 \forall s \in S$
- For $k = 1$ until convergence

$$V_k(s) = R(s) + \gamma \sum_{s' \in S} P(s'|s) V_{k-1}(s') \quad \forall s \in S$$

- Computational complexity: $O(|S|^2)$ for each iteration ($|S| = N$)

Markov Decision Processes (MDP)

- Markov Decision Process is a Markov Reward Process + action: (S, A, P, R, γ)

- S is a (finite) set of states ($s \in S$)
- A is a (finite) set of actions ($a \in A$)
- $P: S, A \rightarrow \Delta(S)$ is dynamics/transition model for **each action** that specifies

$$p(s_{t+1} = s' | s_t = s, a_t = a)$$

- $R: S \rightarrow \mathbb{R}$ is a reward function for **each action**

$$R(s_t = s, a_t = a) = \mathbb{E}[r_t | s_t = s, a_t = a]$$

- Discount factor $\gamma \in [0, 1]$

- **Markov Property:** Future is independent of past given present

- $p(s_{t+1}, r_t | s_t, a_t) = p(s_{t+1}, r_t | s_t, a_t, r_{t-1}, s_{t-1}, a_{t-1}, \dots, r_0, s_0, a_0)$

MDP Policies

- Policy specifies what action to take in each state
 - Can be deterministic or stochastic
- Fully defines the behavior of an agent
 - Policies are *stationary* (time-independent, only dependent on current state)
- Deterministic policy $\pi: S \rightarrow A, a_t = \pi(s_t)$
- Stochastic policy $\pi: S \rightarrow \Delta(A), \pi(a|s) = p(a_t = a | s_t = s)$
 - Given a state, specifies a conditional distribution over actions

MDP + Policy

- MDP + $\pi(a|s)$ = Markov Reward Process
- Precisely, it is the MRP $(S, R^\pi, P^\pi, \gamma)$ where

$$R^\pi(s) = \sum_{a \in A} \pi(a|s) R(s, a)$$

$$P^\pi(s'|s) = \sum_{a \in A} \pi(a|s) P(s'|s, a)$$

- Implies we can use same techniques to evaluate the value of a policy for a MDP as we could compute the value of a MRP, by defining a MRP with R^π and P^π

MDP Policy Evaluation, Iterative Algorithm

- Equivalent to the value computation algorithm of MRP

- Initialize $V_0(s) = 0 \forall s \in S$
- For $k = 1$ until convergence

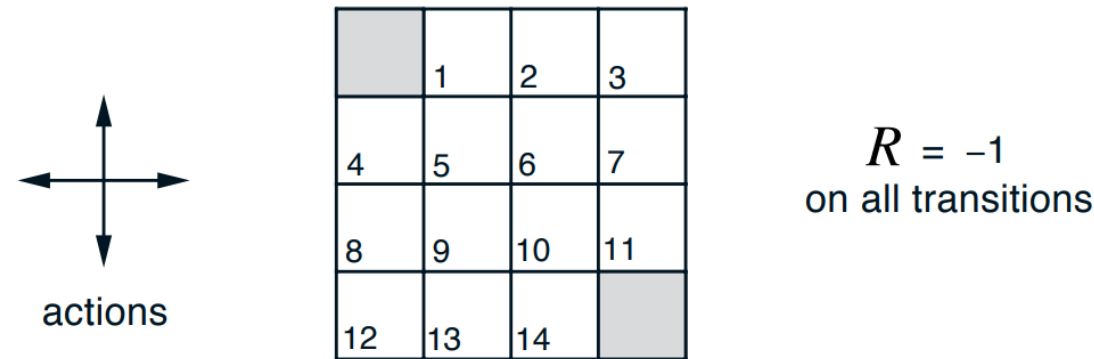
$$V_k^\pi(s) = R^\pi(s) + \gamma \sum_{s' \in S} P^\pi(s'|s) V_{k-1}^\pi(s') \quad \forall s \in S$$

or,

$$V_k^\pi(s) = \sum_{a \in A} \pi(a|s) \left(R(s, a) + \gamma \sum_{s' \in S} P(s'|s, a) V_{k-1}^\pi(s') \right) \quad \forall s \in S$$

- This is a Bellman backup for particular policy

Policy Evaluation Example: 4×4 gridworld



- Non-terminal states: $S = \{1, 2, \dots, 14\}$
- Four actions: $A = \{\text{up, down, right, left}\}$
 - Cause deterministic state transition except the borders
 - E.g., $P(6|5, \text{right}) = 1$, $P(10|5, \text{right}) = 0$, $P(7|7, \text{right}) = 1$
- Undiscounted, infinite horizon task with shaded terminal states
- Reward: $R(s, a) = -1$ for all non-terminal states (0 for terminal states)
- We use equiprobable random policy π_0

Policy Evaluation Example: 4×4 gridworld

V_k for the
Random Policy

$k = 0$

0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

$k = 2$

0.0	-1.7	-2.0	-2.0
-1.7	-2.0	-2.0	-2.0
-2.0	-2.0	-2.0	-1.7
-2.0	-2.0	-1.7	0.0

$k = 10$

0.0	-6.1	-8.4	-9.0
-6.1	-7.7	-8.4	-8.4
-8.4	-8.4	-7.7	-6.1
-9.0	-8.4	-6.1	0.0

$k = 1$

0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0

$k = 3$

0.0	-2.4	-2.9	-3.0
-2.4	-2.9	-3.0	-2.9
-2.9	-3.0	-2.9	-2.4
-3.0	-2.9	-2.4	0.0

$k = \infty$

0.0	-14.	-20.	-22.
-14.	-18.	-20.	-20.
-20.	-20.	-18.	-14.
-22.	-20.	-14.	0.0

Greedy action w.r.t. these value functions
already gives an optimal policy!

Optimal Policy

- Define a partial ordering over policies

$$\pi \geq \pi' \text{ if } V^\pi(s) \geq V^{\pi'}(s), \quad \forall s$$

- For any Markov Decision Process
 - There exists an optimal policy π^* that is better than or equal to all other policies, $\pi^* \geq \pi, \forall \pi$
 - All optimal policies achieve the (unique) optimal value function,

$$V^{\pi^*}(s) = V^*(s)$$

- How do we know there exists an optimal policy/value? Isn't it a partial ordering?
It's tricky..

Policy Search

- We now know that the optimal policy of an MDP is deterministic and stationary.
 - Well, this is only true with infinite-horizon MDP; stationary policy may not be optimal in finite-horizon MDP.
- Number of such policies = $|A|^{|S|}$
- Policy search algorithm should be better than that

MDP Policy Iteration (PI)

- Set $i = 0$
- Initialize $\pi_0(s)$ randomly for all states s
- While $i = 0$ or $\|\pi_i - \pi_{i-1}\|_1 > 0$ (L_1 -norm, measures if the policy changed for any state):
 - $V^{\pi_i} \leftarrow$ MDP V function policy **evaluation** of π_i
 - $\pi_{i+1} \leftarrow$ Policy **improvement**
 - $i = i + 1$

$$\pi_0 \xrightarrow{\text{E}} v_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} v_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \dots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} v_*$$

New Definition: State-Action Value Q

- State-action value of a policy

$$\begin{aligned} Q^\pi(s, a) &= \mathbb{E} \left[\sum_{t \geq 0} \gamma^t R(s_t, \pi(s_t)) \mid s_0 = s, a_0 = a, \pi \right] \\ &= R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' | s, a) V^\pi(s') \end{aligned}$$

- Take action a , then follow the policy π

Policy Improvement

- Compute state-action value of a policy π_i
 - For $s \in S, a \in A$:

$$Q^{\pi_i}(s, a) = R(s, a) + \gamma \sum_{s' \in S} P(s'|s, a) V^{\pi_i}(s')$$

- Compute new policy $\pi_{i+1}, \forall s \in S$

$$\pi_{i+1}(s) = \arg \max_a Q^{\pi_i}(s, a), \quad \forall s \in S$$

MDP Policy Iteration (PI)

- Set $i = 0$
- Initialize $\pi_0(s)$ randomly for all states s
- While $i = 0$ or $\|\pi_i - \pi_{i-1}\|_1 > 0$ (L_1 -norm, measures if the policy changed for any state):
 - $V^{\pi_i} \leftarrow$ MDP V function policy **evaluation** of π_i
 - $\pi_{i+1} \leftarrow$ Policy **improvement**
 - $i = i + 1$

$$\pi_0 \xrightarrow{\text{E}} v_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} v_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \dots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} v_*$$

Delving Deeper Into Policy Improvement Step

$$Q^{\pi_i}(s, a) = R(s, a) + \gamma \sum_{s' \in S} P(s'|s, a) V^{\pi_i}(s')$$

$$\max_a Q^{\pi_i}(s, a) \geq V^{\pi_i}(s) = R(s, \pi_i(s)) + \gamma \sum_{s' \in S} P(s'|s, \pi_i(s)) V^{\pi_i}(s')$$

$$\pi_{i+1}(s) = \arg \max_a Q^{\pi_i}(s, a)$$

- Suppose we take $\pi_{i+1}(s)$ for one action, then follow π_i forever
 - Our expected sum of rewards is at least as good as if we had always followed π_i
- But new proposed policy is always to follow $\pi_{i+1}, \dots$

Monotonic Improvement in Policy

- Recall that $\pi_{i+1} \geq \pi_i$ if $V^{\pi_{i+1}}(s) \geq V^{\pi_i}(s), \forall s \in S$.
- Policy Improvement theorem:
 - $\pi_{i+1} \geq \pi_i$ with strict inequality if π_i is suboptimal, where π_{i+1} is the new policy we get from policy improvement on π_i

Proof: Monotonic Improvement in Policy

$$\begin{aligned} V^{\pi_i}(s) &\leq \max_a Q^{\pi_i}(s, a) \\ &= \max_a R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) V^{\pi_i}(s') \\ &= R(s, \pi_{i+1}(s)) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, \pi_{i+1}(s)) V^{\pi_i}(s') \\ &\leq R(s, \pi_{i+1}(s)) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, \pi_{i+1}(s)) (\max_{a'} Q^{\pi_i}(s', a')) \\ &\vdots \\ &= V^{\pi_{i+1}}(s) \end{aligned}$$

Optimal Bellman Equation

- Now we can prove the existence of $V^*(s)$
- Let that $V^*(s) = \max_{\pi} V^{\pi}(s)$ for certain s .
 - Will there be one π^* that works as a maximizer $\forall s$?
- $V^*(s) = \max_{\pi} V^{\pi}(s) = \max_{\pi} \mathbb{E} [\sum_{t \geq 0} \gamma^t R(s_t, \pi(s_t)) | s_0 = s, \pi]$

$$= \max_{a, \pi'} \left[R(s, a) + \gamma \sum_{s'} P(s' | s, a) V^{\pi'}(s') \right]$$

$$= \max_a \left[R(s, a) + \gamma \sum_{s'} P(s' | s, a) \max_{\pi'} V^{\pi'}(s') \right]$$

$$= \max_a \left[R(s, a) + \gamma \sum_{s'} P(s' | s, a) V^*(s') \right]$$

Optimal Bellman Equation

- So we have

$$V^*(s) = \max_a \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^*(s') \right]$$

- If this has a unique solution, we can pick a maximizer a in state s
 - So we have an optimal policy!
- Unlike Bellman equation, it cannot be solved with matrix operations...

Value Iteration

- Another technique to compute optimal value/policy
 - Use Bellman optimality equation
 - Maintains optimal value of finite horizon, and iterate to consider longer and longer episodes
- Set $k = 1$
- Initialize $V_0(s)$ for all states s
- Loop until convergence:
 - $V_{k+1}(s) = \max_a R(s, a) + \gamma \sum_{s' \in S} P(s'|s, a) V_k(s'), \forall s \in S$
 - $\pi_{k+1}(s) = \arg \max_a R(s, a) + \gamma \sum_{s' \in S} P(s'|s, a) V_k(s'), \forall s \in S$

Also note that this is $Q_k(s, a)$

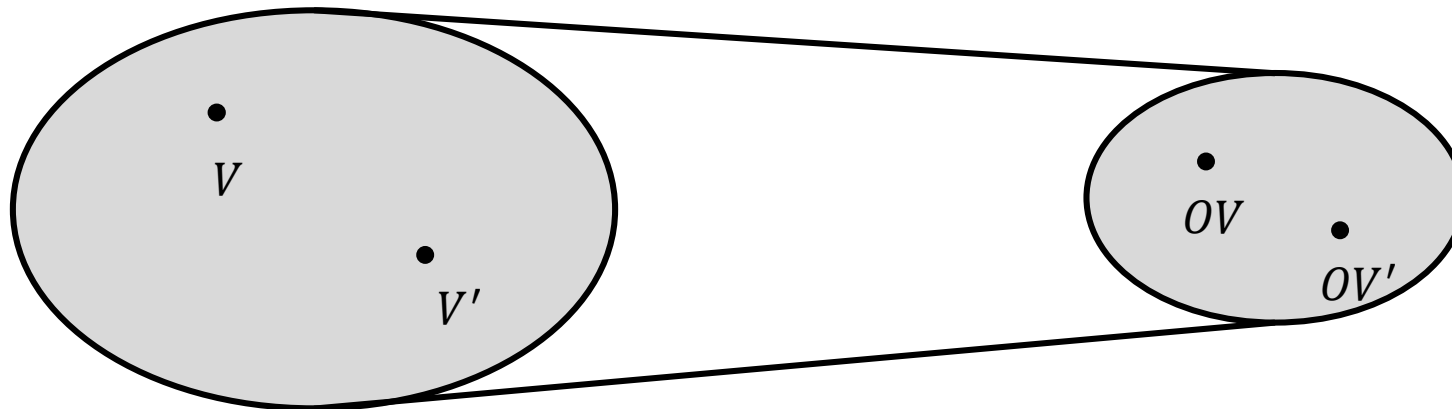


Iteration as Bellman Backup operators

- Bellman backup operator: $B: (S \times \mathbb{R}) \rightarrow (S \times \mathbb{R})$
 - Applied to a value function
 - Returns a new value function
- Bellman optimality backup:
 - $(B^*V)(s) = \max_a R(s, a) + \gamma \sum_{s' \in S} P(s'|s, a)V(s'), \forall s \in S$
 - Optimal value function: fixed point of B^*
- Bellman (expectation) backup:
 - $(B^\pi V)(s) = R(s, \pi(s)) + \gamma \sum_{s' \in S} P(s'|s, \pi(s))V(s'), \forall s \in S$
 - Policy evaluation: fixed point of B^π
 - $V^\pi = B^\pi B^\pi \dots B^\pi V$

Contraction Operator

- Let O be an operator, and $|x|$ denote (any) norm of x
- If $|OV - OV'| \leq |V - V'|$ for any V, V' , then O is a contraction operator
- Banach fixed-point theorem:
 - Contraction mapping has unique fixed point, i.e. solution to $V = OV$ is unique.
- Such a fixed point can be obtained by $V^* = OO \dots OV$



Will Value Iteration Converge?

- Yes, if discount factor $\gamma < 1$, or end up in a terminal state (state with no exit and no reward) with probability 1
- Bellman optimality backup is a contraction if discount factor $\gamma < 1$
- If apply it to two different value functions, distance between value functions shrinks after applying Bellman equation to each

Proof

- Let $\|V - V'\| = \max_s |V(s) - V'(s)|$ be the infinity norm

$$\begin{aligned}\|B^*V_1 - B^*V_2\| &= \left\| \max_a \left(R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) V_1(s') \right) - \max_{a'} \left(R(s, a') + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a') V_2(s') \right) \right\| \\ &\leq \max_a \left\| \left(R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) V_1(s') \right) - \left(R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) V_2(s') \right) \right\| \\ &= \max_a \left\| \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) (V_1(s') - V_2(s')) \right\| \\ &\leq \max_a \left\| \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) \|V_1 - V_2\| \right\| = \max_a \left\| \gamma \|V_1 - V_2\| \sum_{s' \in \mathcal{S}} P(s'|s, a) \right\| = \gamma \|V_1 - V_2\|\end{aligned}$$

- Even if all inequalities are equalities, this is still a contraction if $\gamma < 1$

How about Bellman Expectation Backup?

- Can be shown in similar way

$$\begin{aligned} & \|B^\pi V_1 - B^\pi V_2\| \\ &= \left\| \sum_{a \in A} \pi(a|s) \left(R(s, a) + \gamma \sum_{s' \in S} P(s'|s, a) V_1(s') \right) - \sum_{a' \in A} \pi(a'|s) \left(R(s, a') + \gamma \sum_{s' \in S} P(s'|s, a') V_2(s') \right) \right\| \\ &= \left\| \gamma \sum_{a \in A} \pi(a|s) \sum_{s' \in S} P(s'|s, a) (V_1(s') - V_2(s')) \right\| \\ &\leq \left\| \gamma \sum_{a \in A} \pi(a|s) \sum_{s' \in S} P(s'|s, a) \|V_1 - V_2\| \right\| = \left\| \gamma \|V_1 - V_2\| \sum_{a \in A} \pi(a|s) \sum_{s' \in S} P(s'|s, a) \right\| = \gamma \|V_1 - V_2\| \end{aligned}$$

- So Bellman Expectation Backup is contraction as well

Value Iteration Converges

- Under $\gamma < 1$, we know Bellman optimality backup is a contraction mapping
- Then Bellman optimality backup operator has a unique fixed point:
 - Which is optimal value function
- $$V^*(s) = (BV^*)(s) = \max_a R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) V^*(s')$$
$$= \max_{\pi} V^{\pi}(s)$$
- Therefore, even if policies are under partial ordering, there exists a unique optimal value function, and optimal policies that gives optimal value function.

Value Iteration for Finite Horizon

- V_k = optimal value if making k more decisions
 - π_k = optimal policy if making k more decisions
- Initialize $V_0(s) = 0$ for all states s
 - For $k = 1:H$
 - For each state s
 - $V_{k+1}(s) = \max_a R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) V_k(s')$
 - $\pi_{k+1}(s) = \arg \max_a R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) V_k(s')$

Value vs Policy Iteration

- Value iteration:
 - Compute optimal value for horizon = k
 - Note this can be used to compute optimal policy if horizon = k
 - Increment k
 - Closely related to: Q-learning
- Policy iteration:
 - Compute infinite horizon value of a policy
 - Use to select another (better) policy
 - Closely related to: policy gradient / actor-critic

What we covered so far

- Defined MP, MRP, MDP, Bellman (expectation) operator, Bellman optimality operator, contraction, policy, state value (V-value), state-action value (Q-value)
- Proved that Bellman equation and Bellman optimality equation have unique solution, and monotonic improvement of policy
- Two finite MDP planning algorithms: policy evaluation & control when we have true model (of how the world works)
 - Policy Iteration
 - Value Iteration



End of Lecture

강화학습

LG Aimers



고려대학교 인공지능학과 이병준
Lecture 2: Model-Free Policy Evaluation

Recall

- Definition of Return, G_t (for a MRP)

- Discounted sum of rewards from time step t to horizon

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 r_{t+3} + \dots$$

- Definition of State Value Function, $V^\pi(s)$

- Expected return from starting in state s under policy π

$$V^\pi(s) = \mathbb{E}_\pi[G_t | s_t = s] = \mathbb{E}_\pi[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 r_{t+3} + \dots | s_t = s]$$

- Definition of State-Action Value Function, $Q^\pi(s, a)$

- Expected return from starting in state s , taking action a and then following policy π

$$\begin{aligned} Q^\pi(s, a) &= \mathbb{E}_\pi[G_t | s_t = s, a_t = a] \\ &= \mathbb{E}_\pi[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 r_{t+3} + \dots | s_t = s, a_t = a] \end{aligned}$$

This Lecture: Policy Evaluation

- Estimating the expected return of a particular policy if don't have access to true MDP models
 - Monte Carlo policy evaluation
 - Temporal Difference (TD)
 - Certainty Equivalence with dynamic programming
 - Metrics to evaluate and compare algorithms

Monte Carlo (MC) Policy Evaluation

- $G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 r_{t+3} + \dots$ in MDP M under policy π
- $V^\pi(s) = \mathbb{E}_{\tau \sim \pi}[G_t | s_t = s]$
 - Expectation over trajectories τ generated by following π
- Simple idea: Value = mean return
- If trajectories are all finite, sample set of trajectories & average returns

Monte Carlo (MC) Policy Evaluation

- If trajectories are all finite, sample set of trajectories & average returns
- Does not require MDP dynamics/rewards
- Does not assume state is Markov (Markov Property)
- Can **only** be applied to episodic MDPs
(ends up in terminal state with probability 1)
 - Averaging over returns from a complete episode
 - Requires each episode to terminate

Monte Carlo (MC) On Policy Evaluation

- Aim: estimate $V^\pi(s)$ given episodes generated under policy π
 - $(s_1, a_1, r_1), (s_2, a_2, r_2), \dots$ where the actions are sampled from π
- $G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 r_{t+3} + \dots$ in MDP M under policy π
- $V^\pi(s) = \mathbb{E}_\pi[G_t | s_t = s]$
- MC computes empirical mean return
- Often do this in an incremental fashion
 - After each episode, update estimate of V^π

First-Visit Monte Carlo (MC) On Policy Evaluation

- Initialize $N(s) = 0, G(s) = 0 \quad \forall s \in S$
- Loop
 - Sample episode $i = (s_{i,1}, a_{i,1}, r_{i,1}), (s_{i,2}, a_{i,2}, r_{i,2}), \dots, s_{i,T_i}$
 - Define $G_{i,t} = r_{i,t} + \gamma r_{i,t+1} + \gamma^2 r_{i,t+2} + \dots + \gamma^{T_i-t} r_{i,T_i}$ as return from time step t onwards in i^{th} episode
 - For each time step t till the end of the episode i
 - If this is the first time t that state s is visited in episode i
 - Increment counter of total first visits: $N(s) = N(s) + 1$
 - Increment total return $G(s) = G(s) + G_{i,t}$
 - Update estimate $V^\pi(s) = G(s)/N(s)$

Evaluate the Quality of a Policy Estimation Approach: Bias, Variance and MSE

- Consider a statistical model that is parameterized by θ and that determines a probability distribution over observed data $P(x|\theta)$
- Consider a statistic $\hat{\theta}$ that provides an estimate of θ and is a function of observed data x
 - E.g. for a Gaussian distribution with known variance, the average of a set of i.i.d data points is an estimate of the mean of the Gaussian
- Definition: the bias of an estimator $\hat{\theta}$ is:

$$Bias_{\theta}(\hat{\theta}) = \mathbb{E}_{x|\theta}[\hat{\theta}] - \theta$$

- Definition: the variance of an estimator $\hat{\theta}$ is:

$$Var(\hat{\theta}) = \mathbb{E}_{x|\theta} [(\hat{\theta} - \mathbb{E}[\hat{\theta}])^2]$$

- Definition: mean squared error (MSE) of an estimator $\hat{\theta}$ is:

$$MSE(\hat{\theta}) = Var(\hat{\theta}) + Bias_{\theta}(\hat{\theta})^2$$

First-Visit Monte Carlo (MC) On Policy Evaluation

- Initialize $N(s) = 0, G(s) = 0 \quad \forall s \in S$
 - Loop
 - Sample episode $i = (s_{i,1}, a_{i,1}, r_{i,1}), (s_{i,2}, a_{i,2}, r_{i,2}), \dots, s_{i,T_i}$
 - Define $G_{i,t} = r_{i,t} + \gamma r_{i,t+1} + \gamma^2 r_{i,t+2} + \dots + \gamma^{T_i-t} r_{i,T_i}$ as return from time step t onwards in i^{th} episode
 - For each time step t till the end of the episode i
 - If this is the first time t that state s is visited in episode i
 - Increment counter of total first visits: $N(s) = N(s) + 1$
 - Increment total return $G(s) = G(s) + G_{i,t}$
 - Update estimate $V^\pi(s) = G(s)/N(s)$
- V^π estimator is an unbiased estimator of true $\mathbb{E}_\pi[G_t | s_t = s]$
 - By law of large numbers, as $N(s) \rightarrow \infty, V^\pi(s) \rightarrow \mathbb{E}_\pi[G_t | s_t = s]$

Every-Visit Monte Carlo (MC) On Policy Evaluation

- Initialize $N(s) = 0, G(s) = 0 \quad \forall s \in S$
 - Loop
 - Sample episode $i = (s_{i,1}, a_{i,1}, r_{i,1}), (s_{i,2}, a_{i,2}, r_{i,2}), \dots, s_{i,T_i}$
 - Define $G_{i,t} = r_{i,t} + \gamma r_{i,t+1} + \gamma^2 r_{i,t+2} + \dots + \gamma^{T_i-t} r_{i,T_i}$ as return from time step t onwards in i^{th} episode
 - For each time step t till the end of the episode i
 - state s is the state visited at time step t in episode i
 - Increment counter of total first visits: $N(s) = N(s) + 1$
 - Increment total return $G(s) = G(s) + G_{i,t}$
 - Update estimate $V^\pi(s) = G(s)/N(s)$
- V^π every-visit MC estimator is a biased estimator of V^π
 - But consistent estimator and often has better MSE

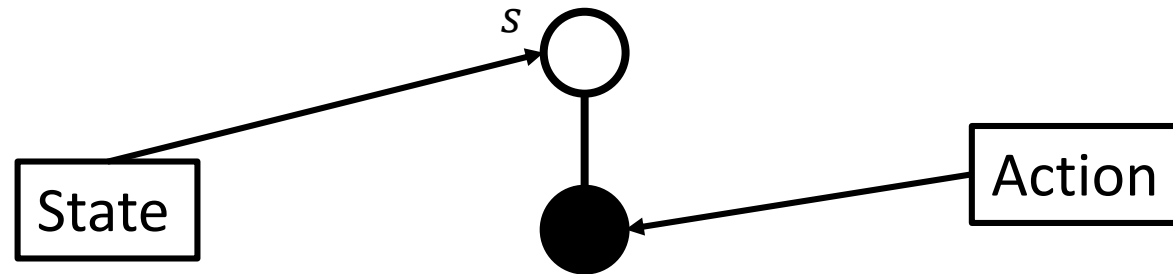
Incremental Monte Carlo (MC) On Policy Evaluation

- After each episode $i = (s_{i,1}, a_{i,1}, r_{i,1}), (s_{i,2}, a_{i,2}, r_{i,2}), \dots$
- Define $G_{i,t} = r_{i,t} + \gamma r_{i,t+1} + \gamma^2 r_{i,t+2} + \dots$ as return from time step t onwards in i^{th} episode
- For state s visited at time step t in episode i
 - Increment counter of total visits: $N(s) = N(s) + 1$
 - Update estimate
 - $V^\pi(s) = V^\pi(s) \frac{N(s)-1}{N(s)} + \frac{G_{i,t}}{N(s)} = V^\pi(s) + \frac{1}{N(s)} (G_{i,t} - V^\pi(s))$

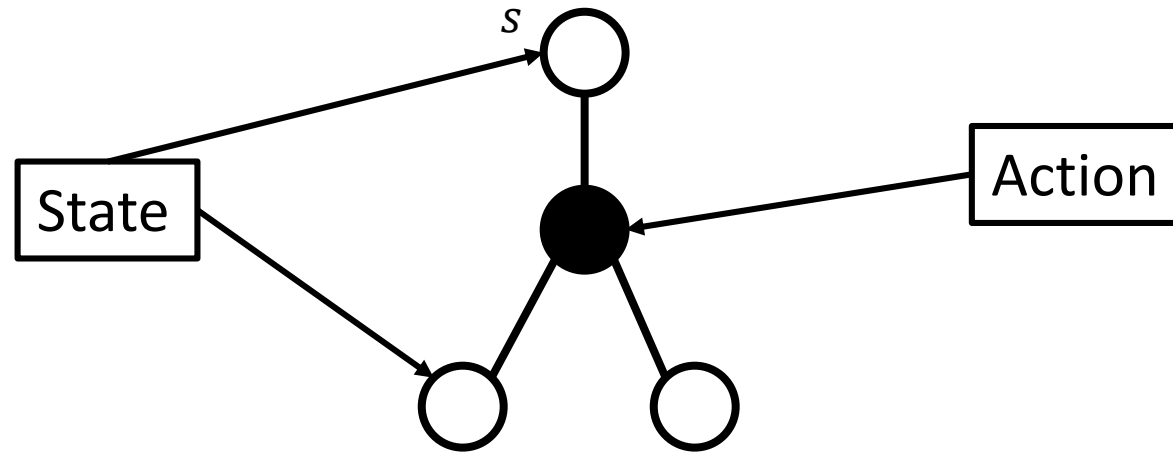
Incremental Monte Carlo (MC) On Policy Evaluation

- Sample episode $i = (s_{i,1}, a_{i,1}, r_{i,1}), (s_{i,2}, a_{i,2}, r_{i,2}), \dots, s_{i,T_i}$
- $G_{i,t} = r_{i,t} + \gamma r_{i,t+1} + \gamma^2 r_{i,t+2} + \dots + \gamma^{T_i-t} r_{i,T_i}$
- For $t = 1:T_i$ where T_i is the length of the i^{th} episode
 - $V^\pi(s_{i,t}) = V^\pi(s_{i,t}) + \alpha (G_{i,t} - V^\pi(s_{i,t}))$

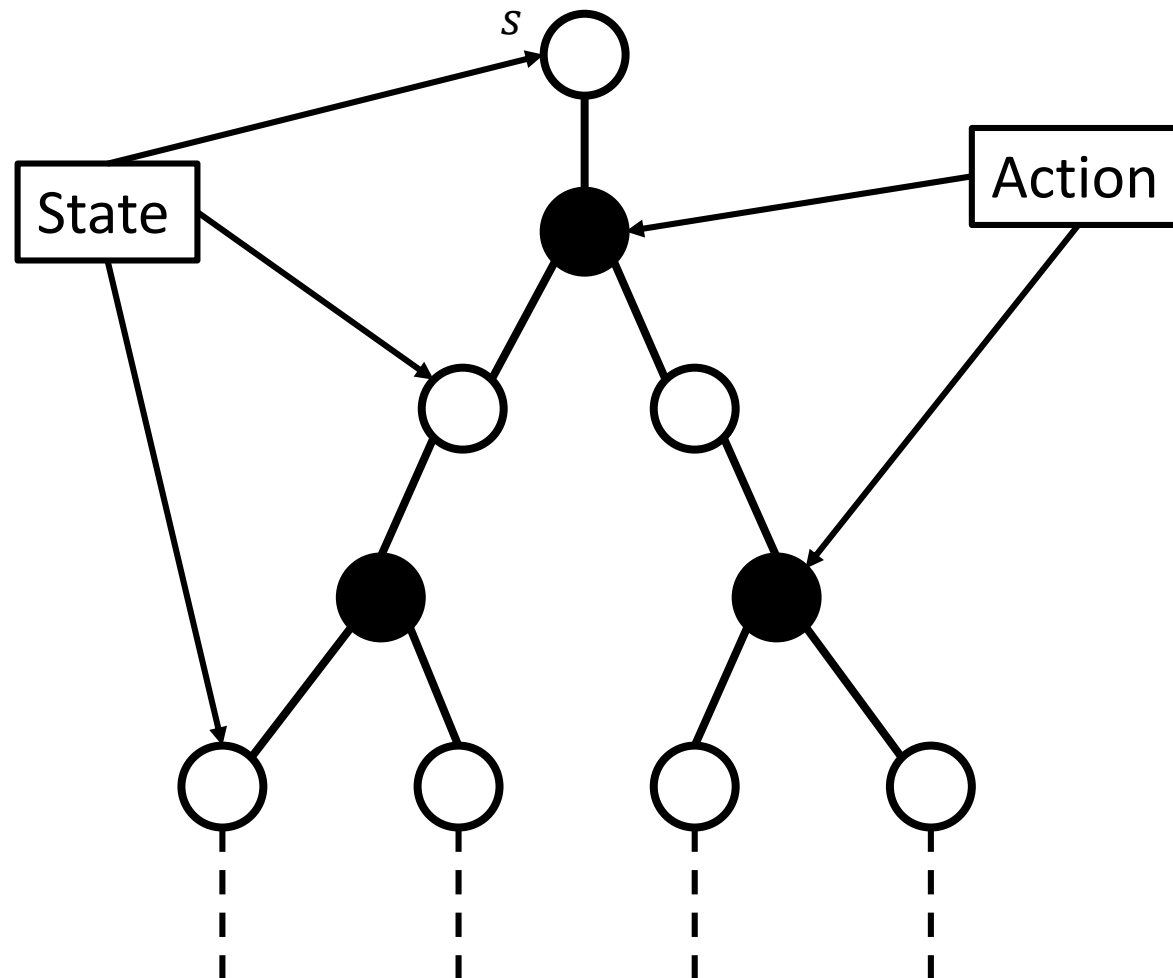
Policy Evaluation Diagram



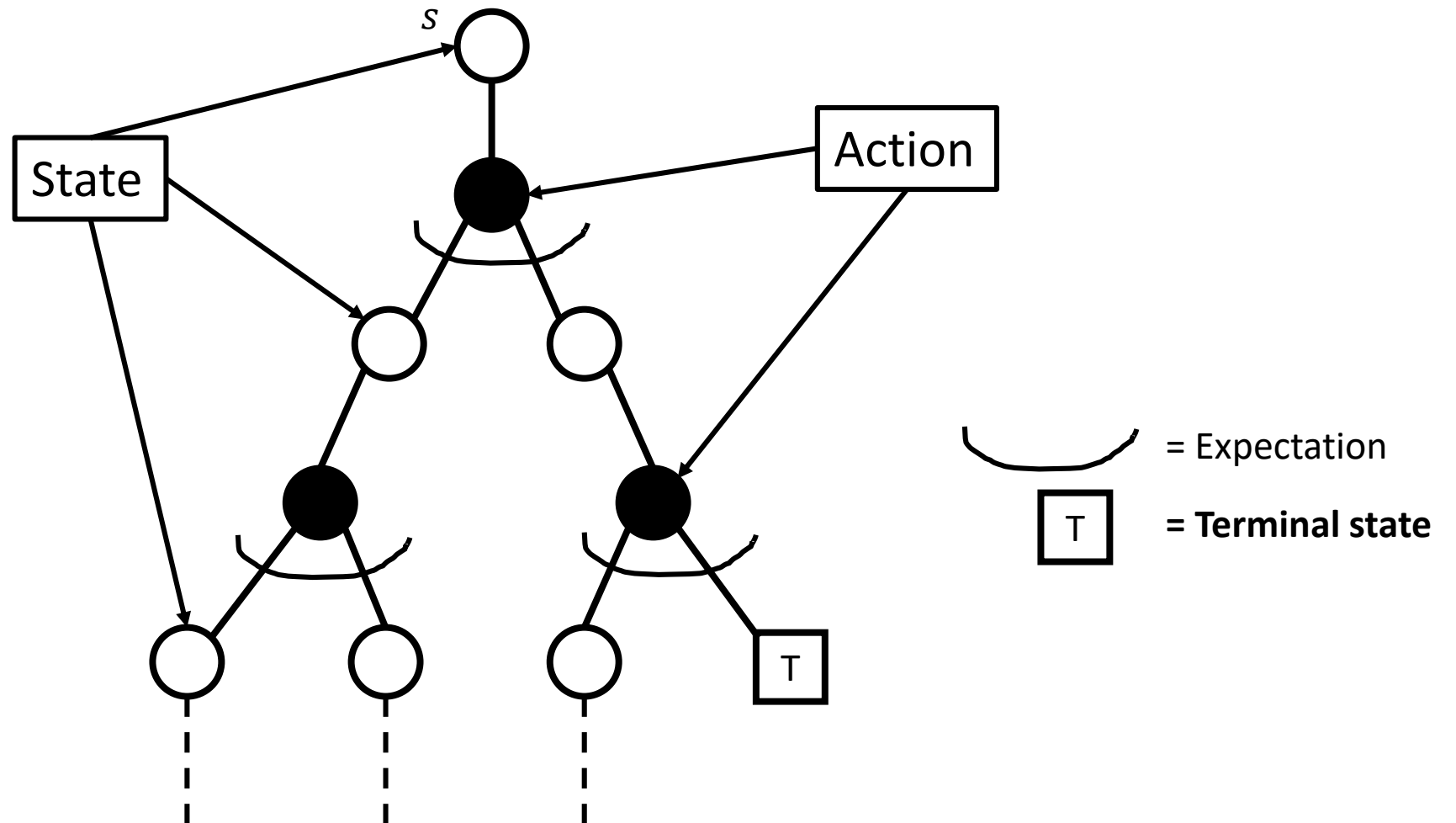
Policy Evaluation Diagram



Policy Evaluation Diagram

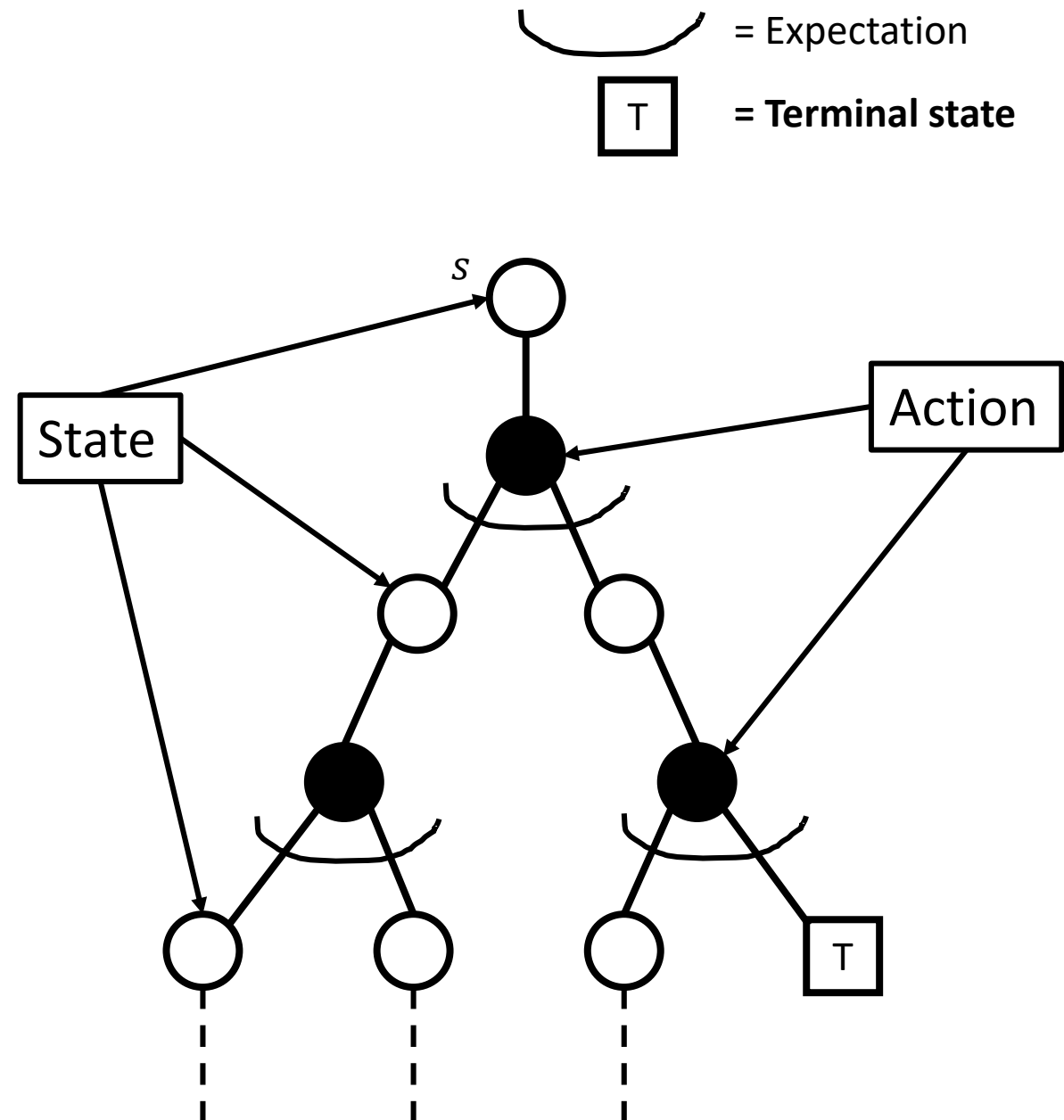


Policy Evaluation Diagram



Policy Evaluation Diagram

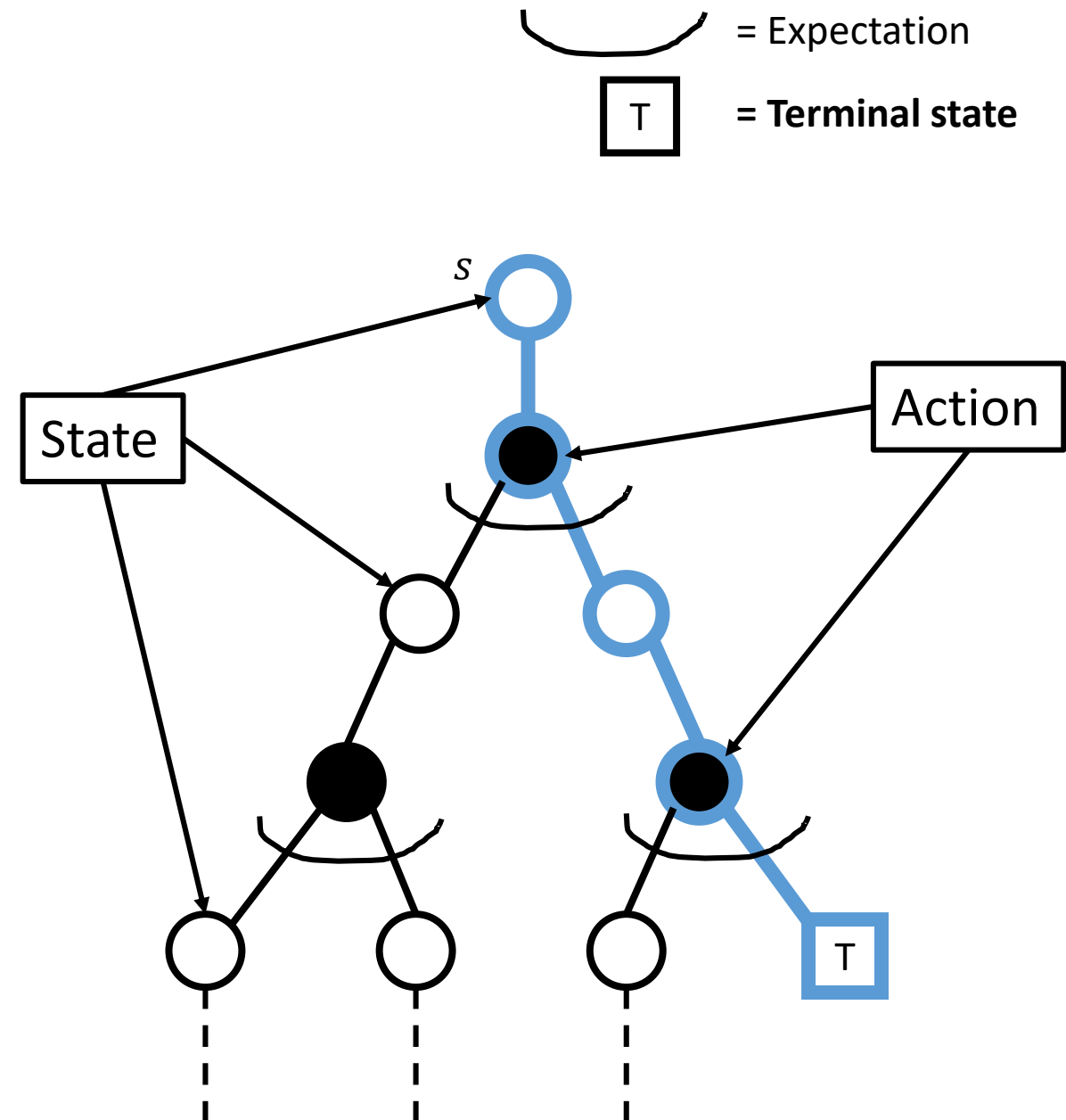
$$V^\pi(s) = V^\pi(s) + \alpha \left(G_{i,t} - V^\pi(s) \right)$$



Policy Evaluation Diagram

$$V^\pi(s) = V^\pi(s) + \alpha \left(G_{i,t} - V^\pi(s) \right)$$

MC updates the value estimate using a **sample** of the return to approximate an expectation



Monte Carlo (MC) Policy Evaluation Key Limitations

- Generally high variance estimator
 - Reducing variance can require a lot of data
 - In cases where data is very hard or expensive to acquire, or the stakes are high, MC may be impractical
- Requires episodic settings
 - Episode must end before data from episode can be used to update V

Temporal Difference Learning

- “If one had to identify one idea as central and novel to reinforcement learning, it would undoubtedly be temporal-difference (TD) learning.” – Sutton and Barto 2017
- Combination of Monte Carlo & dynamic programming methods
- Model-free
- Can be used in episodic or infinite-horizon non-episodic settings
- Immediately updates estimate of V after each (s, a, r, s') tuple

Temporal Difference Learning for Estimating V

- Aim: estimate $V^\pi(s)$ given episodes generated under policy π
 - $G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 r_{t+3} + \dots$ in MDP M under policy π
 - $V^\pi(s) = \mathbb{E}_\pi[G_t | s_t = s]$
- Recall Bellman operator (if know MDP models)
 - $B^\pi V(s) = R(s, \pi(s)) + \gamma \sum_{s' \in \mathcal{S}} p(s' | s, \pi(s)) V(s')$
- In incremental every-visit MC, update estimating using 1 sample of return (for the current i^{th} episode)
 - $V^\pi(s) = V^\pi(s) + \alpha(G_{i,t} - V^\pi(s))$
- Insight: have an estimate of V^π , use to estimate expected return
 - $V^\pi(s) = V^\pi(s) + \alpha([r_t + \gamma V^\pi(s_{t+1})] - V^\pi(s))$

Temporal Difference [$TD(0)$] Learning

- Aim: estimate $V^\pi(s)$ given episodes generated under policy π
 - $(s_1, a_1, r_1), (s_2, a_2, r_2), \dots$ where the actions are sampled from π
- Simplest TD learning: $TD(0)$: update value towards estimated return $r_t + \gamma V^\pi(s_{t+1})$
 - $$V^\pi(s_t) = V^\pi(s_t) + \alpha \left(\underbrace{[r_t + \gamma V^\pi(s_{t+1})]}_{\text{TD target}} - V^\pi(s_t) \right)$$
- TD error:
 - $\delta_t = r_t + \gamma V^\pi(s_{t+1}) - V^\pi(s_t)$
- Can immediately update value estimate after (s, a, r, s') tuple
- Don't need episodic setting

Temporal Difference [$TD(0)$] Learning

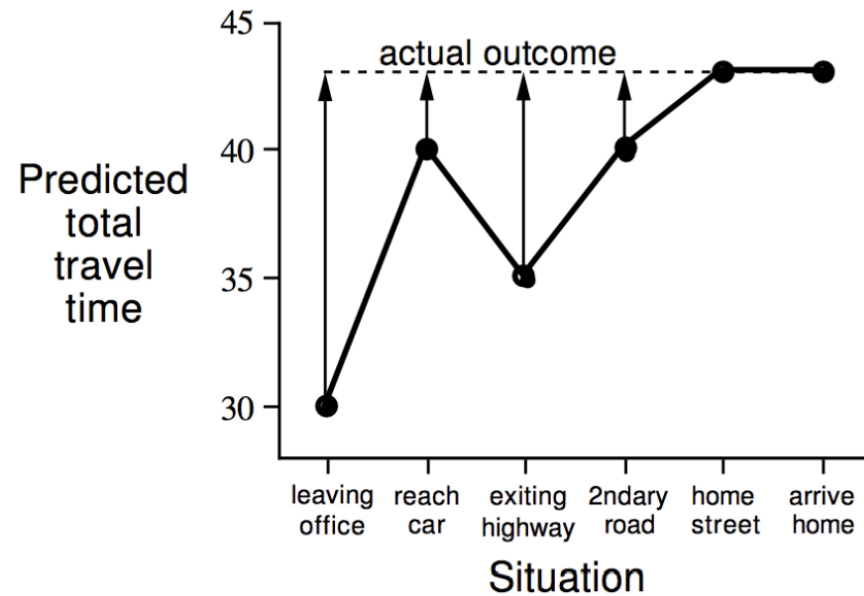
- Input: α
- Initialize $V^\pi(s) = 0, \forall s \in S$
- Loop
 - Sample tuple (s_t, a_t, r_t, s_{t+1})
 - $V^\pi(s_t) = V^\pi(s_t) + \alpha([r_t + \gamma V^\pi(s_{t+1})] - V^\pi(s_t))$

Driving Home Example

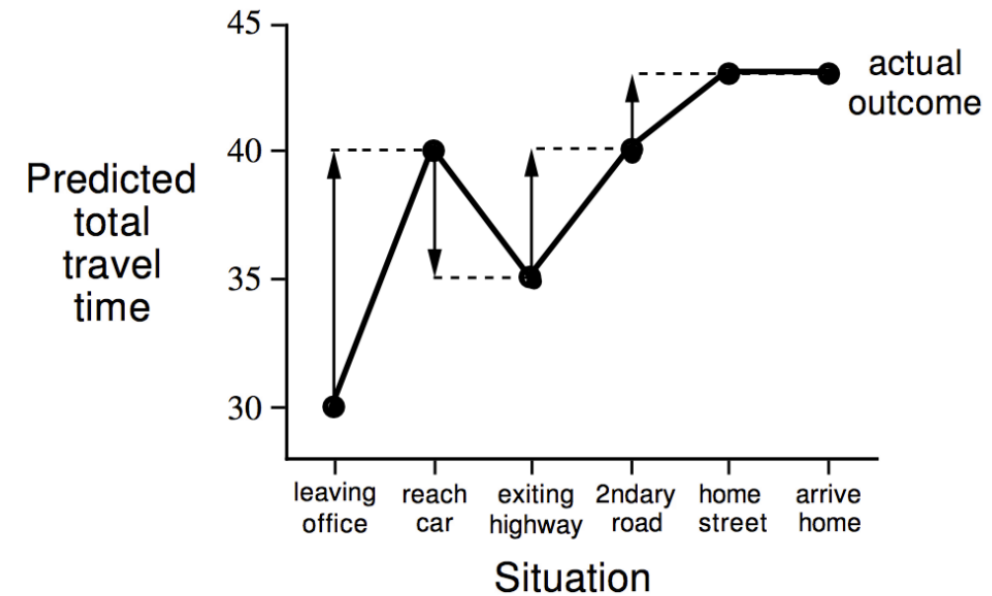
State	Elapsed Time (minutes)	Predicted Time to Go	Predicted Total Time
leaving office	0	30	30
reach car, raining	5	35	40
exit highway	20	15	35
behind truck	30	10	40
home street	40	3	43
arrive home	43	0	43

Driving Home Example: MC vs. TD

Changes recommended by
Monte Carlo methods ($\alpha=1$)



Changes recommended
by TD methods ($\alpha=1$)



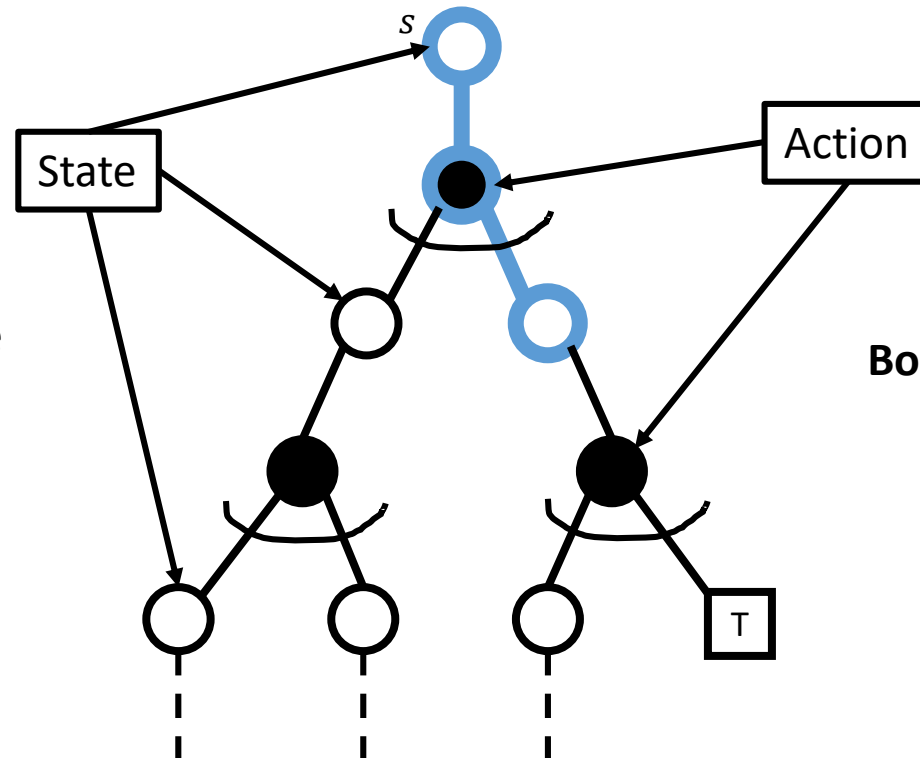
Basic advantages and disadvantages

- TD can learn **before** knowing the final outcome
 - TD can learn online after every step
 - MC must wait until end of episode before return is known
- TD can learn **without** the final outcome
 - TD can learn from incomplete sequences
 - MC can only learn from complete sequences
 - TD works in continuing (non-terminating) environments
 - MC only works for episodic (terminating) environments

Temporal Difference (TD) Policy Evaluation

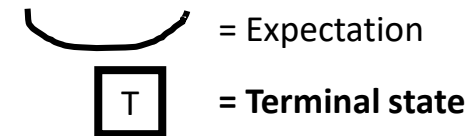
- $V^\pi(s_t) = R(s_t, \pi(s_t)) + \gamma \sum_{s_{t+1}} P(s_{t+1}|s_t, \pi(s_t))V^\pi(s_{t+1})$
- $V^\pi(s_t) = V^\pi(s_t) + \alpha([r_t + \gamma V^\pi(s_{t+1})] - V^\pi(s_t))$

TD updates the value estimate using a **sample** of s_{t+1} to approximate an expectation



TD updates the value estimate by **bootstrapping**, uses estimate of $V(s_{t+1})$

Bootstrapping: Update for V uses an estimate



Summary: Temporal Difference Learning

- Combination of Monte Carlo & dynamic programming methods
- Model-free
- **Bootstraps and samples**
- Can be used in episodic or infinite-horizon non-episodic settings
- Immediately updates estimate of V after each (s, a, r, s') tuple

Recall: Dynamic Programming for Policy Evaluation

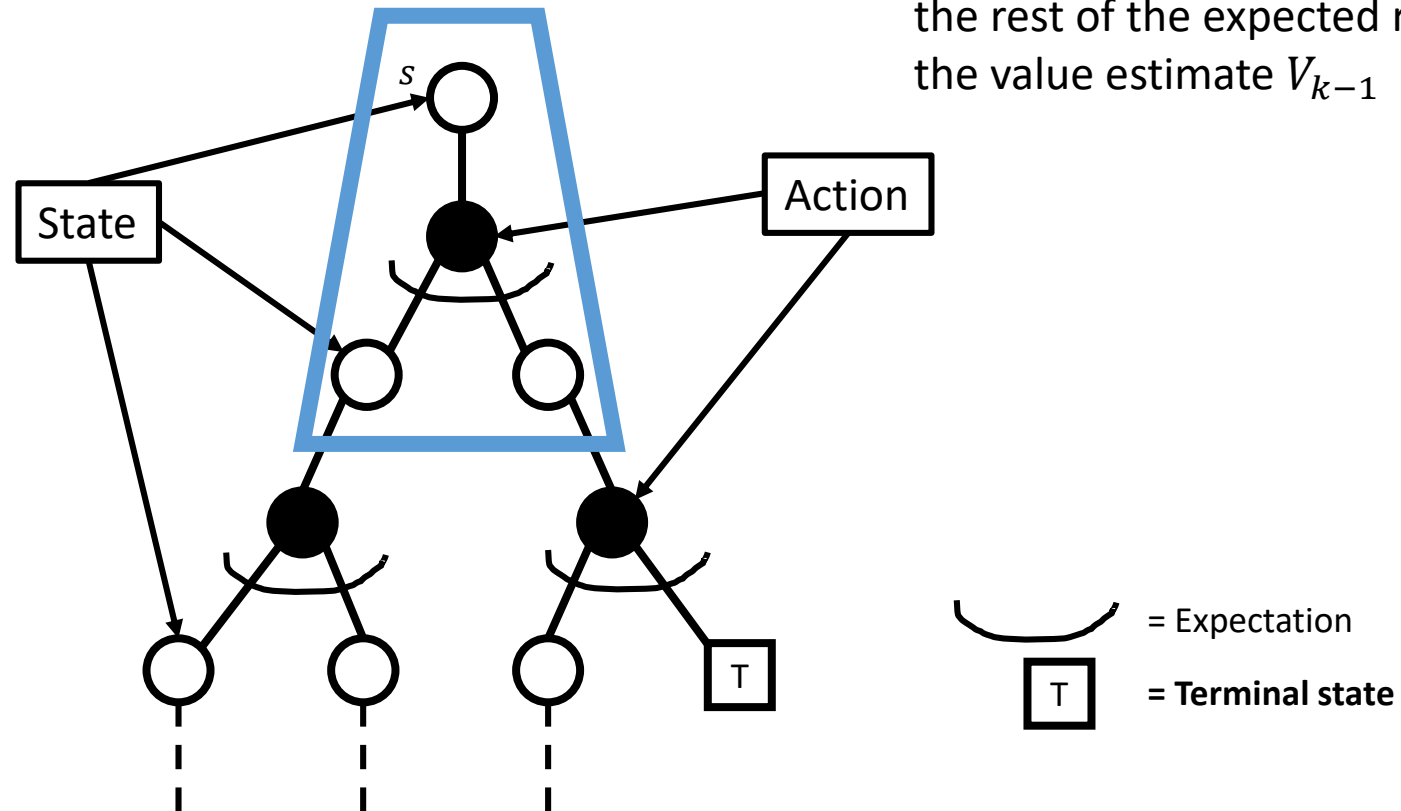
- If we knew dynamics and reward model, we can do policy evaluation (with DP)
- Initialize $V_0^\pi(s) = 0, \forall s \in S$
- For $k = 1$ until convergence
 - $V_k^\pi(s) = R(s, \pi(s)) + \gamma \sum_{s' \in S} P(s'|s, \pi(s)) V_{k-1}^\pi(s'), \forall s$
- $V_k^\pi(s)$ is exactly the k -horizon value of state s under policy π
- $V_k^\pi(s)$ is an **estimate of the infinite horizon** value of state s under policy π
 - $V^\pi(s) = \mathbb{E}_\pi[G_t | s_t = s] \approx \mathbb{E}_\pi[r_t + \gamma V_{k-1} | s_t = s]$

Dynamic Programming Policy Evaluation

- $V^\pi(s) \leftarrow \mathbb{E}_\pi[r_t + \gamma V_{k-1} | s_t = s]$

DP computes this, **bootstrapping** the rest of the expected return by the value estimate V_{k-1}

Know model $P(s'|s, a)$:
reward and expectation over next states computed exactly



What about when we don't know the models?

- Certainty Equivalent Learning:
 - Estimate model based on current samples, run the algorithm based on the model
 - Most basic model-based option for policy evaluation without true models

- After each (s, a, r, s') tuple
 - Recompute maximum likelihood MDP for (s, a)
 - $\hat{P}(s'|s, a) = \frac{1}{N(s, a)} \sum_{k=1}^K \sum_{t=1}^{L_k-1} \mathbb{I}(s_{k,t} = s, a_{k,t} = a, s_{k,t+1} = s')$
 - $\hat{R}(s, a) = \frac{1}{N(s, a)} \sum_{k=1}^K \sum_{t=1}^{L_k-1} \mathbb{I}(s_{k,t} = s, a_{k,t} = a) r_{k,t}$
 - Compute V^π using MLE MDP (using any method from last lecture)

Certainty Equivalent Learning

- After each (s, a, r, s') tuple
 - Recompute maximum likelihood MDP for (s, a)
 - $\hat{P}(s'|s, a) = \frac{1}{N(s,a)} \sum_{k=1}^K \sum_{t=1}^{L_k-1} \mathbb{I}(s_{k,t} = s, a_{k,t} = a, s_{k,t+1} = s')$
 - $\hat{R}(s, a) = \frac{1}{N(s,a)} \sum_{k=1}^K \sum_{t=1}^{L_k-1} \mathbb{I}(s_{k,t} = s, a_{k,t} = a) r_{k,t}$
 - Compute V^π using MLE MDP (using any method from last lecture)
- Cost: Updating MLE model and MDP planning at each update ($O(|S|^3)$ for analytic matrix solution, $O(|S|^2|A|)$ for iterative methods)
- Very data efficient and very computationally expensive
- Consistent (will converge to right estimate for Markov models)
- Can also easily be used for off-policy evaluation

Properties of Algorithms for Evaluation

	DPCE	MC	TD
Can use without access to true MDP models	✓	✓	✓
Usable in continuing (non-episodic) setting	✓		✓
Assumes Markov Process	✓		✓
Converges to true value in limit (consistency)	✓	✓	✓
Unbiased estimate of value		✓	

- DPCE = Dynamic Programming with certainty equivalence estimates
- MC = Monte Carlo
- TD = Temporal Difference
- Here, we are only discussing with exact (tabular) value functions.

Some Important Properties to Evaluate

- Bias/variance characteristics
- Data efficiency
- Computational efficiency
- Mostly focus on comparing MC and TD methods; but we will connect back to DPCE methods later

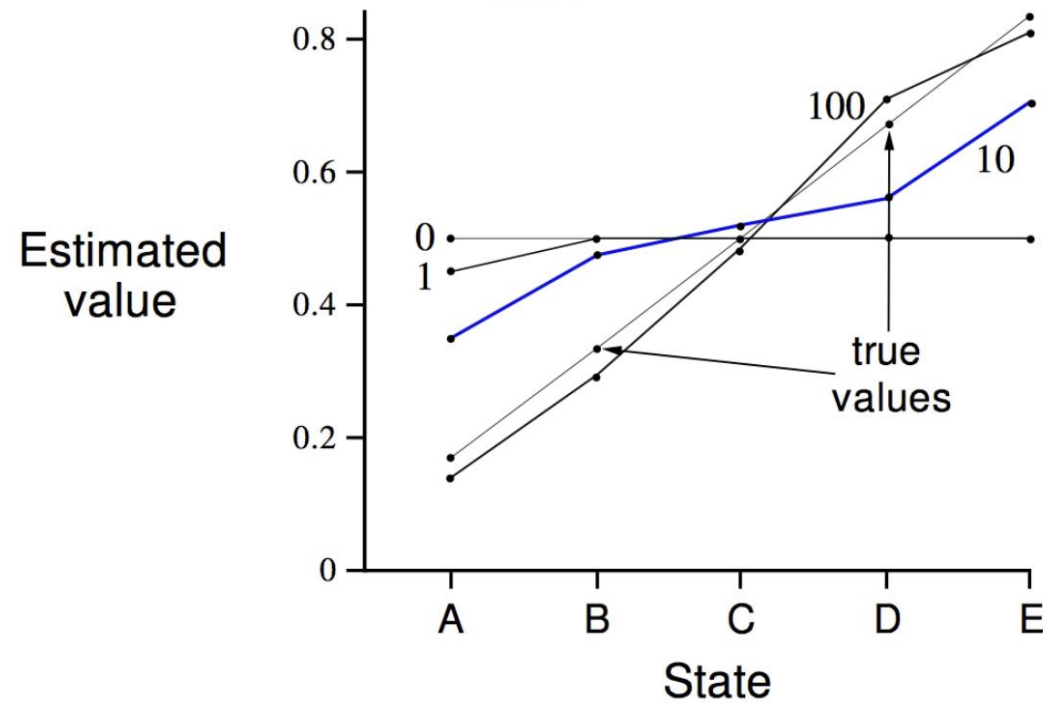
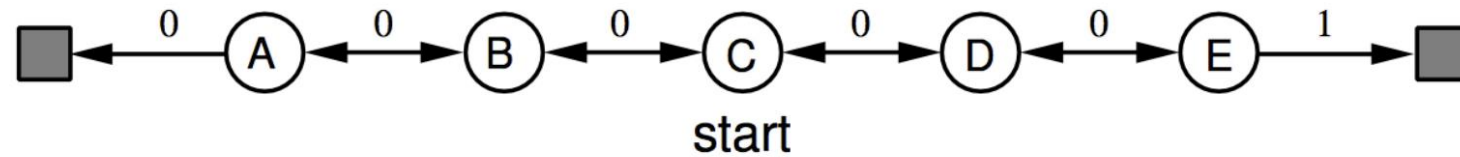
Bias/Variance of MF Evaluation Algorithms

- Return $G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 r_{t+3} + \dots$ is an **unbiased** estimate of $V^\pi(s_t)$
 - Function of multi-step sequence of random actions, states & rewards
- TD target $[r_t + \gamma V(s_{t+1})]$ is a **biased** estimate of $V^\pi(s_t)$
 - But much lower variance than a single return G_t
 - Only has one random action, reward and next state
 - Note that true TD target $[r_t + \gamma V^\pi(s_{t+1})]$ is an **unbiased** estimate!

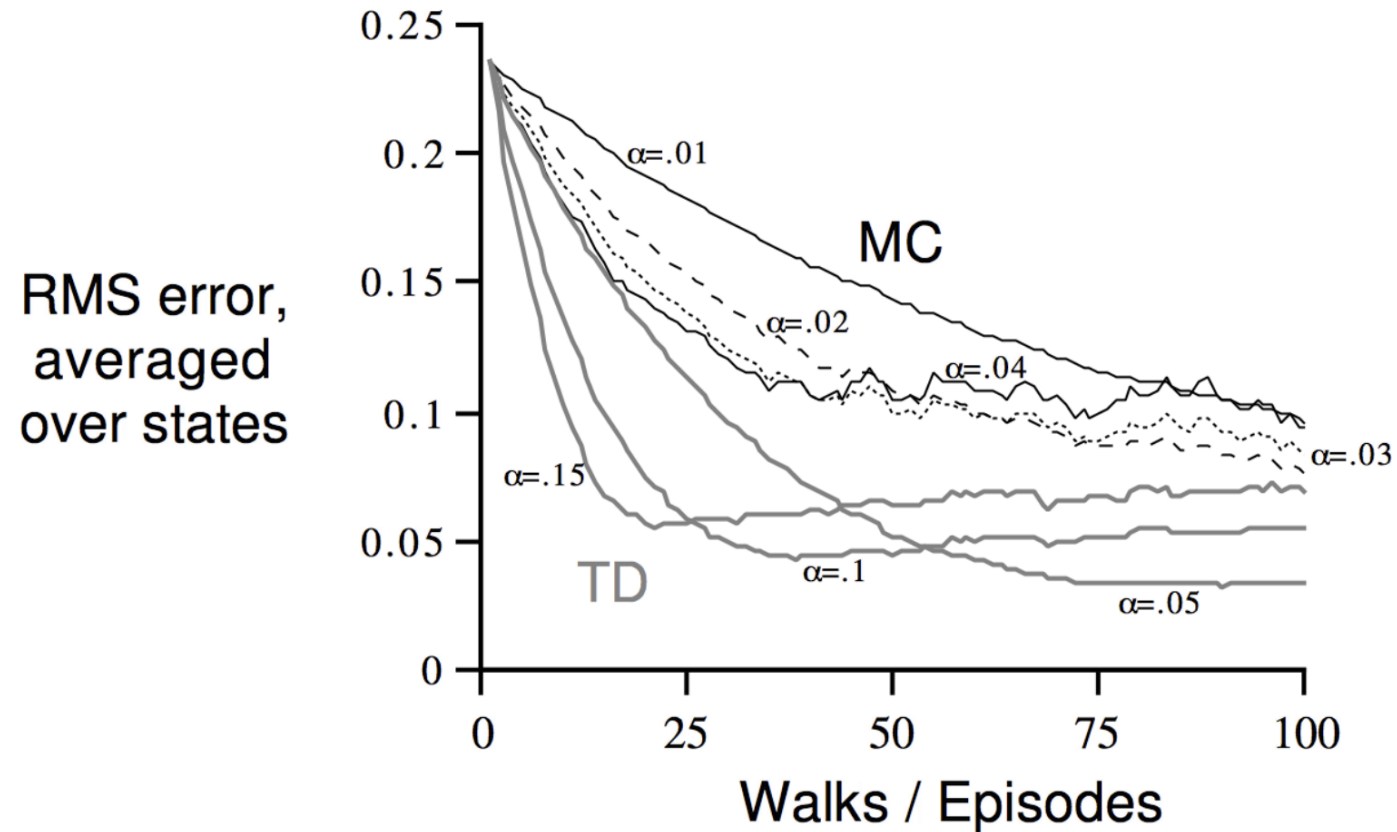
Bias/Variance of MF Evaluation Algorithms

- MC
 - Unbiased (for first visit) / High variance
 - Consistent (converges to true) even with function approximation (good!)
 - Not very sensitive to initial value
 - Very simple to understand and use
- TD
 - Some bias / Lower variance
 - TD(0) converges to true value with tabular representation
 - Does not always converge to true when with function approximation
 - Usually more efficient than MC
 - More sensitive to initial value

Random Walk Example (Ch 6.2, Sutton & Barto, 2018)



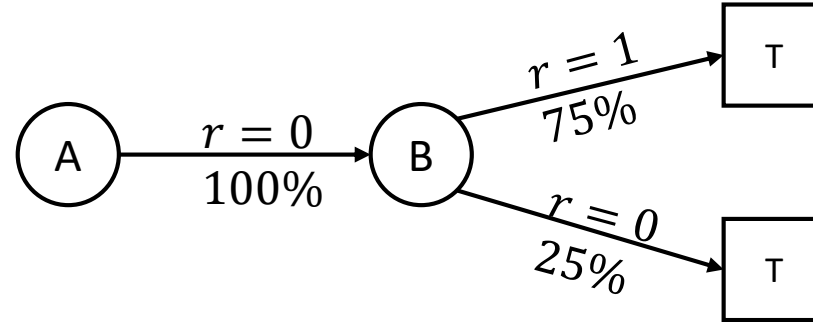
Random Walk Example (Ch 6.2, Sutton & Barto, 2018)



Batch MC and TD

- MC and TD converge: $V(s) \rightarrow V^\pi(s)$ as experience $\rightarrow \infty$
- By what about batch solution for finite experience?
- Batch (offline) solution for finite dataset
 - Given set of K episodes
 - Repeatedly sample an episode from K
 - Apply MC or TD(0) to the sampled episode
- What do MC and TD(0) converge to?

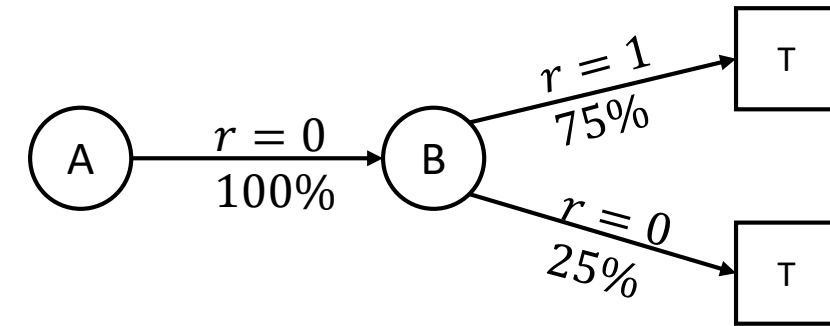
AB Example (Ch. 6.3, Sutton & Barto, 2018)



- Two states A, B with $\gamma = 1$
- Given 8 episodes of experience (random initial state):
 - $(A, 0), (B, 0)$
 - $(B, 1) \times 6$
 - $(B, 0)$
- Imagine run TD updates over data infinite number of times
- $V(B) = ?$

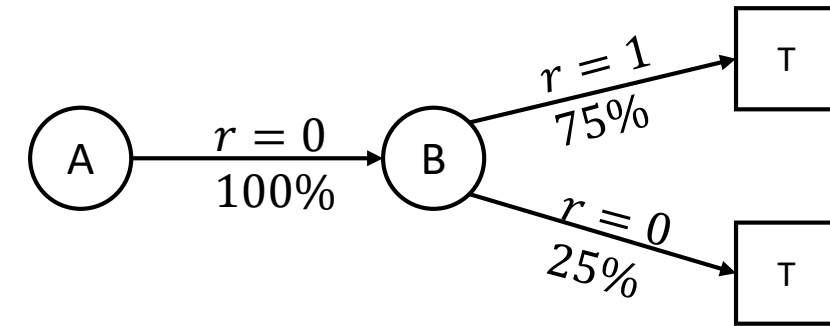
AB Example (Ch. 6.3, Sutton & Barto, 2018)

- $$V^\pi(s_t) = V^\pi(s_t) + \alpha \left(\underbrace{[r_t + \gamma V^\pi(s_{t+1})]}_{\text{TD target}} - V^\pi(s_t) \right)$$
- Two states A, B with $\gamma = 1$
- Given 8 episodes of experience (random initial state):
 - $(A, 0), (B, 0)$
 - $(B, 1) \times 6$
 - $(B, 0)$
- Imagine run TD updates over data infinite number of times
- $V(B) = \frac{6}{8} = 0.75$ by TD or MC
- What about $V(A)$?



AB Example (Ch. 6.3, Sutton & Barto, 2018)

- $$V^\pi(s_t) = V^\pi(s_t) + \alpha \left(\underbrace{[r_t + \gamma V^\pi(s_{t+1})]}_{\text{TD target}} - V^\pi(s_t) \right)$$
- Two states A, B with $\gamma = 1$
- Given 8 episodes of experience (random initial state):
 - $(A, 0), (B, 0)$
 - $(B, 1) \times 6$
 - $(B, 0)$
- Imagine run TD updates over data infinite number of times
- $V(B) = \frac{6}{8} = 0.75$ by TD or MC
- What about $V(A)$? $V^{MC}(A) = 0, V^{TD}(A) = 0.75$



Batch MC and TD: Convergence

- Monte Carlo in batch setting converges to min MSE (mean squared error)
 - Minimize loss with respect to observed returns
 - $\sum_{k=1}^K \sum_{t=1}^{L_k} \left(G_t^k - V(s_t^k) \right)^2$
 - In AB example, $V(A) = 0$
- TD(0) converges to DP policy V^π for the MDP with the maximum likelihood model estimates
 - i.e., same as DPCE!
 - Maximum likelihood Markov decision process model
 - $\hat{P}(s'|s, a) = \frac{1}{N(s, a)} \sum_{k=1}^K \sum_{t=1}^{L_k-1} \mathbb{I}(s_{k,t} = s, a_{k,t} = a, s_{k,t+1} = s')$
 - $\hat{R}(s, a) = \frac{1}{N(s, a)} \sum_{k=1}^K \sum_{t=1}^{L_k-1} \mathbb{I}(s_{k,t} = s, a_{k,t} = a) r_{k,t}$
 - Compute V^π using this model
 - In AB example, $V(A) = 0.75$

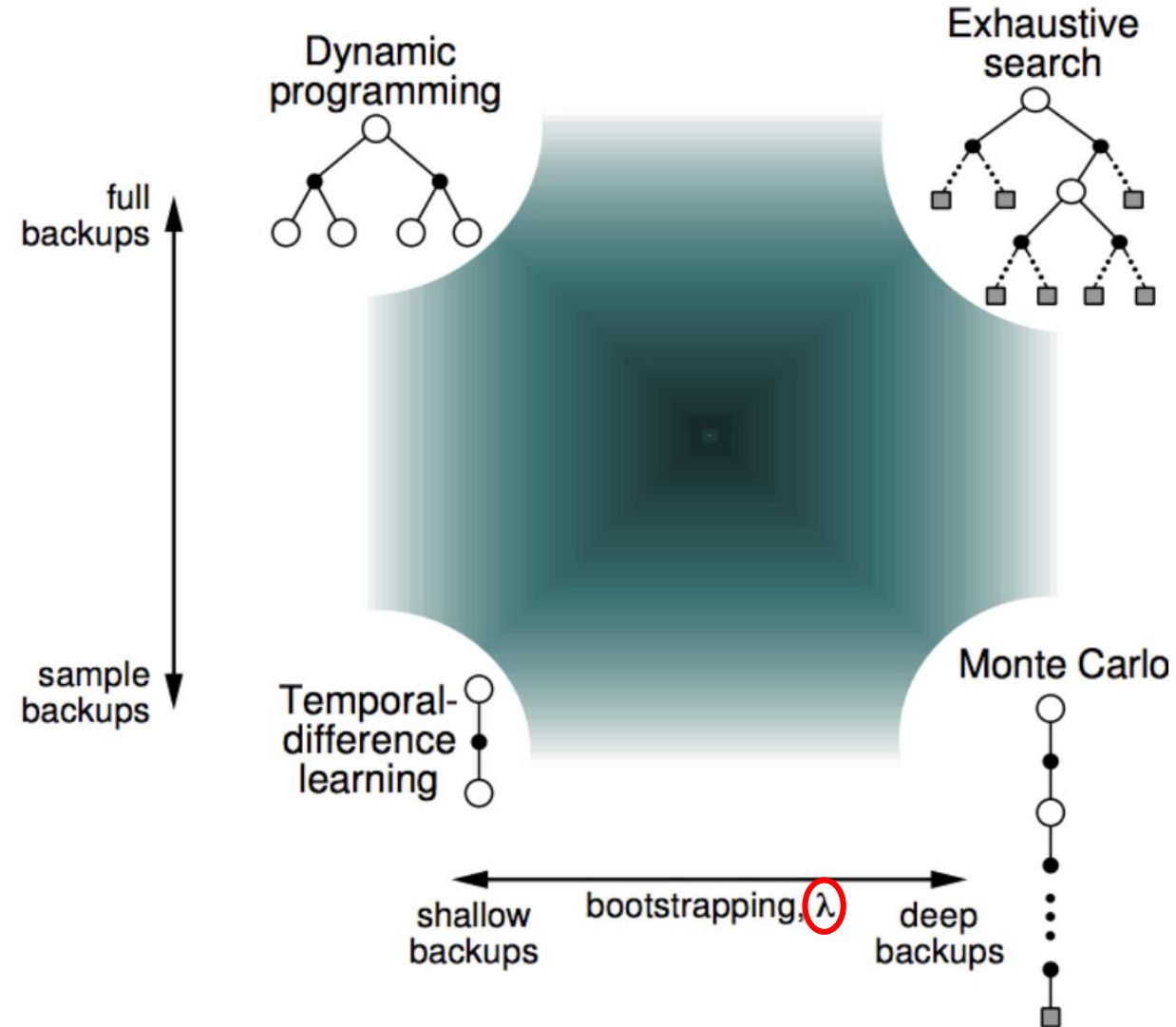
Some Important Properties

- Data efficiency & Computational efficiency
- In simplest TD, use (s, a, r, s') once to update $V(s)$
 - $O(1)$ operation per update
 - In an episode of length L , $O(L)$
- In MC, have to wait until episode finishes, then also $O(L)$
- MC can be more data efficient than simple TD
- But TD exploits Markov structure
 - If in Markov domain, leveraging this is helpful
- Dynamic programming with certainty equivalence also uses Markov structure

Bootstrapping and Sampling

- Bootstrapping: update involves an estimate
 - MC does not bootstrap
 - DP bootstraps
 - TD bootstraps
- Sampling: update samples an expectation
 - MC samples
 - DP does not sample
 - TD samples

Unified View of Reinforcement Learning





Thank you!

강화학습

LG Aimers

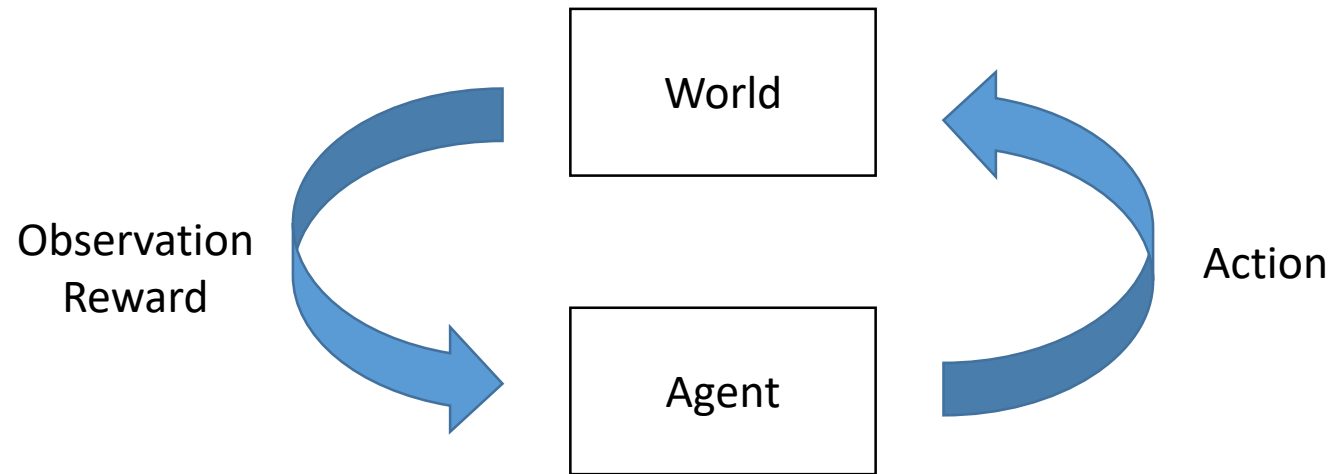


고려대학교 인공지능학과 이병준
Lecture 3: Model-Free Control

This Lecture: Model-free Control

- Last Time:
 - Policy evaluation without known dynamics & reward models
 - ***Estimate*** the value function of certain policy
- Today:
 - Control (making decisions) without a model of how the world works
 - ***Optimize*** the policy based on policy evaluations

Reinforcement Learning



- Goal: Learn to select actions to maximize total expected future reward

Model-free Control Examples

- Many applications can be modeled as a MDP:
 - Elevator
 - Parallel Parking
 - Ship Steering
 - Bioreactor
 - Helicopter
 - Aeroplane Logistics
 - Robocup Soccer
 - Video Games
 - Portfolio Management
 - Protein Folding
 - Robot Walking
 - Game of Go
- For many of these and other problems either:
 - MDP model is unknown but can be sampled
 - MDP model is known but it is computationally infeasible to use directly, except through sampling

Recall Policy Iteration

- Initialize policy π
- Repeat:
 - Policy evaluation: compute V^π
 - Policy improvement: update π

Greedy policy
improvement over $V^\pi(s)$:
Requires model knowledge

Greedy policy
improvement over
 $Q^\pi(s)$:
Does not requires model
knowledge


$$\pi'(s) = \arg \max_a R(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s, a) V^\pi(s') = \arg \max_a Q^\pi(s, a)$$

- Now want to do the above two steps without access to the true dynamics and reward models
- Last lecture introduced methods for model-free policy evaluation

Model Free Policy Iteration

- Initialize policy π
- Repeat:
 - Policy evaluation: compute Q^π
 - Policy improvement: update π

Model Free Generalized Policy Iteration

- Given an estimate $Q^{\pi_i}(s, a) \forall s, a$
- Update new policy

$$\pi_{i+1}(s) = \arg \max_a Q^{\pi_i}(s, a)$$

Example of Greedy Action Selection

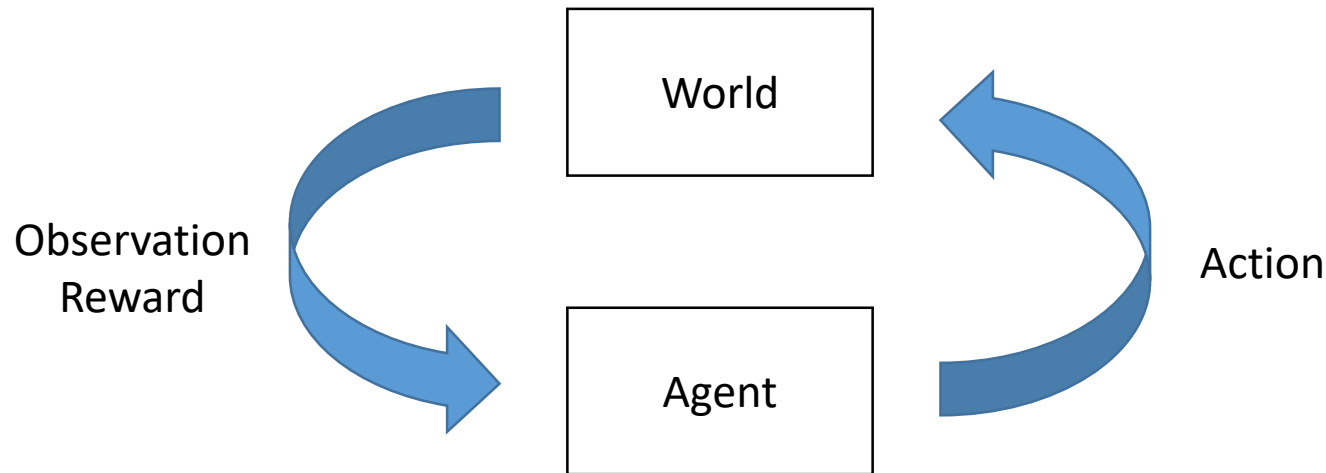


"Behind one door is tenure - behind the other is flipping burgers at McDonald's."

Copyright © 2003 David Farley, d-farley@ibiblio.org

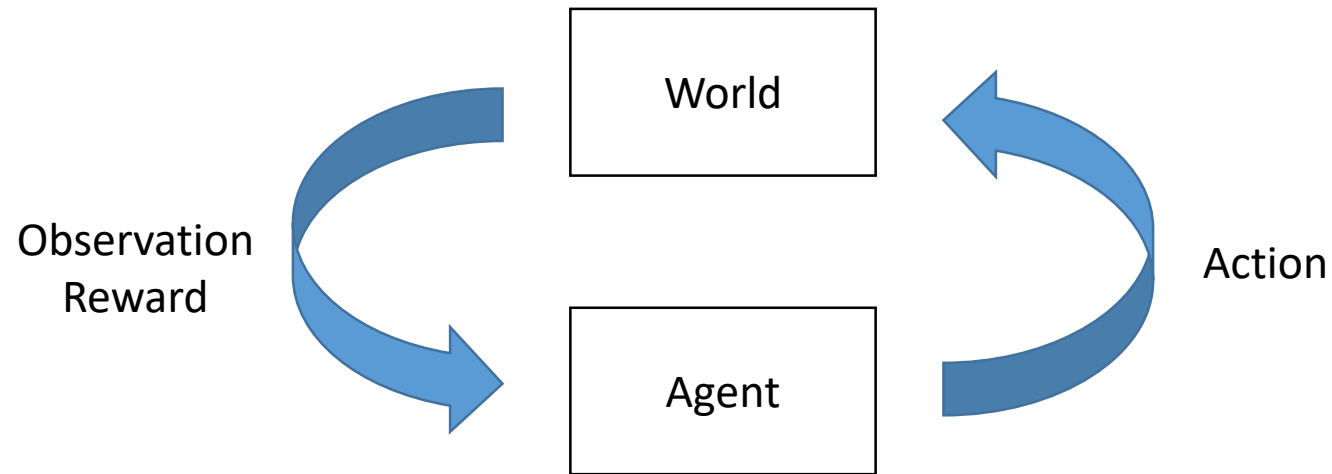
- There are two doors in front of you.
- You open the left door and get reward 0
 $V(\text{left}) = 0$
- You open the right door and get reward +1
 $V(\text{right}) = +1$
- You open the right door and get reward +3
 $V(\text{right}) = +2$
- You open the right door and get reward +2
 $V(\text{right}) = +2$
- ...Are you sure you've chosen the best door?

The Problem of Exploration



- Goal: Learn to select actions to maximize total expected future reward
- Problem: Can't learn about actions without trying them
- Problem: But if we try new actions, we spend less time taking actions that our past experience suggests (high reward expected)

Explore vs. Exploit



- Explore: take actions haven't tried much in a state
- Exploit: take actions estimate will yield high discounted expected reward

Policy Evaluation with Exploration

- Want to compute a model-free estimate of Q^π
- In general seems subtle
 - Need to try all (s, a) pairs but then follow π
 - Want to ensure resulting estimate Q^π is good enough so that policy improvement is a monotonic operator
- For certain classes of policies can ensure all (s, a) pairs are tried such that asymptotically Q^π converges to the true value

ϵ -greedy Policies

- Simple idea to balance exploration and exploitation:
 - all actions are tried with non-zero probability
- Then an ϵ -greedy policy w.r.t. a state-action value $Q(s, a)$ is $\pi(a|s) =$
 - $\arg \max_a Q(s, a)$, w. prob $1 - \epsilon + \frac{\epsilon}{|A|}$
 - $a' \neq \arg \max_a Q(s, a)$, w. prob $\frac{\epsilon}{|A|}$

Theorem

For any ϵ -greedy policy π_i , the ϵ -greedy policy w.r.t. Q^{π_i} , π_{i+1} is a monotonic improvement $V^{\pi_{i+1}} \geq V^{\pi_i}$

Recall Monte Carlo Policy Evaluation, **Now for Q**

1: Initialize $Q(s, a) = 0, N(s, a) = 0, \forall (s, a), k = 1$, Input $\epsilon = 1, \pi$

2: **Loop**

3: Sample k^{th} episode $(s_{k,1}, a_{k,1}, r_{k,1}, s_{k,2}, \dots, s_{k,T})$ given π

4: Compute $G_{k,t} = r_{k,t} + \gamma r_{k,t+1} + \gamma^2 r_{k,t+2} + \dots \gamma^{T-t} r_{k,T} \quad \forall t$

5: **For** $t = 1, \dots, T$ **do**

6: **If** First visit to (s, a) in episode k **then**

7: $N(s, a) = N(s, a) + 1$

8: $Q(s_t, a_t) = Q(s_t, a_t) + \frac{1}{N(s, a)} (G_{k,t} - Q(s_t, a_t))$

9: **End if**

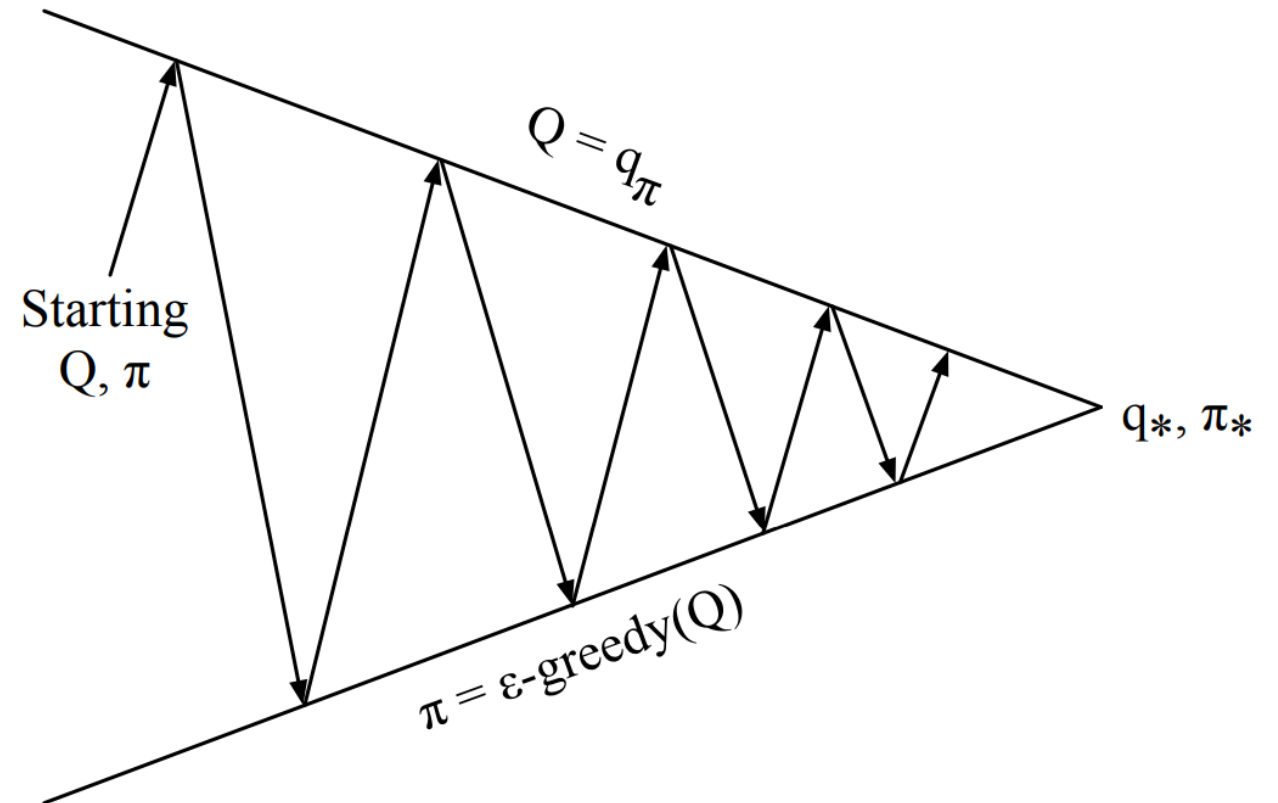
10: **End for**

11: $k = k + 1$

12: **End loop**

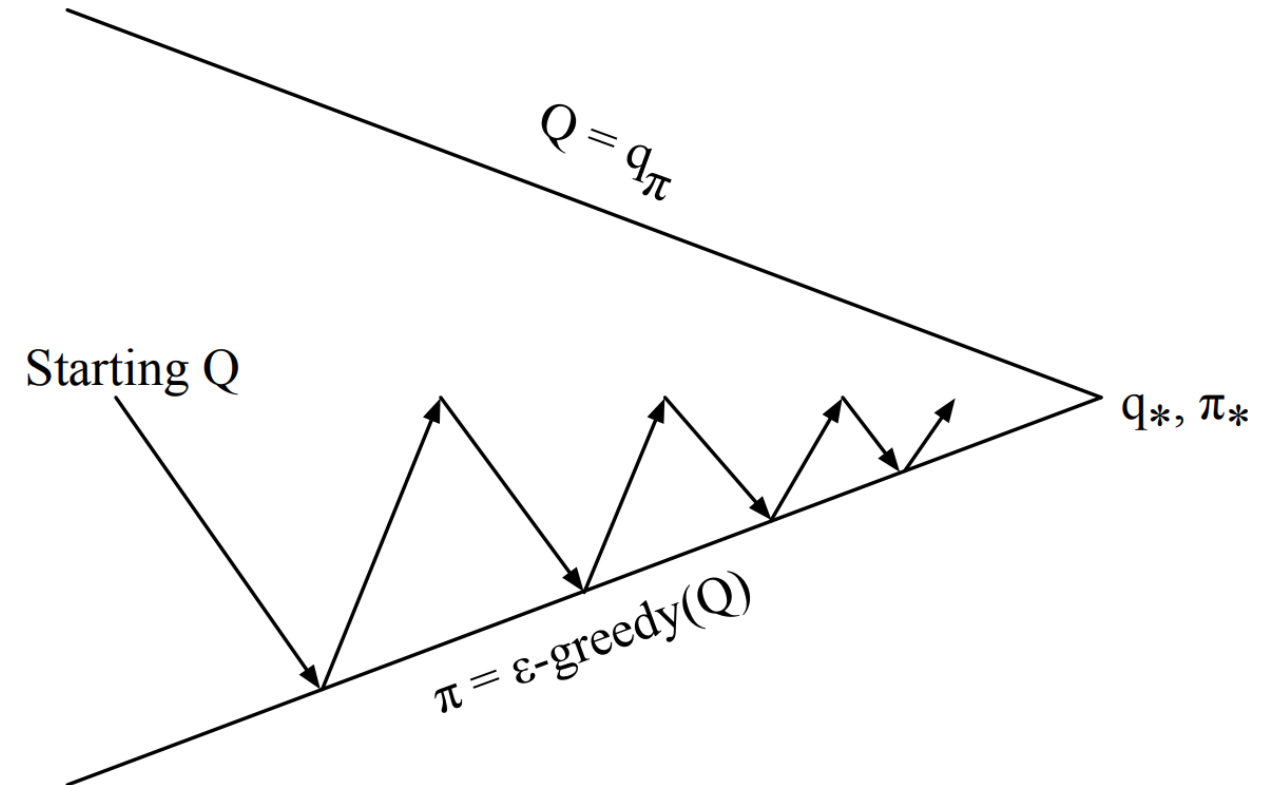
Monte-Carlo Policy Iteration?

- Policy evaluation:
 - Monte-Carlo policy evaluation, $Q = Q^\pi$
- Policy improvement:
 - ϵ -greedy policy improvement
- Takes too much time/samples



Monte-Carlo Control

- Every episode:
 - Policy evaluation:
 - Monte-Carlo policy evaluation
 $Q \approx Q^\pi$
 - Policy improvement:
 - ϵ -greedy policy improvement



Monte Carlo Online Control / On Policy Improvement

1: Initialize $Q(s, a) = 0, N(s, a) = 0, \forall (s, a), k = 1$, Input $\epsilon = 1, \pi$

2: $\pi_k = \epsilon$ -greedy(Q) // *Create initial ϵ -greedy policy*

3: **Loop**

4: Sample k^{th} episode $(s_{k,1}, a_{k,1}, r_{k,1}, s_{k,2}, \dots, s_{k,T})$ given π_k

5: Compute $G_{k,t} = r_{k,t} + \gamma r_{k,t+1} + \gamma^2 r_{k,t+2} + \dots \gamma^{T-t} r_{k,T} \quad \forall t$

6: **For** $t = 1, \dots, T$ **do**

7: **If** First visit to (s, a) in episode k **then**

8: $N(s, a) = N(s, a) + 1$

9: $Q(s_t, a_t) = Q(s_t, a_t) + \frac{1}{N(s,a)} (G_{k,t} - Q(s_t, a_t))$

10: **End if**

11: **End for**

12: $k = k + 1, \epsilon = 1/k$

13: $\pi_k = \epsilon$ -greedy(Q) // *Policy improvement*

14: **End loop**

MC control with ϵ -greedy policies

- Is the prior algorithm a reasonable one?
- Will it eventually converge to the optimal Q^* function?
- Now see a set of conditions that is sufficient to ensure this desirable outcome

Greedy in the Limit of Infinite Exploration (GLIE)

Definition of GLIE

- All state-action pairs are visited an infinite number of times

$$\lim_{i \rightarrow \infty} N_i(s, a) \rightarrow \infty$$

- Behavior policy (policy used to act in the world) converges to greedy policy

$$\lim_{i \rightarrow \infty} \pi(a|s) \rightarrow \arg \max_a Q(s, a) \text{ with probability 1}$$

- A simple GLIE strategy is ϵ -greedy where ϵ is reduced to 0 with the following rate: $\epsilon_i = \frac{1}{i}$

GLIE Monte-Carlo Control

Theorem

GLIE Monte-Carlo control converges to the optimal state-action value function $Q(s, a) \rightarrow Q^*(s, a)$.

MC vs. TD Control

- Temporal-difference (TD) learning has several advantages over Monte-Carlo (MC)
 - Lower variance
 - Online
 - Incomplete sequences
- Natural idea: use TD instead of MC in our control loop
 - Apply TD to $Q(S, A)$
 - Use ϵ -greedy policy improvement

Model-free Policy Iteration with TD Methods

- Use temporal difference methods for policy evaluation step

- Initialize policy π
- Repeat:
 - Policy evaluation: compute Q^π using temporal difference updating with ϵ -greedy policy
 - Policy improvement: Same as Monte Carlo policy improvement, set π to ϵ -greedy (Q^π)

- Important issue: is it necessary for policy evaluation to converge to the true value of Q^π before updating the policy?
- Answer: As we've done with MC control, no. A single update of TD (policy evaluation updating of Q) is sufficient!
 - Unlike MC, we can do it each timestep

General Form of SARSA Algorithm

1: Set initial ϵ -greedy policy π randomly, $t = 0$, initial state $s_t = s_0$

2: Take $a_t \sim \pi(s_t)$

3: Observe (r_t, s_{t+1})

4: Loop

5: Take action $a_{t+1} \sim \pi(s_{t+1})$ // *Sample action from policy*

6: Observe (r_{t+1}, s_{t+2})

```

7:   Update  $Q$  given  $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$ : //  $\leftarrow$  Why algorithm name is SARSA
       $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha(r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t))$ 

```

8: Perform policy improvement:
 $\pi(s_t) = \arg \max_a Q(s_t, a)$ with probability $1 - \epsilon$, else random

9: $t = t + 1$

10: End loop

Convergence Properties of SARSA

Theorem

SARSA for finite-state and finite-action MDPs converges to the optimal action-value, $Q(s, a) \rightarrow Q^*(s, a)$, under the following conditions:

1. The policy sequence $\pi_t(a|s)$ satisfies the condition of GLIE
2. The step-sizes α_t satisfy the Robbins-Munro sequence such that

$$\begin{aligned}\sum_{t=1}^{\infty} \alpha_t &= \infty \\ \sum_{t=1}^{\infty} \alpha_t^2 &< \infty\end{aligned}$$

- E.g. $\alpha = \frac{1}{T}$ satisfies the above condition.

Q-Learning: Learning the Optimal State-Action Value

- SARSA is an **on-policy** learning algorithm
- SARSA estimates the value of the current **behavior** policy (policy using to take actions in the world)
- And then updates the policy trying to estimate
- Alternatively, can we directly estimate the value of π^* while acting with another behavior policy π_b ?
- Yes! Q-learning, an **off-policy** RL algorithm

On and Off-Policy Learning

- On-policy learning
 - Direct experience
 - “Learn on the job”
 - Learn to estimate and evaluate a policy π from experience obtained from following that policy π
- Off-policy learning
 - “Look over someone’s shoulder”
 - Learn to estimate and evaluate a policy π using experience gathered from following a different policy π_b

Off-Policy Learning

- Evaluate target policy $\pi(a|s)$ to compute $V^\pi(s)$ or $Q^\pi(s, a)$
- While following behavior policy $\pi_b(a|s)$

$$\{s_0, a_0, r_0, \dots, s_T\} \sim \pi_b$$

- Why is this important?
 - Learn from observing humans or other agents
 - Re-use experience generated from old policies $\pi_1, \pi_2, \dots, \pi_{t-1}$
 - Learn about **optimal** policy while following **exploratory** policy
 - Learn about **multiple** policies while following **one** policy

Importance Sampling

- Estimate the expectation of a different distribution

$$\mathbb{E}_{X \sim P}[f(X)] = \sum P(X)f(X)$$

$$= \sum Q(X) \frac{P(X)}{Q(X)} f(X)$$

$$= \mathbb{E}_{X \sim Q} \left[\frac{P(X)}{Q(X)} f(X) \right]$$

Importance Sampling for Off-Policy Monte-Carlo

- Use returns generated from π_b to evaluate π
- Weight return G_t according to similarity between policies
- Multiply importance sampling corrections along whole episode

$$G_t^{\pi/\pi_b} = \frac{P(G_t|\pi)}{P(G_t|\pi_b)} G_t = \frac{\pi(a_t|s_t)}{\pi_b(a_t|s_t)} \frac{\pi(a_{t+1}|s_{t+1})}{\pi_b(a_{t+1}|s_{t+1})} \cdots \frac{\pi(a_T|s_T)}{\pi_b(a_T|s_T)} G_t$$

- Update value towards corrected return

$$V(s_t) \leftarrow V(s_t) + \alpha \left(G_t^{\pi/\pi_b} - V(s_t) \right)$$

- Cannot use if π_b is zero when π is non-zero
- Importance sampling can dramatically increase variance

Importance Sampling for Off-Policy TD

- Use TD targets generated from π_b to evaluate π
- Weight TD target $r_t + \gamma V(s_{t+1})$ by importance sampling
- Only need a single importance sampling correction

$$V(s_t) \leftarrow V(s_t) + \alpha \left(\frac{\pi(a_t|s_t)}{\pi_b(a_t|s_t)} (r_t + \gamma V(s_{t+1})) - V(s_t) \right)$$

- Much lower variance than Monte-Carlo importance sampling
- Policies only need to be similar over a single step

Q-Learning

- We now consider off-policy learning of action-values $Q(s, a)$
- **No** importance sampling is required
- Next action is chosen using behavior policy $a_{t+1} \sim \pi_b(\cdot | s_t)$
- But we consider alternative successor action $a' \sim \pi(\cdot | s_t)$
- And update $Q(s_t, a_t)$ towards value of alternative action

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha(r_t + \gamma Q(s_{t+1}, a') - Q(s_t, a_t))$$

Off-Policy Control with Q-Learning

- We now allow both behavior and target policies to **improve**
- The target policy π is **greedy** with respect to $Q(s, a)$

$$\pi(s_{t+1}) = \arg \max_{a'} Q(s_{t+1}, a')$$

- The behavior policy π_b is e.g. **ϵ -greedy** with respect to $Q(s, a)$
- The Q-learning target then simplifies:

$$\begin{aligned} & r_{t+1} + \gamma Q(s_{t+1}, a') \\ &= r_{t+1} + \gamma Q(s_{t+1}, \arg \max_{a'} Q(s_{t+1}, a')) \\ &= r_{t+1} + \max_{a'} \gamma Q(s_{t+1}, a') \end{aligned}$$

Q-Learning: Learning the Optimal State-Action Value

- Key idea: Maintain state-action Q estimates and use to bootstrap— use the value of the best future action
- Recall SARSA:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left((r_t - \gamma Q(s_{t+1}, a_{t+1})) - Q(s_t, a_t) \right)$$

- Q-learning:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left((r_t - \gamma \max_{a'} Q(s_{t+1}, a')) - Q(s_t, a_t) \right)$$

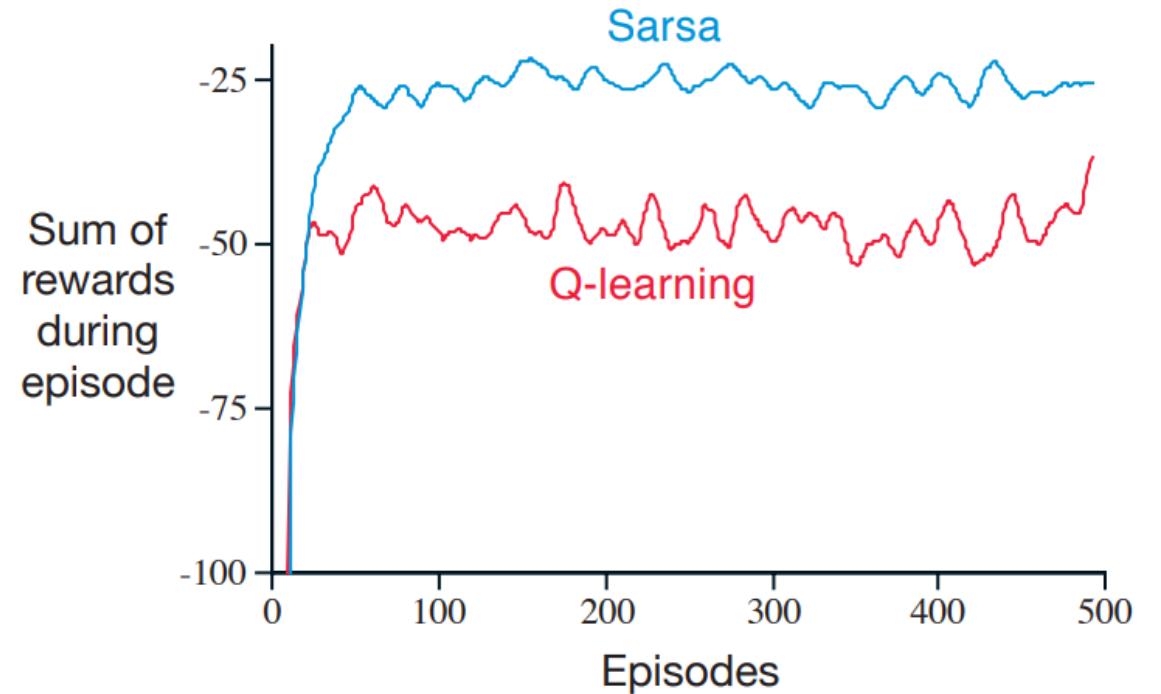
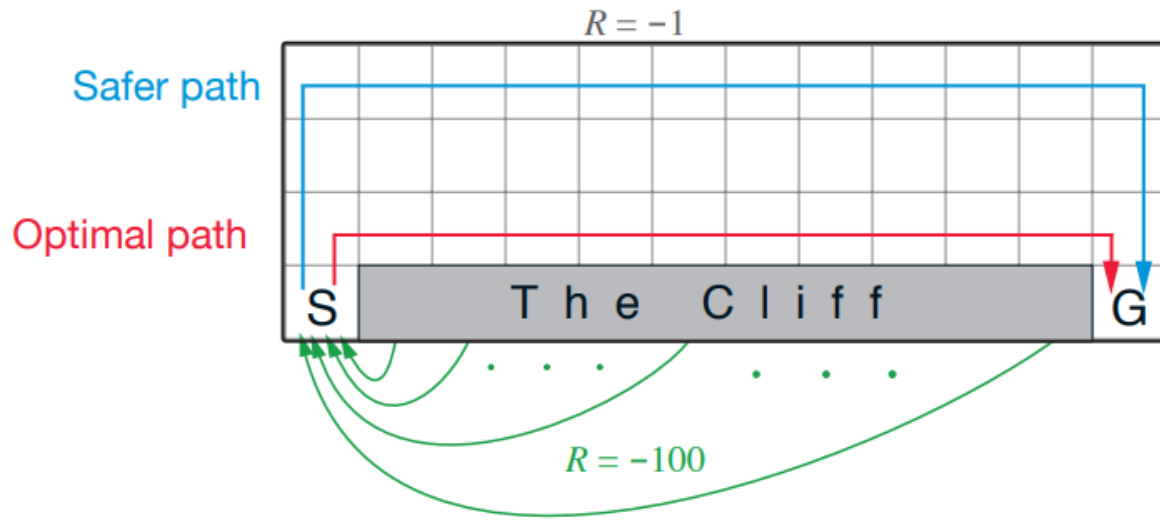
Q-Learning with ϵ -greedy Exploration

- 1: Initialize $Q(s, a), \forall s \in S, a \in A$
- 2: Initialize $t = 0, s_t = s_0$
- 3: Set π_b to be ϵ -greedy with respect to Q
- 4: **Loop**
- 5: Take action $a_t \sim \pi_b(s_t)$ *// Sample action from policy*
- 6: Observe (r_t, s_{t+1})
- 7: Update Q :
$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left(r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right)$$
- 8: Perform policy improvement:
$$\pi(s_t) = \arg \max_a Q(s_t, a) \text{ with probability } 1 - \epsilon, \text{ else random}$$
- 9: $t = t + 1$
- 10: **End loop**

Q-Learning with ϵ -greedy Exploration

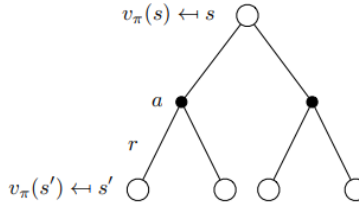
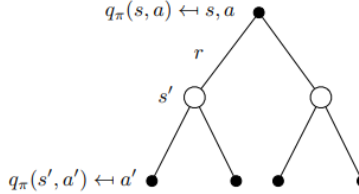
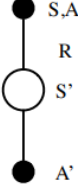
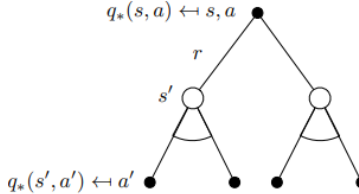
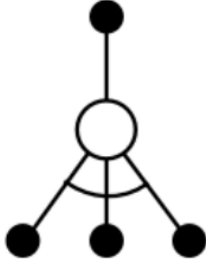
- What conditions are sufficient to ensure that Q-learning with ϵ -greedy exploration converges to optimal Q^* ?
 - Visit all (s, a) pairs infinitely often, and the step-sizes α_t satisfy the Robbins-Munro sequence.
 - Note: the algorithm does not have to be greedy in the limit of infinite exploration (GLIE) to satisfy this (could keep ϵ large).
- What conditions are sufficient to ensure that Q-learning with ϵ -greedy exploration converges to optimal π^* ?
 - The algorithm is GLIE, along with the above requirement to ensure the Q value estimates converge to the optimal Q .

Cliff Walking Example



- Interim performance: SARSA $>$ Q-learning
- Q-learning learns optimal policy, however

Relationship Between DP and TD

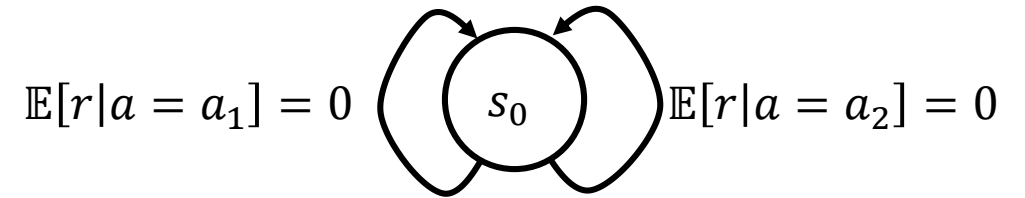
	<i>Full Backup (DP)</i>	<i>Sample Backup (TD)</i>
Bellman Expectation Equation for $v_{\pi}(s)$	 Iterative Policy Evaluation	 TD Learning
Bellman Expectation Equation for $q_{\pi}(s, a)$	 Q-Policy Iteration	 Sarsa
Bellman Optimality Equation for $q_{*}(s, a)$	 Q-Value Iteration	 Q-Learning

Relationship Between DP and TD

<i>Full Backup (DP)</i>	<i>Sample Backup (TD)</i>
Iterative Policy Evaluation	TD Learning
$V(s) \leftarrow \mathbb{E} [R + \gamma V(S') \mid s]$	$V(S) \stackrel{\alpha}{\leftarrow} R + \gamma V(S')$
Q-Policy Iteration	Sarsa
$Q(s, a) \leftarrow \mathbb{E} [R + \gamma Q(S', A') \mid s, a]$	$Q(S, A) \stackrel{\alpha}{\leftarrow} R + \gamma Q(S', A')$
Q-Value Iteration	Q-Learning
$Q(s, a) \leftarrow \mathbb{E} \left[R + \gamma \max_{a' \in \mathcal{A}} Q(S', a') \mid s, a \right]$	$Q(S, A) \stackrel{\alpha}{\leftarrow} R + \gamma \max_{a' \in \mathcal{A}} Q(S', a')$

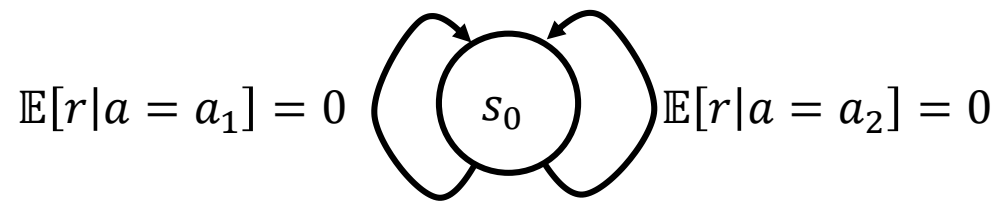
where $x \stackrel{\alpha}{\leftarrow} y \equiv x \leftarrow x + \alpha(y - x)$

Maximization Bias



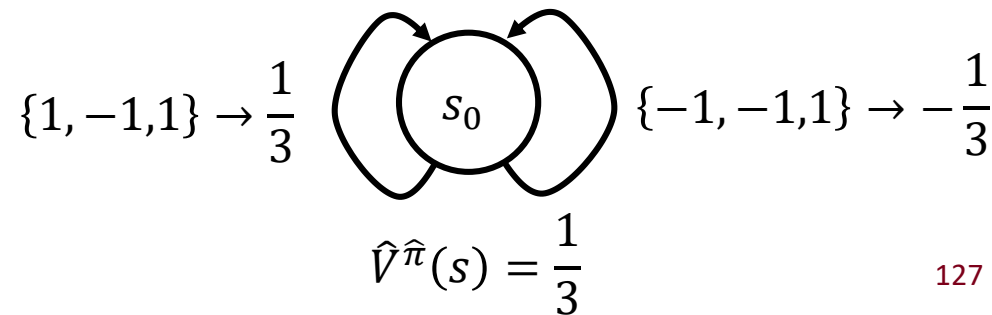
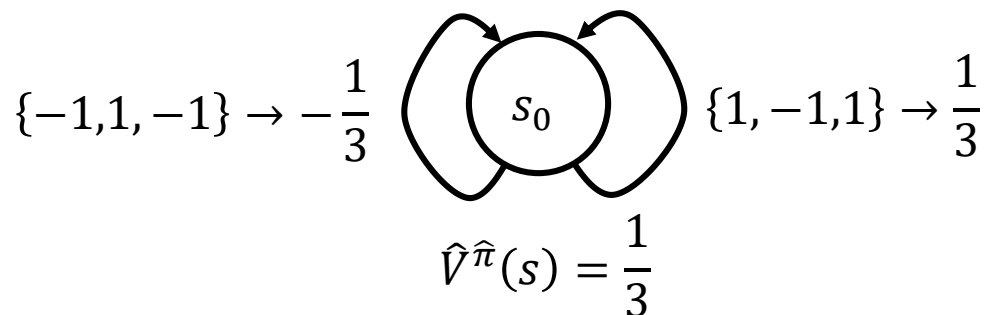
- Consider single-state MDP ($|S| = 1$) with 2 actions, and both actions have 0-mean random rewards, ($\mathbb{E}[r|a = a_1] = \mathbb{E}[r|a = a_2] = 0$).
- Then $Q(s, a_1) = Q(s, a_2) = 0 = V(s)$
- Assume there are prior samples of taking action a_1 and a_2
- Let $\hat{Q}(s, a_1), \hat{Q}(s, a_2)$ be the finite sample estimate of Q
- Use an unbiased estimator for Q : e.g. $\hat{Q}(s, a_1) = \frac{1}{n(s, a_1)} \sum_{i=1}^{n(s, a_1)} r_i(s, a_1)$
- Let $\hat{\pi} = \arg \max_a \hat{Q}(s, a)$ be the greedy policy with respect to the estimated Q

Maximization Bias



- Even though each estimate of the state-action values is unbiased:
- The estimate of $\hat{\pi}$'s value $\hat{V}^{\hat{\pi}}$ can be biased:

$$\begin{aligned}\mathbb{E}[\hat{V}^{\hat{\pi}}(s)] &= \mathbb{E}\left[\max\left(\hat{Q}(s, a_1), \hat{Q}(s, a_2)\right)\right] \\ &\geq \max(\mathbb{E}[\hat{Q}(s, a_1)], \mathbb{E}[\hat{Q}(s, a_2)]) \\ &= \max(0, 0) = V^{\pi}(s)\end{aligned}$$



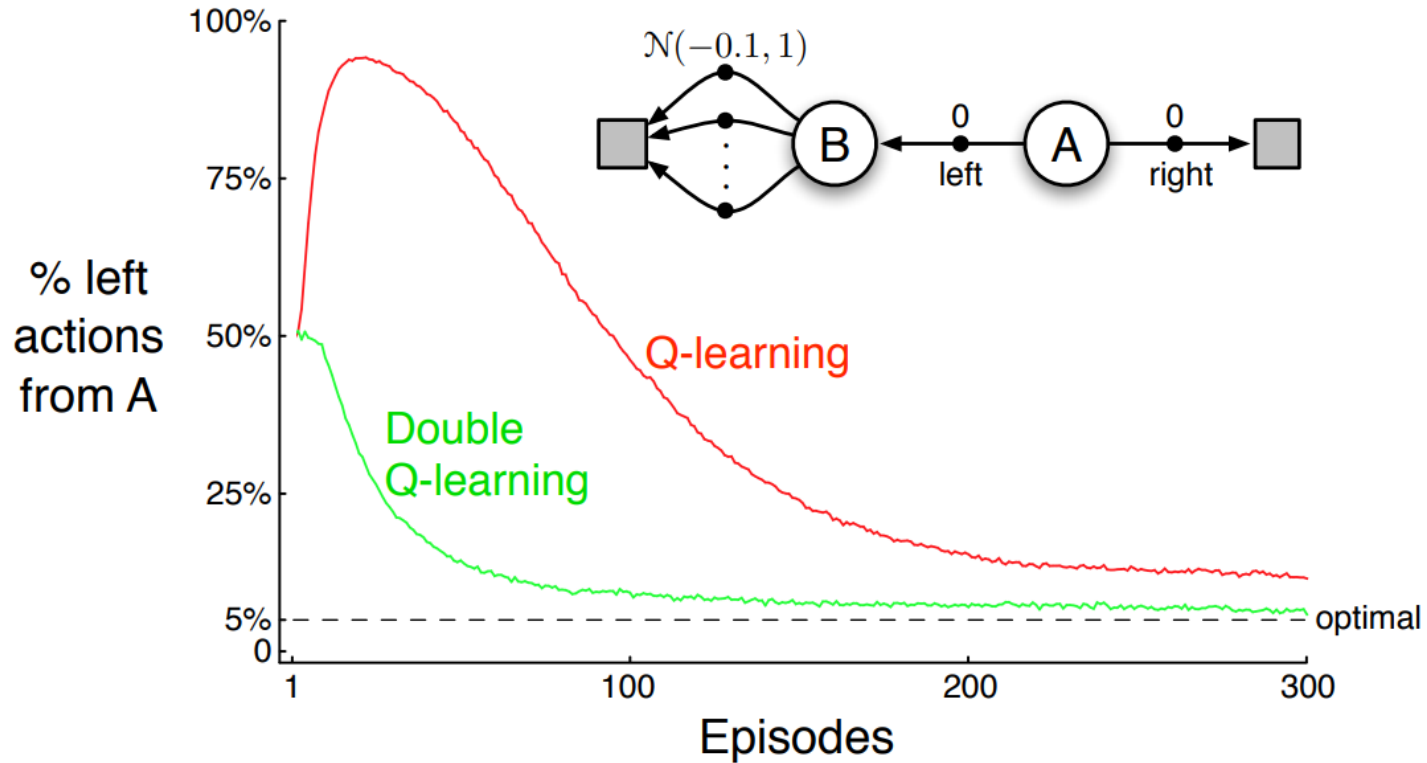
Double Q-Learning

- The greedy policy with respect to estimated Q values can yield a maximization bias during finite-sample learning
- Avoid using max of estimates as estimate of max of true values
- This is because **the estimator** and **the maximizer** are correlated
 - Instead split samples and use to create two independent unbiased estimates of $Q_1(s_1, a_i)$ and $Q_2(s_1, a_i) \forall a$.
 - Use one estimate to select max action: $a^* = \arg \max_a Q_1(s_1, a)$
 - Use other estimate to estimate value of a^* : $Q_2(s, a^*)$
 - Yields unbiased estimate: $\mathbb{E}[Q_2(s, a^*)] = Q(s, a^*)$
 - If acting online, can alternate samples used to update Q_1 and Q_2 , using the other to select the action chosen

Double Q-Learning

```
1: Initialize  $Q_1(s, a)$  and  $Q_2(s, a), \forall s \in S, a \in A$ 
2: Initialize  $t = 0, s_t = s_0$ 
3: Loop
4:     Select action  $a_t$  using  $\epsilon$ -greedy  $\pi(s) = \arg \max_a Q_1(s_t, a) + Q_2(s_t, a)$  // behavior policy does not matter
5:     Observe  $(r_t, s_{t+1})$ 
6:     If (with 0.5 probability) then
7:         Update  $Q_1$ :  $Q_1(s_t, a_t) \leftarrow Q_1(s_t, a_t) + \alpha \left( r_t + \gamma Q_2 \left( s_{t+1}, \arg \max_a Q_1(s_{t+1}, a) \right) - Q_1(s_t, a_t) \right)$ 
8:     Else
9:         Update  $Q_2$ :  $Q_2(s_t, a_t) \leftarrow Q_2(s_t, a_t) + \alpha \left( r_t + \gamma Q_1 \left( s_{t+1}, \arg \max_a Q_2(s_{t+1}, a) \right) - Q_2(s_t, a_t) \right)$ 
10:    End if
11:     $t = t + 1$ 
12: End loop
```

Double Q-Learning



- Due to the maximization bias, Q-learning spends much more time selecting suboptimal actions than double Q-learning ($\epsilon = 0.1$)

Summary: Evaluation to Control

- Last lecture: how good is a specific policy?
 - Given no access to the decision process model parameters
 - Instead have to estimate from data / experience
- This: how can we learn a good policy?

Recall: Reinforcement Learning Involves

- Decision optimization
 - Goal is to identify a policy with high expected rewards
- Sequentiality (delayed consequence)
 - May take many timesteps to evaluate whether an earlier decision was good or not
- Exploration
 - Necessary to try different actions to learn what actions can lead to high rewards
- Generalization
 - This is what we are going to do from next lectures – Value function approximation!

Thank you!

강화학습

LG Aimers



고려대학교 인공지능학과 이병준
Lecture 4: Function Approximation

Last Lecture: Tabular Representations for Model-free Control

- Last lecture: how to learn a good policy from experience
- So far, have been assuming we can represent the value function or state-action value function as a vector/ matrix
 - Tabular representation
- Many real world problems have enormous state and/or action spaces
 - Backgammon: 10^{20} states
 - Computer Go: 10^{170} states
 - Helicopter: continuous state space
- Tabular representation is insufficient
 - How can we scale up the model-free methods for prediction and control from the last two lectures?

Motivation for Function Approximation

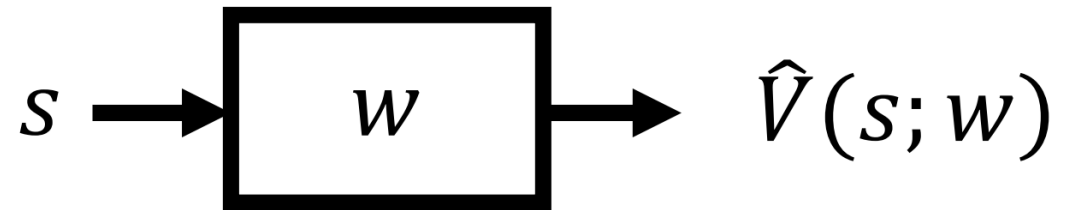
- Don't want to have to explicitly store or learn for every single state a
 - Dynamics or reward model
 - Value
 - State-action value
 - Policy
- Want more compact representation that generalizes across state or states and actions

Benefits of Function Approximation

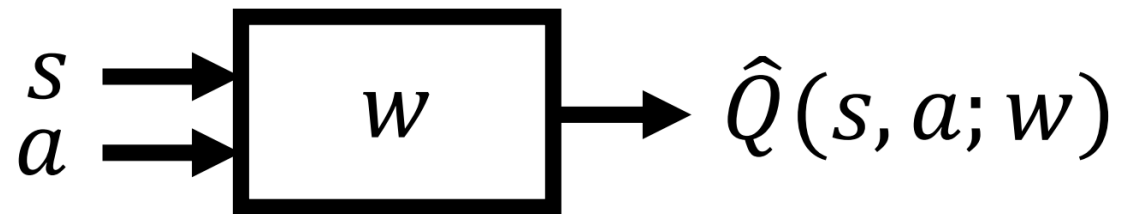
- Reduce memory needed to store $(P, R)/V/Q/\pi$
- Reduce computation needed to compute $(P, R)/V/Q/\pi$
- Reduce experience needed to find a good $(P, R)/V/Q/\pi$

Value Function Approximation (VFA)

- Represent a (state-action/state) value function with a parameterized function instead of a table



- Which function approximator?



Function Approximators

- Many possible function approximators including
 - Linear combinations of features
 - Neural networks
 - Decision trees
 - Nearest neighbors
 - Fourier/ wavelet bases
- In this class we will focus on function approximators that are differentiable (Why?)
- Two very popular classes of differentiable function approximators
 - Linear feature representations (Today)
 - Neural networks (Next lecture)

Review: Gradient Descent

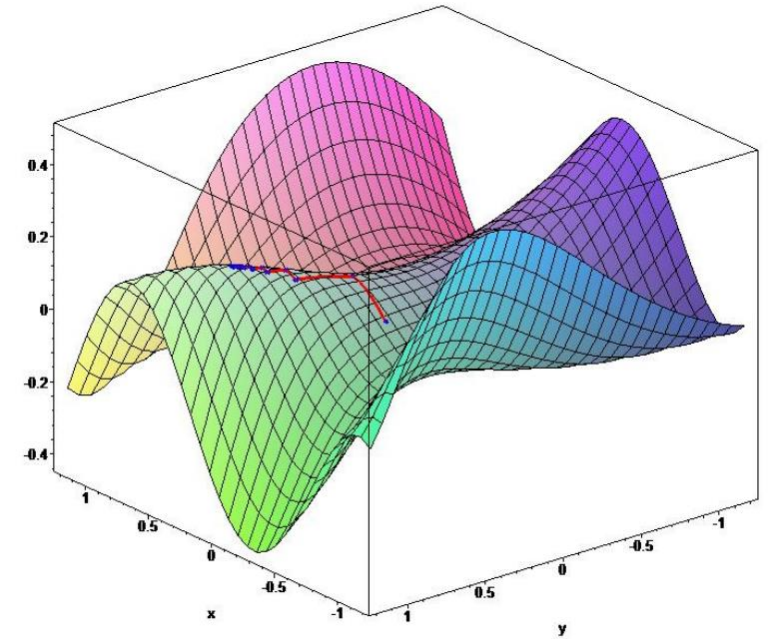
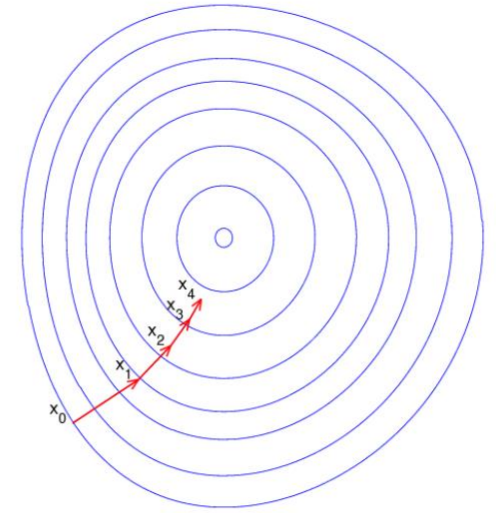
- Consider a function $J(\mathbf{w})$ that is a differentiable function of a parameter vector $\mathbf{w}$
- Goal is to find parameter $\mathbf{w}$ that minimizes J
- The gradient of $J(\mathbf{w})$ is

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \begin{pmatrix} \frac{\partial J(\mathbf{w})}{\partial w_1} \\ \vdots \\ \frac{\partial J(\mathbf{w})}{\partial w_n} \end{pmatrix}$$

- Adjust $\mathbf{w}$ in direction of negative gradient

$$\Delta \mathbf{w} = -\frac{1}{2} \alpha \nabla_{\mathbf{w}} J(\mathbf{w})$$

- Where α is step-size parameter



Value Function Approximation for Policy Evaluation with an Oracle

- First assume we could query any state s and an oracle would return the true value for $V^\pi(s)$
- Similar to supervised learning: assume given $(s, V^\pi(s))$ pairs
- The objective is to find the best approximate representation of V^π given a particular parameterized function $\hat{V}(s; \mathbf{w})$

Stochastic Gradient Descent

- Goal: find parameter vector $\mathbf{w}$ minimizing mean-squared error between approximate value fn $\hat{V}(s; \mathbf{w})$ and true value fn $V^\pi(s)$

$$J(\mathbf{w}) = \mathbb{E}_\pi \left[\left(V^\pi(s) - \hat{V}(s; \mathbf{w}) \right)^2 \right]$$

- Gradient descent finds a local minimum

$$\Delta \mathbf{w} = -\frac{1}{2} \alpha \nabla_{\mathbf{w}} J(\mathbf{w}) = \alpha \mathbb{E}_\pi \left[\left(V^\pi(s) - \hat{V}(s; \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{V}(s; \mathbf{w}) \right]$$

- Stochastic gradient decent samples the gradient

$$\Delta \mathbf{w} = \alpha \left(V^\pi(s) - \hat{V}(s; \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{V}(s; \mathbf{w})$$

- Expected update is equal to full gradient update

Model Free VFA Policy Evaluation

- Don't actually have access to an oracle to tell true $V^\pi(s)$ for any state s
- Now consider how to do model-free value function approximation for prediction / evaluation / policy evaluation without a model

Model Free VFA Prediction / Policy Evaluation

- Recall model-free policy evaluation (Lecture 3)
 - Following a fixed policy π (or had access to prior data)
 - Goal is to estimate V^π and/or Q^π
- Maintained a lookup table to store estimates V^π and/or Q^π
- Updated these estimates after each episode (Monte Carlo methods) or after each step (TD methods)
- Now: in value function approximation, change the estimate update step to include fitting the function approximator

Outline for Today

- Value function approximation
- **Monte Carlo policy evaluation with linear function approximation**
- TD policy evaluation with linear function approximation
- Control methods with linear value function approximation
- Linear Value-function Geometry

Feature Vectors

- Use a feature vector to represent a state s

$$\mathbf{x}(s) = \begin{pmatrix} x_1(s) \\ x_2(s) \\ \vdots \\ x_n(s) \end{pmatrix}$$

- For example:
 - Distance of robot from landmarks
 - Trends in the stock market
 - Piece and pawn configurations in chess

Linear Value Function Approximation for Prediction With An Oracle

- Represent a value function (or state-action value function) for a particular policy with a weighted linear combination of features

$$\hat{V}(s; \mathbf{w}) = \sum_{j=1}^n x_j(s) w_j = \mathbf{x}(s)^\top \mathbf{w}$$

- Objective function is

$$J(\mathbf{w}) = \mathbb{E}_\pi \left[\left(V^\pi(s) - \hat{V}(s; \mathbf{w}) \right)^2 \right]$$

Stochastic gradient descent converges on global optimum

- Update rule is:

$$\Delta \mathbf{w} = \alpha \left(V^\pi(s) - \hat{V}(s; \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{V}(s; \mathbf{w}) = \alpha \left(V^\pi(s) - \hat{V}(s; \mathbf{w}) \right) \mathbf{x}(s)$$

Update = stepsize $\times$ prediction error $\times$ feature value

Table Lookup Features

- Table lookup is a special case of linear value function approximation
- Using table lookup features

$$\mathbf{x}^{table}(s) = \begin{pmatrix} \mathbb{I}(s = s_1) \\ \vdots \\ \mathbb{I}(s = s_n) \end{pmatrix}$$

- Parameter vector $\mathbf{w}$ gives value of each individual state

$$\hat{V}(s; \mathbf{w}) = \begin{pmatrix} \mathbb{I}(s = s_1) \\ \vdots \\ \mathbb{I}(s = s_n) \end{pmatrix} \cdot \begin{pmatrix} w_1 \\ \vdots \\ w_n \end{pmatrix} = \sum_{i=1}^n \mathbb{I}(s = s_i) w_i$$

Incremental Prediction Algorithms

- Have assumed true value function $V^\pi(s)$ given by an oracle
- But in RL there is no oracle, only rewards
- In practice, we substitute a target for $V^\pi(s)$
 - For MC, the target is the return G_t
 - $\Delta \mathbf{w} = \alpha \left(G_t - \hat{V}(s_t; \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{V}(s_t; \mathbf{w})$
 - For TD(0), the target is the TD target $r_t + \gamma \hat{V}(s_{t+1}; \mathbf{w})$
 - $\Delta \mathbf{w} = \alpha \left(r_t + \gamma \hat{V}(s_{t+1}; \mathbf{w}) - \hat{V}(s_t; \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{V}(s_t; \mathbf{w})$
 - For TD(λ), the target is the λ -return G_t^λ
 - $\Delta \mathbf{w} = \alpha \left(G_t^\lambda - \hat{V}(s_t; \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{V}(s_t; \mathbf{w})$

Monte-Carlo with Value Function Approximation

- Return G_t is an unbiased, (very) noisy sample of true value $V^\pi(s_t)$
- Therefore can reduce MC VFA to doing supervised learning on a set of (state, return) pairs:

$$\langle s_1, G_1 \rangle, \langle s_2, G_2 \rangle, \dots, \langle s_T, G_T \rangle$$

- For example, when using linear VFA for policy evaluation:

$$\begin{aligned}\Delta \mathbf{w} &= \alpha \left(G_t - \hat{V}(s_t; \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{V}(s_t; \mathbf{w}) \\ &= \alpha (G_t - \mathbf{x}(s_t)^\top \mathbf{w}) \mathbf{x}(s_t)\end{aligned}$$

MC Linear Value Function Approximation for Policy Evaluation

```
1: Initialize  $\mathbf{w} = 0, k = 1$ 
2: Loop
3:   Sample  $k^{\text{th}}$  episode  $(s_{k,1}, a_{k,1}, r_{k,1}, s_{k,2}, \dots, s_{k,T})$  given  $\pi$ 
4:   Compute  $G_{k,t} = r_{k,t} + \gamma r_{k,t+1} + \gamma^2 r_{k,t+2} + \dots \gamma^{T-t-1} r_{k,T} \quad \forall t$ 
5:   For  $t = 1, \dots, T$  do
6:     Compute state feature vector  $\mathbf{x}(s_t)$ 
7:     Update weights:
           
$$\mathbf{w} \leftarrow \mathbf{w} + \alpha [G_t - \mathbf{x}(s_t)^\top \mathbf{w}] \mathbf{x}(s_t)$$

8:   End for
9:    $k = k + 1$ 
10: End loop
```

Convergence Guarantees for Linear Value Function Approximation for Policy Evaluation

- Define the mean squared error of a linear value function approximation for a particular policy π relative to the true value as (Value Error)

$$VE_{\mu}(\mathbf{w}) = \sum_{s \in \mathcal{S}} \mu(s) \left(V^{\pi}(s) - \hat{V}^{\pi}(s; \mathbf{w}) \right)^2$$

- Where
 - $\mu(s)$: probability of visiting state s under policy π . Note $\sum_s \mu(s) = 1$
 - $\hat{V}^{\pi}(s; \mathbf{w}) = \mathbf{x}(s)^{\top} \mathbf{w}$: a linear value function approximation

Convergence Guarantees for Linear Value Function Approximation for Policy Evaluation

- Define the mean squared error of a linear value function approximation for a particular policy π relative to the true value as (Value Error)

$$VE_{\mu}(\mathbf{w}) = \sum_{s \in \mathcal{S}} \mu(s) \left(V^{\pi}(s) - \hat{V}^{\pi}(s; \mathbf{w}) \right)^2$$

- Where
 - $\mu(s)$: probability of visiting state s under policy π . Note $\sum_s \mu(s) = 1$
 - $\hat{V}^{\pi}(s; \mathbf{w}) = \mathbf{x}(s)^{\top} \mathbf{w}$: a linear value function approximation
- Monte Carlo policy evaluation with **linear VFA** converges to the weights $\mathbf{w}_{MC}$ which has the minimum mean squared possible with respect to the distribution μ :

$$VE_{\mu}(\mathbf{w}_{MC}) = \min_{\mathbf{w}} VE_{\mu}(\mathbf{w})$$

- It converges to local minimum when nonlinear VFA is used
- This is because it follows general SGD update rule

Recall: Temporal Difference Learning w/ Lookup Table

- Uses bootstrapping and sampling to approximate V^π
- Updates $V^\pi(s)$ after each transition (s, a, r, s') :
$$V^\pi(s) = V^\pi(s) + \alpha(r + \gamma V^\pi(s') - V^\pi(s))$$
- Target is $r + \gamma V^\pi(s')$, a biased estimate of the true value $V^\pi(s)$
- In value function approximation, target is $r + \gamma \hat{V}^\pi(s'; \mathbf{w})$, a biased and approximated estimate of the true value $V^\pi(s)$
- 3 forms of approximation
 - Sampling
 - Bootstrapping
 - Value function approximation

Temporal Difference (TD(0)) Learning with Value Function Approximation

- In value function approximation, target is $r + \gamma \hat{V}^\pi(s'; \mathbf{w})$, a biased and approximated estimate of the true value $V^\pi(s)$

- Therefore can reduce TD(0) VFA to doing supervised learning on a set of data pairs:

$$\langle s_1, r_1 + \gamma \hat{V}^\pi(s_2; \mathbf{w}) \rangle, \langle s_2, r_2 + \gamma \hat{V}^\pi(s_3; \mathbf{w}) \rangle, \dots, \langle s_T, r_T \rangle$$

- With linear VFA:

$$\Delta \mathbf{w} = \alpha \left(r_t + \gamma \hat{V}(s'; \mathbf{w}) - \hat{V}(s; \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{V}(s; \mathbf{w}) = \alpha \delta \mathbf{x}(s)$$

- Is it a valid gradient of:

$$J(\mathbf{w}) = \mathbb{E}_\pi \left[\left(r + \gamma \hat{V}(s'; \mathbf{w}) - \hat{V}(s; \mathbf{w}) \right)^2 \right] ?$$

- It is not; the gradient of target $\hat{V}(s'; \mathbf{w})$ is ignored when computing the update.
- It is called semi-gradient method.

Semi-gradient TD(0)

1: Initialize $\mathbf{w} = 0, k = 1$

2: **Loop**

3: Sample tuple (s_k, a_k, r_k, s_{k+1}) given π

4: Update weights:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha[r_k + \gamma \mathbf{x}(s_{k+1})^\top \mathbf{w} - \mathbf{x}(s_k)^\top \mathbf{w}] \mathbf{x}(s_k)$$

5: $k = k + 1$

6: **End loop**

Semi-gradient TD(0)

- What does semi-gradient TD(0) do?

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha [r_t + \gamma \mathbf{x}(s_{t+1})^\top \mathbf{w}_t - \mathbf{x}(s_t)^\top \mathbf{w}_t] \mathbf{x}(s_t)$$

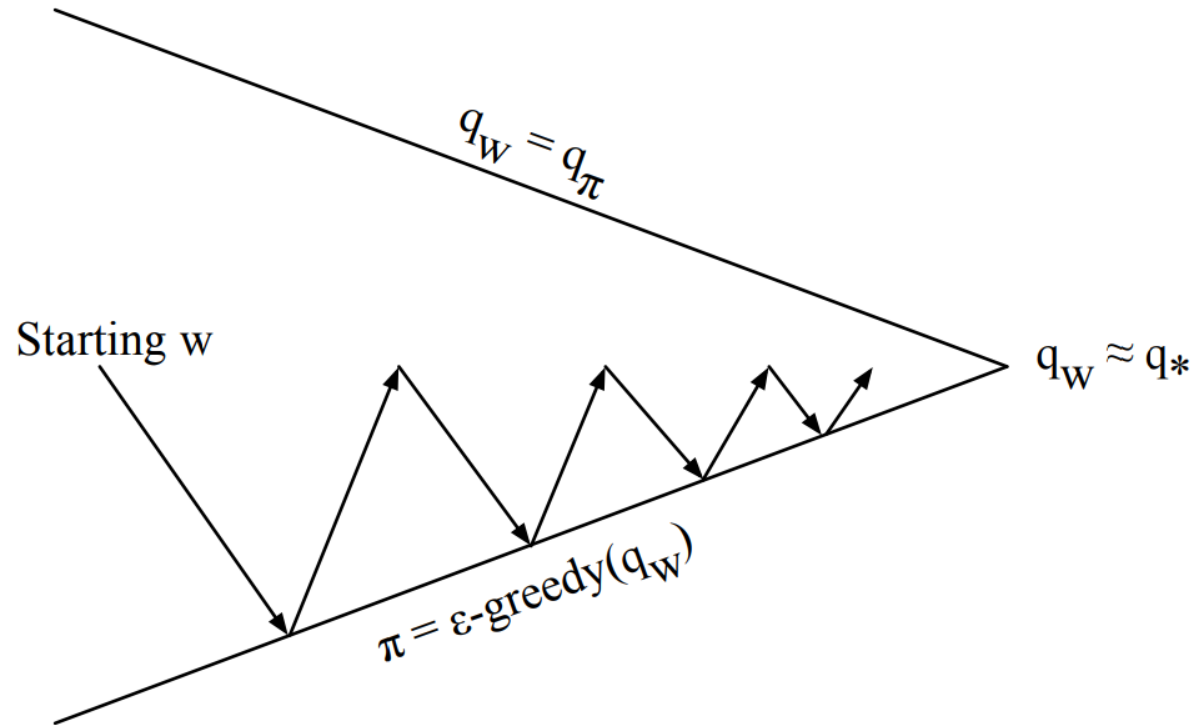
- If the system has reached steady state (since $\Delta \mathbf{w}$ should be 0 to make sure its steady):

$$\mathbb{E}[\mathbf{w}_{t+1} | \mathbf{w}_t] = \mathbf{w}_t + \alpha \left(\mathbb{E}[r_t \mathbf{x}(s_t)] - \mathbb{E} \left[\mathbf{x}(s_t) (\mathbf{x}(s_t) - \gamma \mathbf{x}(s_{t+1}))^\top \right] \mathbf{w}_t \right)$$

- And $\mathbf{w}_{TD} = \left(\mathbb{E} \left[\mathbf{x}(s_t) (\mathbf{x}(s_t) - \gamma \mathbf{x}(s_{t+1}))^\top \right] \right)^{-1} \mathbb{E}[r_t \mathbf{x}(s_t)]$ (TD fixed point)
 - When on-policy, this inverse always exists and the algorithm converges to it given appropriate α scheduling
- TD fixed point is close but different from the optimal point; it is shown that

$$VE_\mu(\mathbf{w}_{TD}) \leq \frac{1}{1 - \gamma} \min_{\mathbf{w}} VE_\mu(\mathbf{w})$$

Control with Value Function Approximation



- Policy evaluation: Approximate policy evaluation: $\hat{Q}(\cdot, \cdot; \mathbf{w}) = Q^\pi$
- Policy improvement: ϵ -greedy policy improvement

Action-Value Function Approximation with an Oracle

- We would like to approximate the action-value function

$$\hat{Q}(s, a; \mathbf{w}) \approx Q^\pi(s, a)$$

- Minimize the mean-squared error between the true action-value function $Q^\pi(s, a)$ and the approximate action-value function:

$$J(\mathbf{w}) = \mathbb{E}_\pi \left[\left(Q^\pi(s, a) - \hat{Q}^\pi(s, a; \mathbf{w}) \right)^2 \right]$$

- Use stochastic gradient descent to find a local minimum

$$\Delta(\mathbf{w}) = -\frac{1}{2} \alpha \nabla_{\mathbf{w}} J(\mathbf{w}) \approx \alpha \left(Q^\pi(s, a) - \hat{Q}^\pi(s, a; \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{Q}(s, a; \mathbf{w})$$

Linear Action-Value Function Approximation

- Use features to represent both the state and action

$$\mathbf{x}(s, a) = \begin{pmatrix} x_1(s, a) \\ x_2(s, a) \\ \vdots \\ x_n(s, a) \end{pmatrix}$$

- Represent action-value function with a weighted linear combination of features

$$\hat{Q}(s, a; \mathbf{w}) = \mathbf{x}(s, a)^\top \mathbf{w} = \sum_{j=1}^n x_j(s, a) w_j$$

- Stochastic gradient descent update

$$\Delta \mathbf{w} = \alpha \left(Q^\pi(s, a) - \hat{Q}(s, a; \mathbf{w}) \right) \mathbf{x}(s, a)$$

Incremental Model-Free Control Approaches

- Similar to policy evaluation, true state-action value function for a state is unknown and so substitute a target value

- For MC control, use a return G_t as a substitute target

$$\Delta \mathbf{w} = \alpha \left(G_t - \hat{Q}(s_t, a_t; \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{Q}(s_t, a_t; \mathbf{w})$$

- For SARSA, use TD target that leverages the current function approximation value

$$\Delta \mathbf{w} = \alpha \left(r_t + \gamma \hat{Q}(s_{t+1}, a_{t+1}; \mathbf{w}) - \hat{Q}(s_t, a_t; \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{Q}(s_t, a_t; \mathbf{w})$$

- For Q-learning, use greedy TD target that leverages the current function approximation value

$$\Delta \mathbf{w} = \alpha \left(r_t + \gamma \max_{a'} \hat{Q}(s_{t+1}, a'; \mathbf{w}) - \hat{Q}(s_t, a_t; \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{Q}(s_t, a_t; \mathbf{w})$$

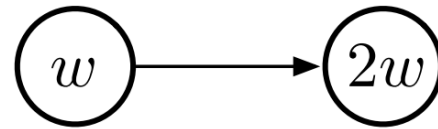
- Note: SARSA and Q-learning are also semi-gradient methods!

Convergence of TD Methods with VFA

- Informally, updates involve doing an (approximate) Bellman backup followed by best trying to fit underlying value function to a particular feature representation
- Bellman operators are contractions, but value function approximation fitting can be an expansion
- In linear case & on-policy, $TD(0)$ & $TD(\lambda)$ & $SARSA(0)$ & $SARSA(\lambda)$ algorithms converge
- Otherwise (non-linear VFA or off-policy), we cannot guarantee convergence

Challenges of Off Policy Control: Simple Example

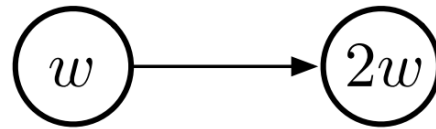
- As a part of a larger MDP, assume two states whose estimated values are of w and $2w$



- Suppose initially $w = 10$
- Then transition is from a state of value 10 to a state of value 20.
- If we do the TD update, the TD target is close to 20, and it will make w increase
- If w increases, the value of both states increase
- With further updates, w will diverge to infinity.

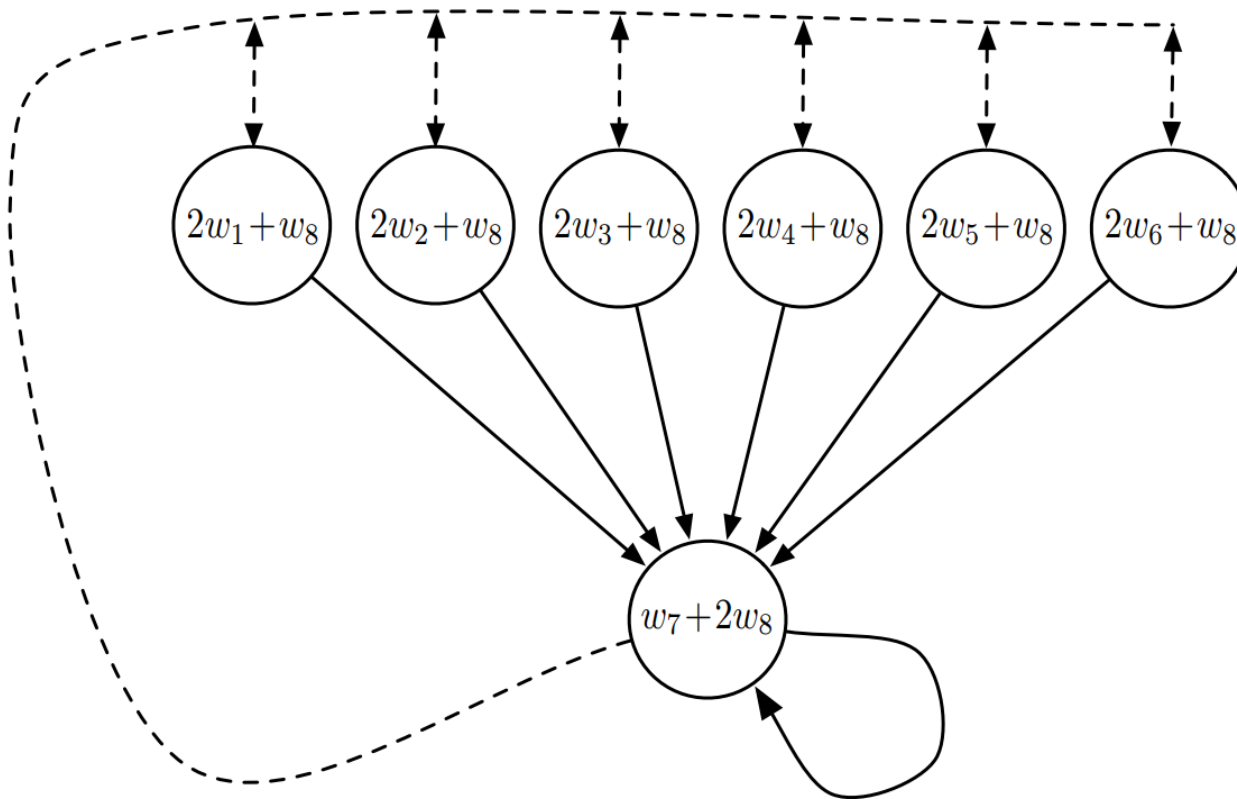
Challenges of Off Policy Control: Simple Example

- As a part of a larger MDP, assume two states whose estimated values are of w and $2w$



- This is due to the combination of multiple issues:
 - If we were not using **VFA**, this wouldn't happen in the first place
 - This can only happen when w is **not updated from other transitions** (e.g. if there is another state that is transitioned from $2w$, w may not diverge due to such transition). This is only possible under **off-policy** training
 - This can only happen under **bootstrapping algorithms**
 - This can only happen with **semi-gradient** algorithms; if we derive a SGD-based algorithm, it will decrease target's value at the same time.

Challenges of Off Policy Control: Baird's Counterexample



$$\pi(\text{solid}|\cdot) = 1$$

$$b(\text{dashed}|\cdot) = 6/7$$

$$b(\text{solid}|\cdot) = 1/7$$

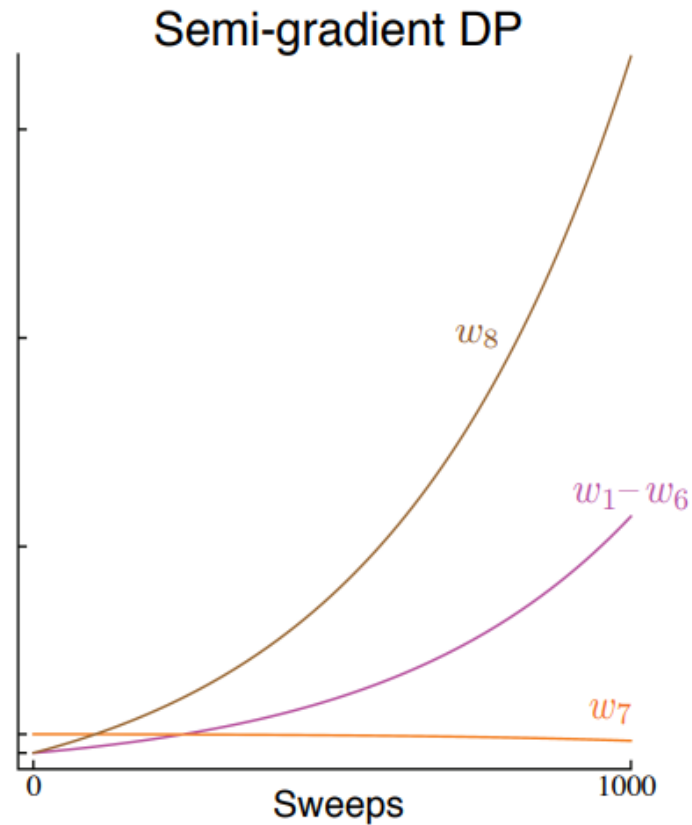
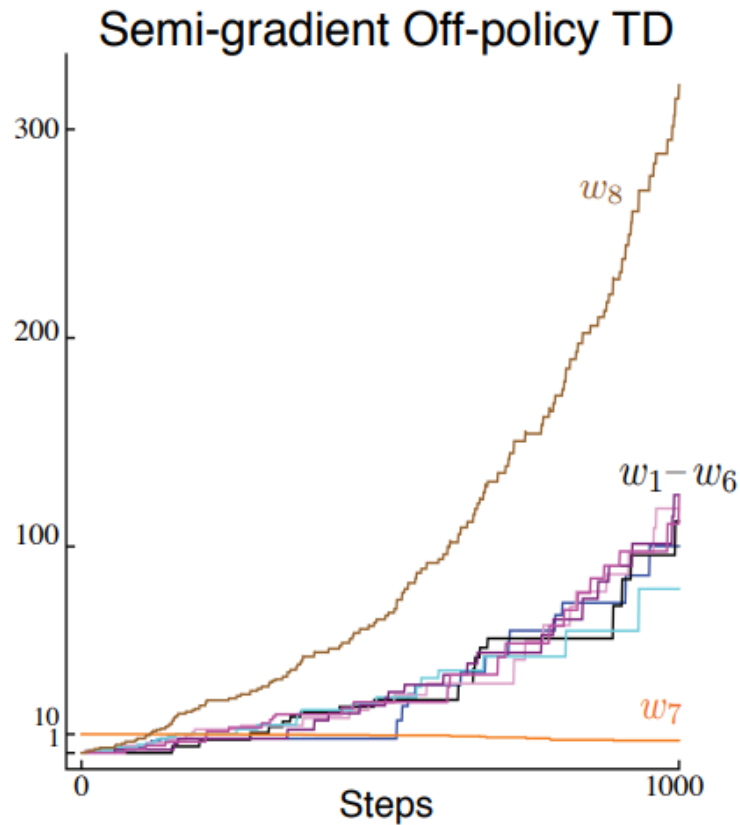
$$\gamma = 0.99$$

- Solid action takes the agent to the bottom state
- Dashed action takes the agent to the top states
- Reward = 0 for all state actions, so optimal $\mathbf{w} = \mathbf{0}$
- Initial $\mathbf{w} = [1, 1, 1, 1, 1, 1, 10, 1]$

$$\Delta \mathbf{w} = \alpha \frac{\pi}{\pi_b} \left(r_t + \gamma \hat{V}(s_{t+1}; \mathbf{w}) - \hat{V}(s_t; \mathbf{w}) \right) \mathbf{x}(s_t)$$

- ~~(top, dash, top), (bot, dash, top)~~
 - Update has no effect due to importance weight
- (bot, solid, bot)
 - Decays w_7 by 0.99
- (top, solid, bot)
 - One of $\{w_1, \dots, w_6\}$ and w_8 becomes larger

Challenges of Off Policy Control: Baird's Counterexample

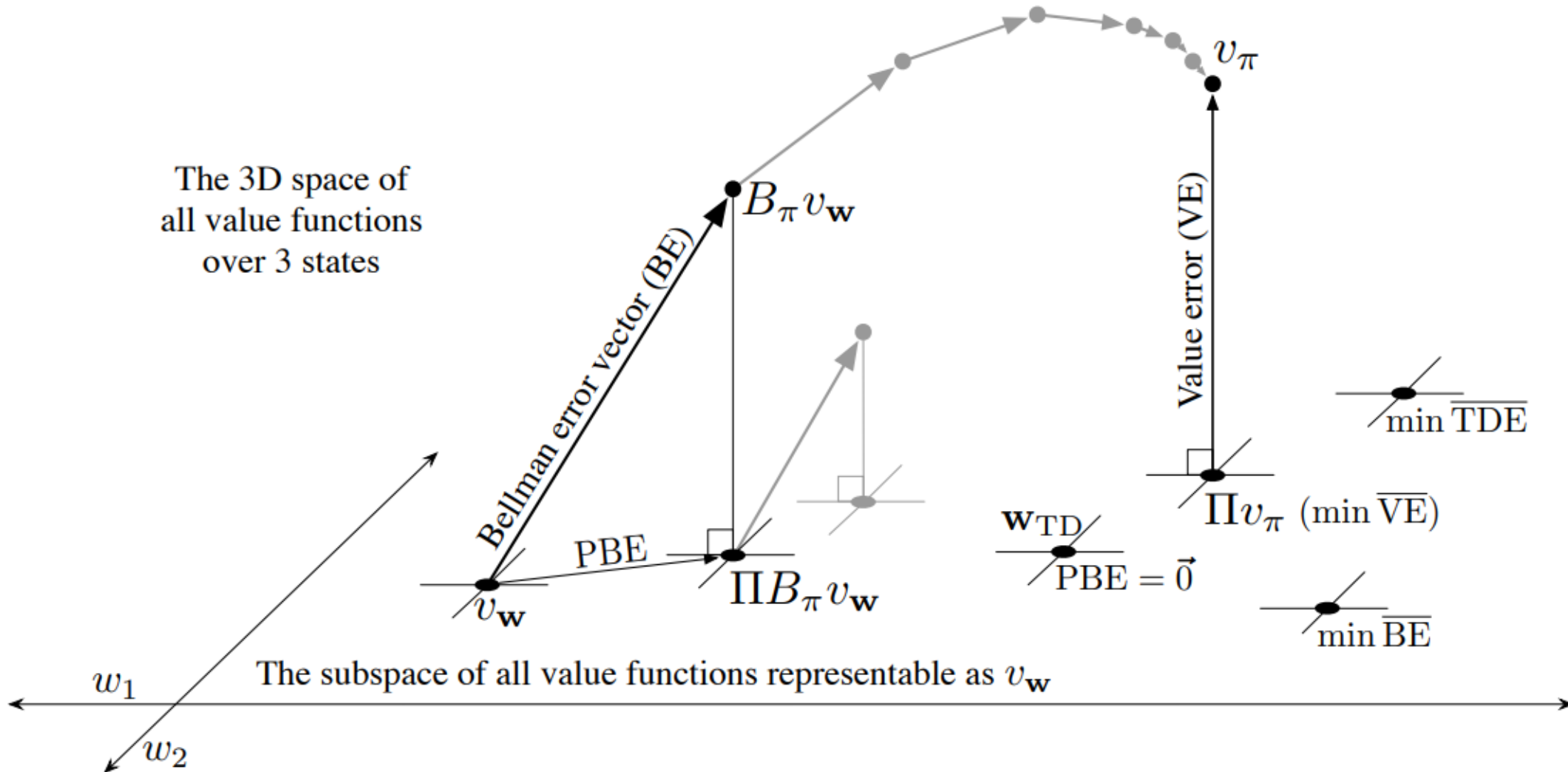


- Semi-gradient Off-policy TD diverges
- Semi-gradient DP diverges as well
- We can also make a parametrization that makes Q based algorithms diverge
- The main reason of instability is the combination of (**deadly triad**):
 - Function approximation
 - Bootstrapping (with semi-gradient training)
 - Off-policy training

Linear Value-function Geometry

- Imagine a space of value functions $\mathcal{V}: S \rightarrow \mathbb{R}$: for finite states, it is $\mathbb{R}^{|S|}$.
 - Most of the value functions in this space is not associated with a policy.
- A space of linearly parametrized value functions $\mathcal{W}$ is a subspace of $\mathcal{V}$.
- We also define a norm $\|V\|_{\mu}^2 = \sum_s \mu(s) V(s)^2$.
- Note that $VE_{\mu}(\mathbf{w}) = \|V(\cdot; \mathbf{w}) - V^{\pi}\|_{\mu}^2$.
- Let's consider the case with three states $S = \{s_1, s_2, s_3\}$ and two parameters $w = [w_1, w_2]^T$
- $\mathcal{V}$ is 3-dimensional space; any value function is a point inside it.
- $\mathcal{W}$ is a plane in $\mathcal{V}$. Any parametrized value function is a point on the plane.

Linear Value-function Geometry



Linear Value-function Geometry

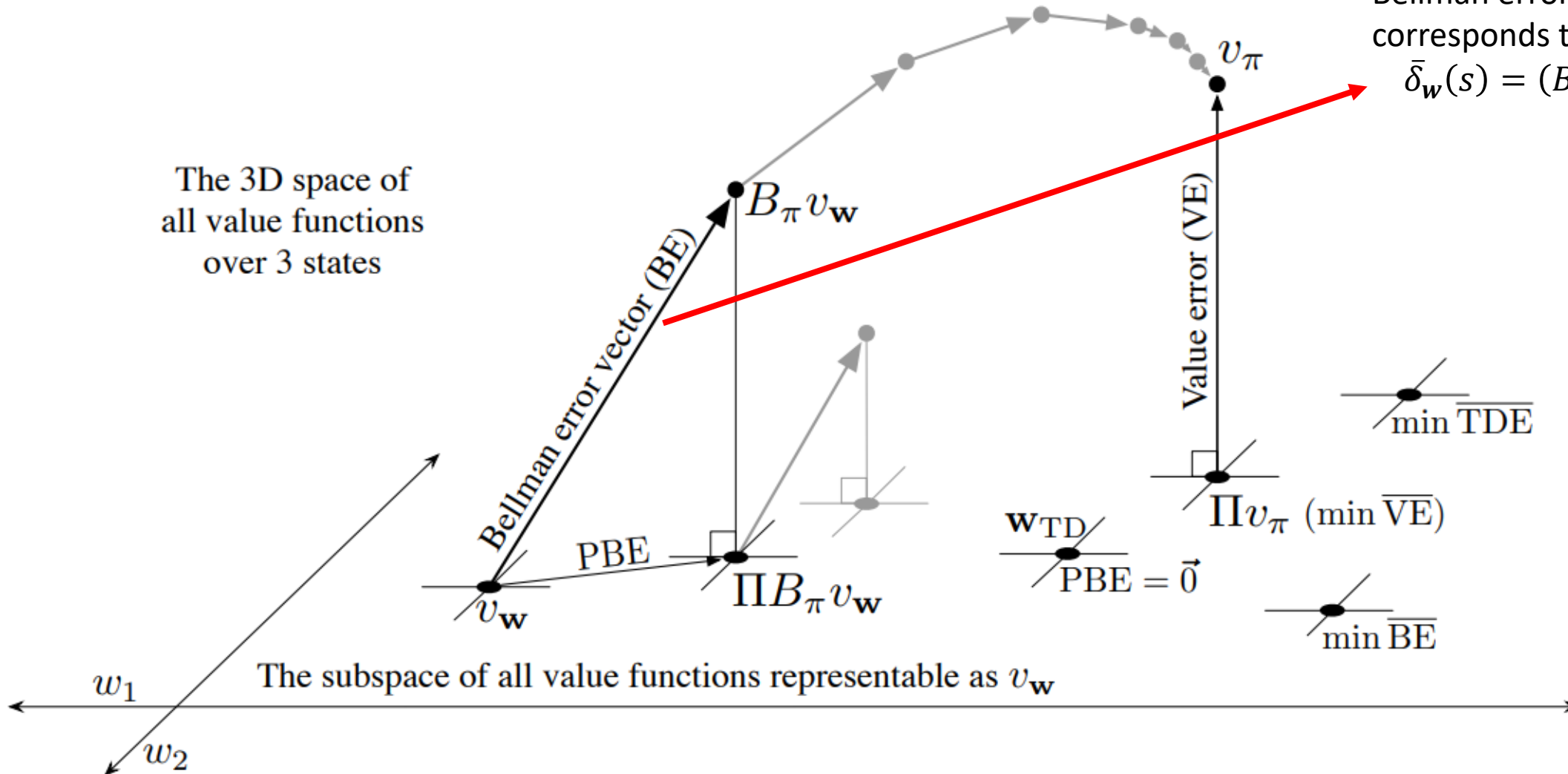
We do Bellman backup in the value function space.

$$(B^\pi V)(s) = \mathbb{E}_{\pi, P}[r + \gamma V(s')]$$

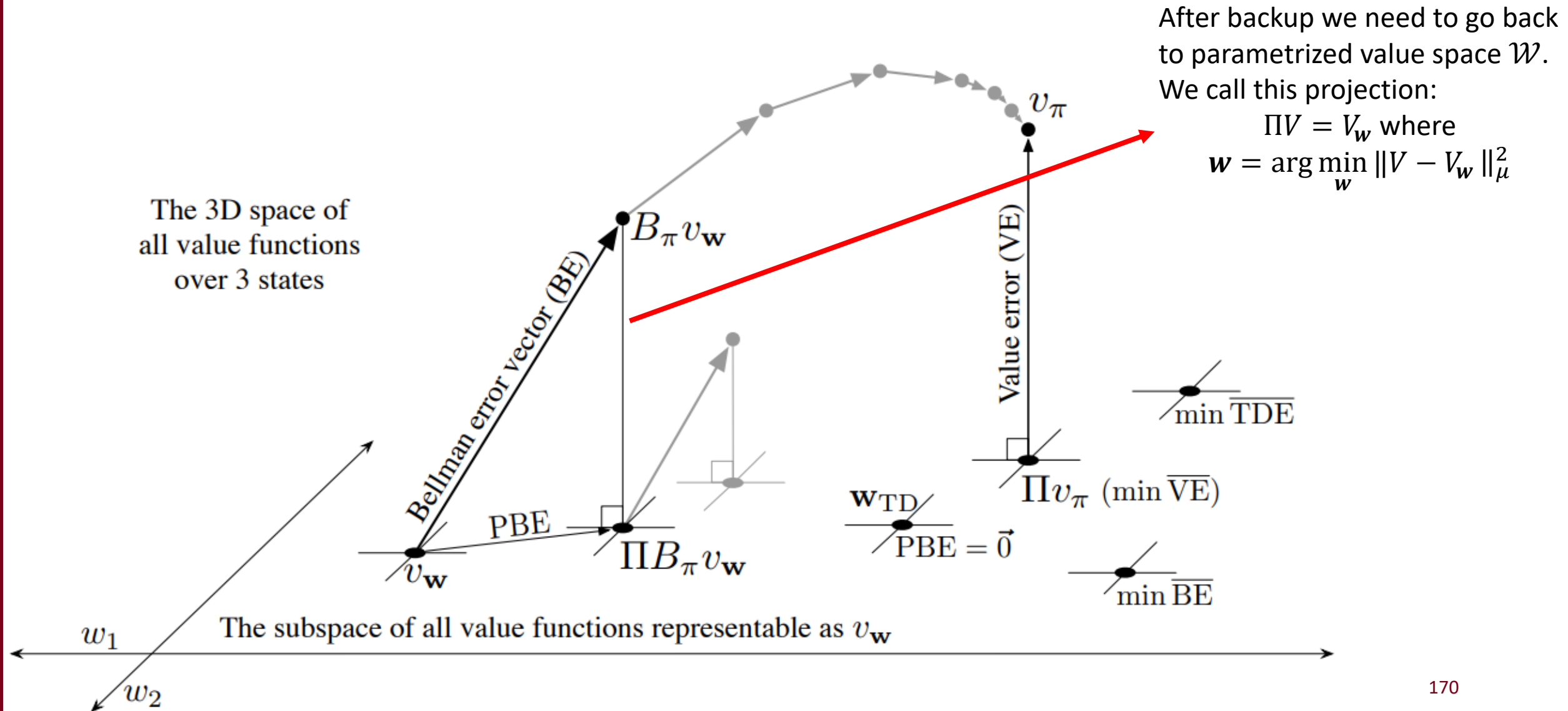
Bellman error vector

corresponds to:

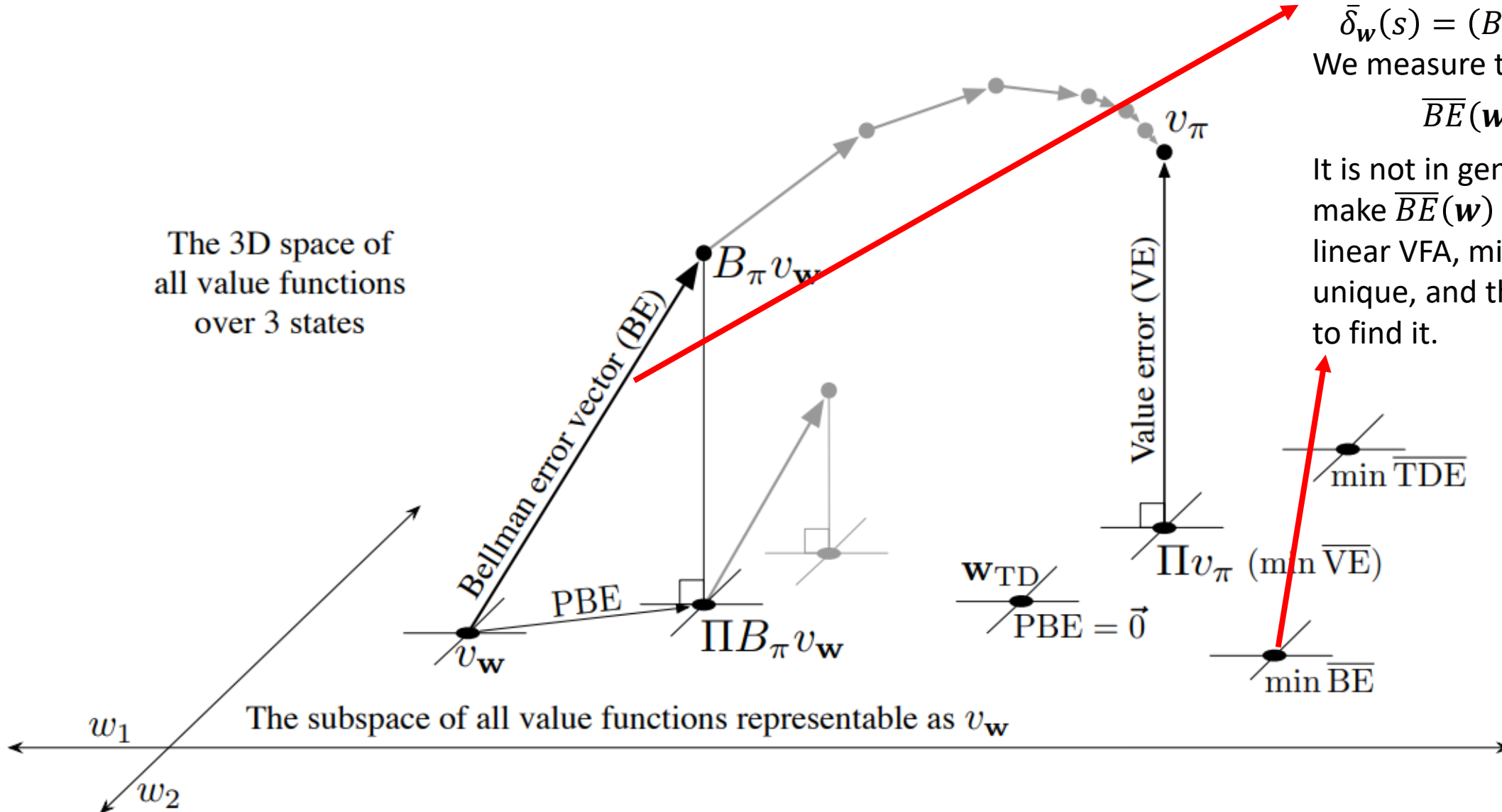
$$\bar{\delta}_w(s) = (B^\pi V_w)(s) - V_w(s)$$



Linear Value-function Geometry



Linear Value-function Geometry



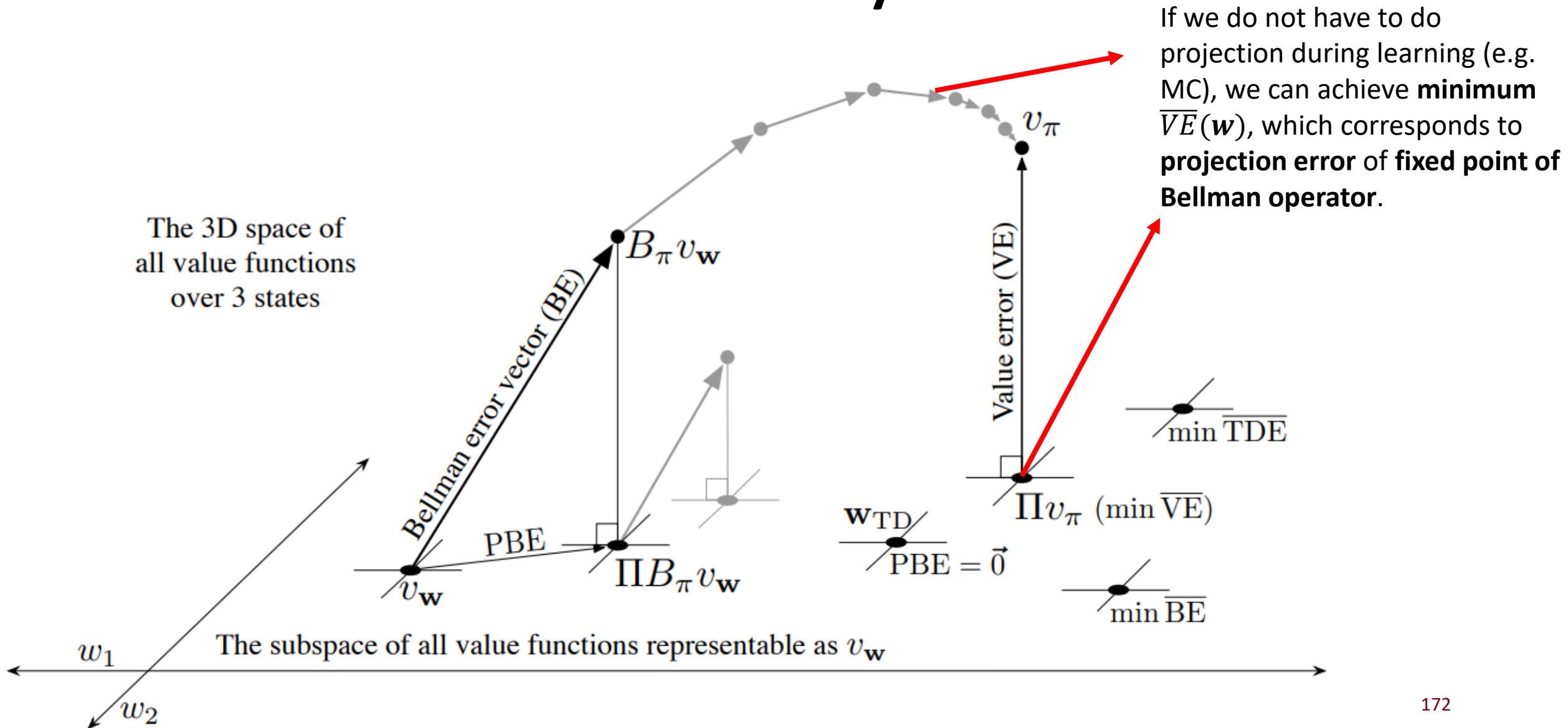
Based on the Bellman error vector:

$\bar{\delta}_w(s) = (B^\pi V_w)(s) - V_w(s)$,
We measure the Bellman error

$$\overline{BE}(w) = \|\bar{\delta}_w\|_\mu^2$$

It is not in general possible to make $\overline{BE}(w) = 0$. However, for linear VFA, minimizer of $\overline{BE}$ is unique, and there are algorithms to find it.

Linear Value-function Geometry

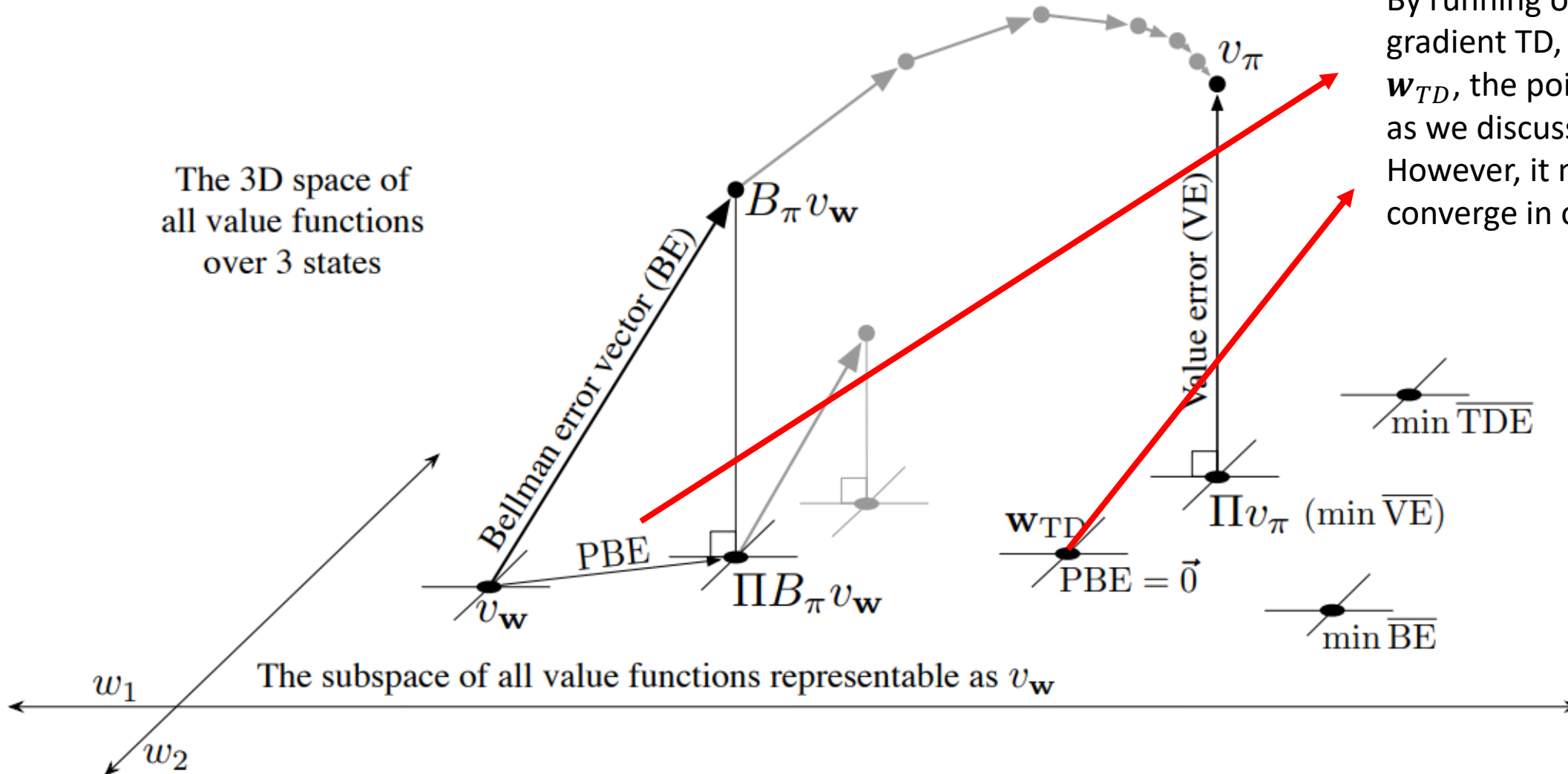


Linear Value-function Geometry

The error we get after both Bellman update and the projection is called projected Bellman error:

$$\overline{PBE}(\mathbf{w}) = \|\Pi \bar{\delta}_{\mathbf{w}}\|_{\mu}^2$$

By running on-policy semi-gradient TD, we converge to $\mathbf{w}_{TD}$, the point where $\overline{PBE} = 0$, as we discussed. However, it may not be able to converge in off-policy setting.



True SGD TD Algorithms

- So far MC algorithms were the only algorithms that are true SGD methods.
- How can we derive a true SGD TD algorithm? We need to start from an objective without true value function V^π .
- How about minimizing TD error $\delta_t = r_t + \gamma \hat{V}(s_{t+1}; \mathbf{w}) - \hat{V}(s_t; \mathbf{w})$?
- Define mean square TD error: $\overline{TDE}(\mathbf{w}) = \|\delta_t\|_\mu^2$

Additional term compared
to semi-gradient TD(0)

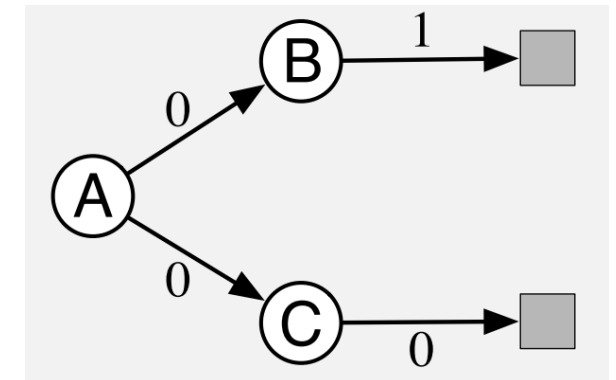


$$\Delta \mathbf{w} = -\frac{1}{2} \alpha \nabla_{\mathbf{w}}(\delta_t^2) = \alpha \left(r_t + \gamma \hat{V}(s_{t+1}; \mathbf{w}) - \hat{V}(s_t; \mathbf{w}) \right) \left(\nabla_{\mathbf{w}} \hat{V}(s_t; \mathbf{w}) - \gamma \nabla_{\mathbf{w}} \hat{V}(s_{t+1}; \mathbf{w}) \right)$$

- This is called the naïve residual-gradient algorithm.
- It converges robustly; but is this what we want?

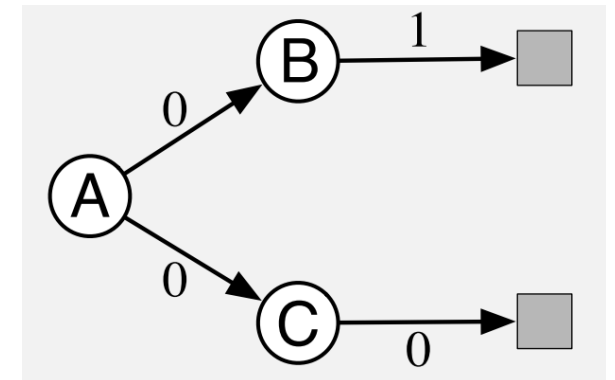
Naivety of Naïve residual-gradient algorithm

- By minimizing $\overline{TDE}$, we are minimizing $\mathbb{E}_\pi[\delta_t^2]$; But what we actually want is $\mathbb{E}_\pi[\delta_t] = 0$.
- Since $\mathbb{E}_\pi[\delta_t^2] = \text{Var}(\delta_t) + \mathbb{E}_\pi[\delta_t]^2$, apart from what we want (latter term), we are suppressing the variance as well! This makes algorithm to converge to strange place
- On the example on the right, the true value is $V(A) = \frac{1}{2}, V(B) = 1, V(C) = 0$.
- In that case:
 - Squared TD error of $A \rightarrow B$: $(V(B) - V(A))^2 = \frac{1}{4}$
 - Squared TD error of $A \rightarrow C$: $(V(C) - V(A))^2 = \frac{1}{4}$
 - Squared TD error of $B \rightarrow T, C \rightarrow T$: 0
- In average: $\overline{TDE} = \frac{1}{8}$



Naivety of Naïve residual-gradient algorithm

- By minimizing $\overline{TDE}$, we are minimizing $\mathbb{E}_\pi[\delta_t^2]$; But what we actually want is $\mathbb{E}_\pi[\delta_t] = 0$.
- Since $\mathbb{E}_\pi[\delta_t^2] = \text{Var}(\delta_t) + \mathbb{E}_\pi[\delta_t]^2$, apart from what we want (latter term), we are suppressing the variance as well! This makes algorithm to converge to strange place
- On the example on the right, the other value is $V(A) = \frac{1}{2}, V(B) = \frac{3}{4}, V(C) = \frac{1}{4}$.
- In that case:
 - Squared TD error of $A \rightarrow B: (V(B) - V(A))^2 = \frac{1}{16}$
 - Squared TD error of $A \rightarrow C: (V(C) - V(A))^2 = \frac{1}{16}$
 - Squared TD error of $B \rightarrow T, C \rightarrow T: \frac{1}{16}$
- In average: $\overline{TDE} = \frac{1}{16}$: this gives better overall loss!



Minimizing the Bellman Error

- Then, can we make an algorithm that minimizes $\mathbb{E}_\pi[\delta_t]^2$? This corresponds to minimizing $\overline{BE}(\mathbf{w})$.
- $$\begin{aligned}\Delta \mathbf{w} &= -\frac{1}{2}\alpha \nabla_{\mathbf{w}}(\mathbb{E}_\pi[\delta_t]^2) \\ &= -\alpha \mathbb{E}_\pi[\delta_t] \nabla_{\mathbf{w}} \mathbb{E}_\pi[\delta_t] \\ &= -\alpha \mathbb{E}_\pi[r_t + \gamma \hat{V}(s_{t+1}; \mathbf{w}) - \hat{V}(s_t; \mathbf{w})] \mathbb{E}_\pi[\nabla_{\mathbf{w}} \delta_t] \\ &= \alpha [\mathbb{E}_\pi[r_t + \gamma \hat{V}(s_{t+1}; \mathbf{w})] - \hat{V}(s_t; \mathbf{w})] \left[\nabla_{\mathbf{w}} \hat{V}(s_t; \mathbf{w}) - \gamma \mathbb{E}_\pi[\nabla_{\mathbf{w}} \hat{V}(s_{t+1}; \mathbf{w})] \right]\end{aligned}$$
- How do we compute this based on samples? If we use same sample for two expectations, it results in naïve residual-gradient algorithm!
- To do this, we need two independent samples of s_{t+1} (double sampling)
 - should observe one s_{t+1} , go back to s_t , observe another s_{t+1}
- This algorithm is called the **residual gradient** algorithm
 - However, double sampling is not practical in most of the cases.

Further Developments of TD algorithms

- So overall, it turns out that the objective we should go for is $\overline{PBE}$
- Semi-gradient TD can achieve $\overline{PBE} = 0$, but it is prone to divergence when off-policy
- Other algorithms that are guaranteed to converge to $\overline{PBE} = 0$, even when off-policy:
 - Least squares Temporal Difference Family (LSTD, LSPI, ...):
 - Solves least squares problem to get minimizer of $\overline{PBE}$
 - Fast convergence but quadratic complexity ($O(d^2)$) per experience sample
 - Gradient Temporal Difference Family (GTD2, GTD(0), TDC, GQ, ...)
 - Derive “true” SGD algorithm that minimizes $\overline{PBE}$
 - Requires learning of a new set of intermediate parameters (2 timescale learning)
 - Linear complexity ($O(d)$) per experience sample, but usually shows slower convergence than semi-gradient TD

Thank you!

강화학습

LG Aimers



고려대학교 인공지능학과 이병준
Lecture 5: Deep Q Learning

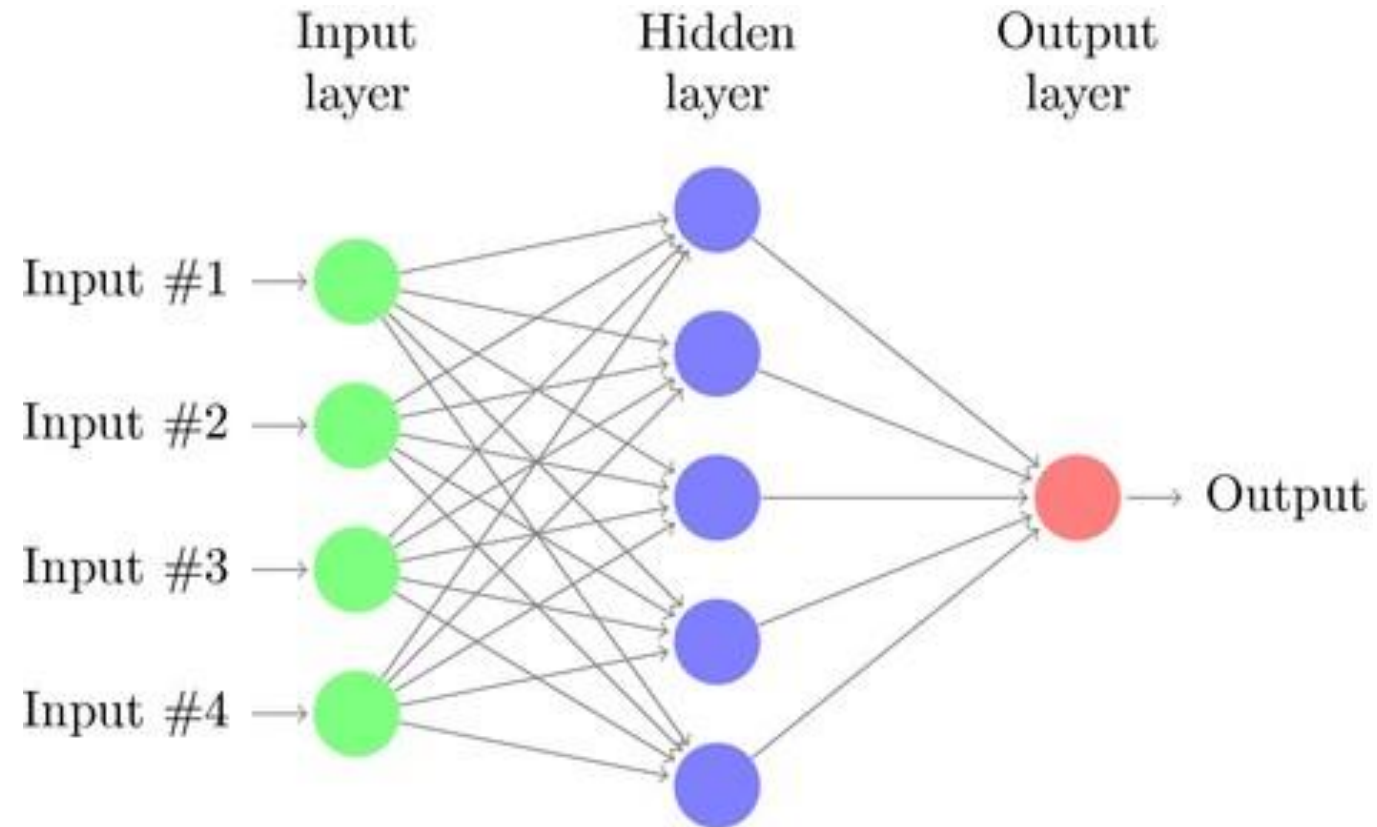
Recall: Control using Value Function Approximation

- Use value function approximation to represent state-action values $\hat{Q}^{\pi}(s, a; \mathbf{w}) \approx Q^{\pi}$
- Interleave
 - Approximate policy evaluation using value function approximation
 - Perform ϵ -greedy policy improvement
- Can be unstable. Generally involves intersection of the following:
 - Function approximation
 - Bootstrapping
 - **Off-policy learning**

RL with Function Approximation

- Linear value function approximators assume value function is a weighted combination of a set of features, where each feature a function of the state
- Linear VFA often work well given the right set of features
- But can require carefully hand designing that feature set
- An alternative is to use a much richer function approximation class that is able to directly go from states without requiring an explicit specification of features
- Local representations including Kernel based approaches have some appealing properties (including convergence results under certain cases) but can't typically scale well to enormous spaces and datasets

Neural Networks



Deep Neural Networks (DNN)

- Composition of multiple functions
- Can use the chain rule to backpropagate the gradient
- Major innovation: tools to automatically compute gradients for a DNN
- To fit the parameters, require a loss function (MSE, log likelihood etc)

The Benefit of Deep Neural Network Approximators

- Linear VFA often work well given the right set of features
- But can require carefully hand designing that feature set
- Alternative: Deep neural networks
 - Uses distributed representations instead of local representations
 - Universal function approximator
 - Can potentially need exponentially less nodes/parameters (compared to a shallow net) to represent the same function
 - Can learn the parameters using stochastic gradient descent

Why Do We Care About CNNs?

- CNNs extensively used in computer vision
- If we want to go from pixels to decisions, likely useful to leverage insights for visual input
- Images have local structure and correlation
- Images have distinctive features in frequency domains



Convolutional NN

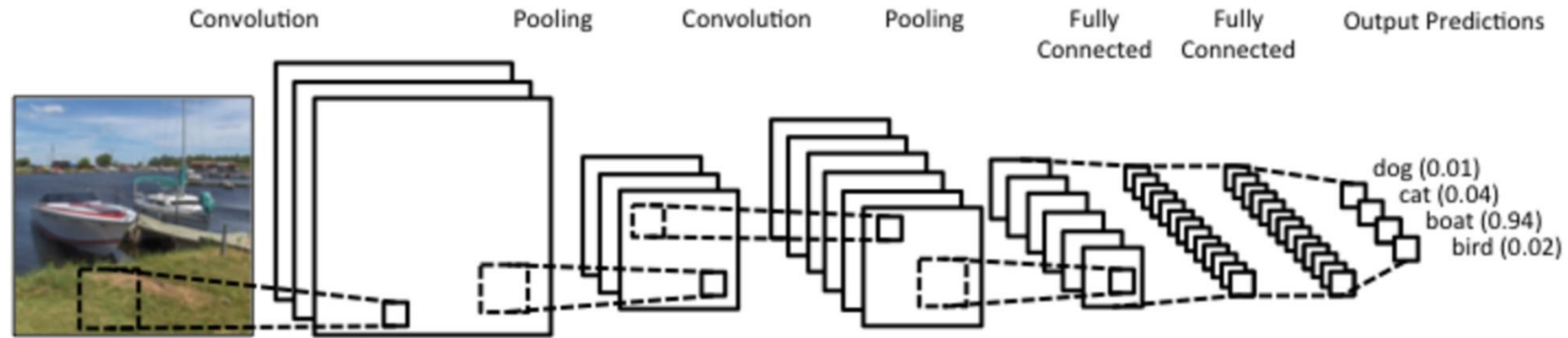
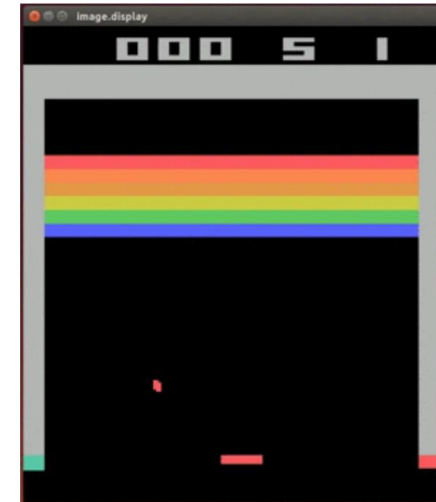


Image: <http://d3kbpzbmcyntmx.cloudfront.net/wp-content/uploads/2015/11/Screen-Shot-2015-11-07-at-7.26.20-AM.png>

- Consider local structure and common extraction of features
- Not fully connected
- Locality of processing
- Weight sharing for parameter reduction
- Learn the parameters of multiple convolutional filter banks
- Compress to extract salient features & favor generalization

Generalization

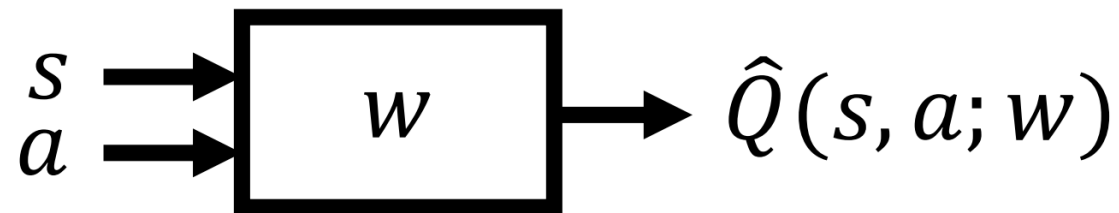
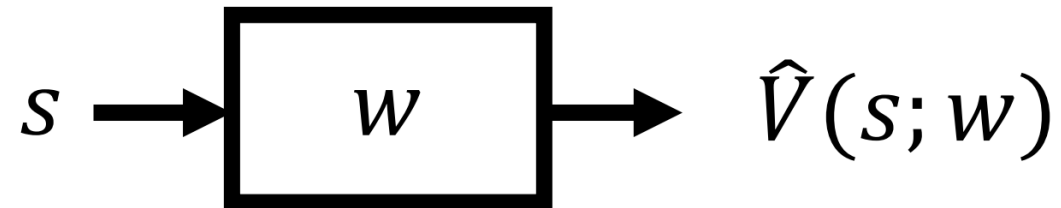
- Back to RL...
- Using function approximation to help scale up to making decisions in really large domains
- Deep RL use deep neural networks to represent
 - Value, Q function
 - Policy
 - Model



Deep Q-Networks (DQNs)

- Represent state-action value function by Q-network with weights $\mathbf{w}$

$$\hat{Q}(s, a; \mathbf{w}) \approx Q(s, a)$$



Recall: Incremental Model-Free Control Approaches

- Similar to policy evaluation, true state-action value function for a state is unknown and so substitute a target value
- For MC control, use a return G_t as a substitute target

$$\Delta \mathbf{w} = \alpha \left(\mathbf{G}_t - \hat{Q}(s_t, a_t; \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{Q}(s_t, a_t; \mathbf{w})$$

- For SARSA, use TD target that leverages the current function approximation value

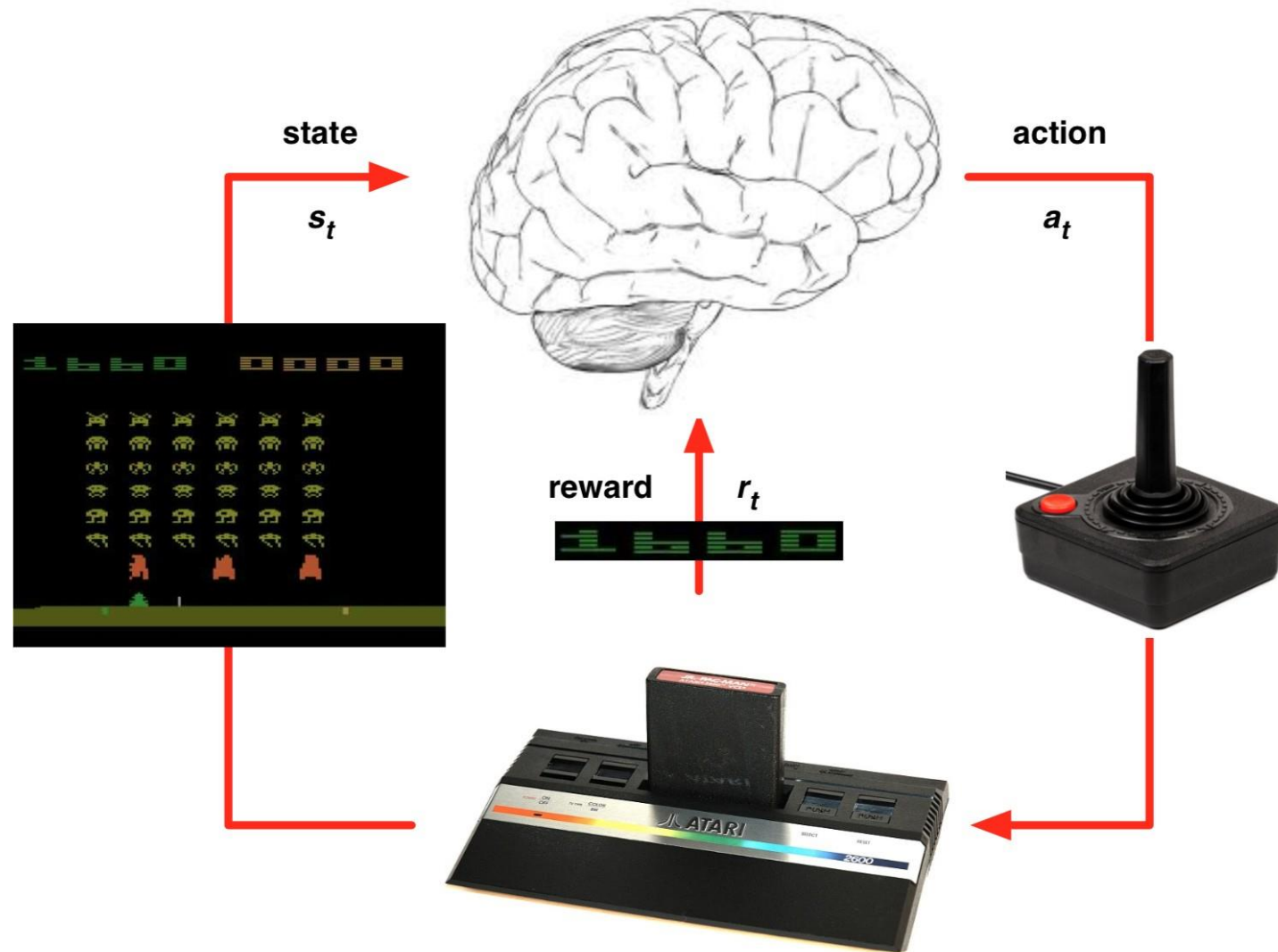
$$\Delta \mathbf{w} = \alpha \left(r_t + \gamma \hat{Q}(s_{t+1}, a_{t+1}; \mathbf{w}) - \hat{Q}(s_t, a_t; \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{Q}(s_t, a_t; \mathbf{w})$$

- For Q-learning, use greedy TD target that leverages the current function approximation value

$$\Delta \mathbf{w} = \alpha \left(r_t + \gamma \max_{a'} \hat{Q}(s_{t+1}, a'; \mathbf{w}) - \hat{Q}(s_t, a_t; \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{Q}(s_t, a_t; \mathbf{w})$$

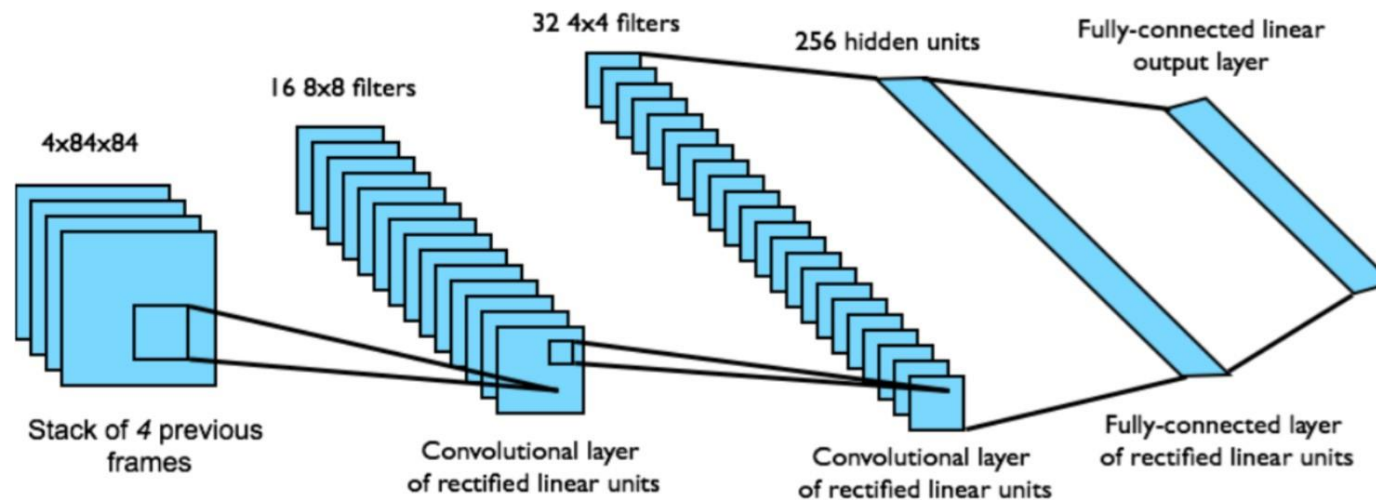
- Note: SARSA and Q-learning are also semi-gradient methods!

Using these ideas to Deep RL in Atari



Deep RL in ATARI

- End-to-end learning of values $Q(s, a)$ from pixels s
- Input state s is stack of raw pixels from last 4 frames (why?)
- Output is $Q(s, a)$ for 18 joystick/button positions
- Reward is change in score for that step



- Network architecture and hyperparameters fixed across all games

Q-Learning with Value Function Approximation

- Q-learning converges to the optimal $Q^*(s, a)$ using table lookup representation
- In value function approximation Q-learning we can minimize MSE loss by stochastic gradient descent using a target Q estimate instead of true Q (as we saw with linear VFA)
 - Which resulted in minimizing $\overline{PBE}$ by semi-gradient TD
 - But Q-learning with VFA can diverge
- Although there are plenty reasons on divergence, Deep Q-learning (DQN) sees that the issues are:
 - Correlations between samples
 - Non-stationary targets
- What does these mean?

Recall: Q-learning with Value Function Approximation

- For Q-learning, use greedy TD target that leverages the current function approximation value

$$\Delta \mathbf{w} = \alpha \left(\underbrace{r_t + \gamma \max_{a'} \hat{Q}(s_{t+1}, a'; \mathbf{w})}_{\text{target}} - \hat{Q}(s_t, a_t; \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{Q}(s_t, a_t; \mathbf{w})$$

- How is it different from usual SGD? What can we do to make it work more like SGD?
 1. Training target is dependent on $\mathbf{w}$ as well, making the gradient invalid
 - Can we make it somehow independent to $\mathbf{w}$, while estimating Q^* ?
 2. Subsequent updates are correlated to each other
 - We assume i.i.d. data for SGD, but experience samples in a trajectory are highly correlated
- If we fix those issues, the algorithm will work more like SGD! (stable local convergence)

DQNs: Experience Replay

- To help remove correlations, store dataset (called a replay buffer) $\mathcal{D}$ from prior experience

s_1, a_1, r_1, s_2	$\rightarrow s, a, r, s'$
s_2, a_2, r_2, s_3	
s_3, a_3, r_3, s_4	
...	
s_t, a_t, r_t, s_{t+1}	

- To perform experience replay, repeat the following:
 - $a_t \sim \pi_b(a|s)$, $(r_t, s_{t+1}) \sim P(r, s' | s_t, a_t)$: act according to the behavior policy that explores well
 - $\mathcal{D}' = \mathcal{D} \cup \{(s_t, a_t, r_t, s_{t+1})\}$: store the experience to the dataset
 - $(s, a, r, s') \sim \mathcal{D}$: sample an experience tuple from the dataset
 - Update $\mathbf{w}$ based on the sampled experience

DQNs: Fixed Q-Targets

- To help improve stability, fix the **target weights** used in the target calculation for multiple updates
- Target network uses a different set of weights than the weights being updated
- Let parameters $\mathbf{w}^-$ be the set of weights used in the target, and $\mathbf{w}$ be the weights that are being updated
- Slight change to computation of target value:

$$\Delta \mathbf{w} = \alpha \left(\underbrace{r + \gamma \max_{a'} \hat{Q}(s', a'; \mathbf{w}^-)}_{\text{target}} - \hat{Q}(s, a; \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{Q}(s, a; \mathbf{w})$$

- As long as target weight $\mathbf{w}^-$ is fixed, target is independent to $\mathbf{w}$, and the update above forms a valid **SGD**!
 - Which guarantees convergence to the local optimum
- Cost: doubled memory requirements

DQN

Pseudocode

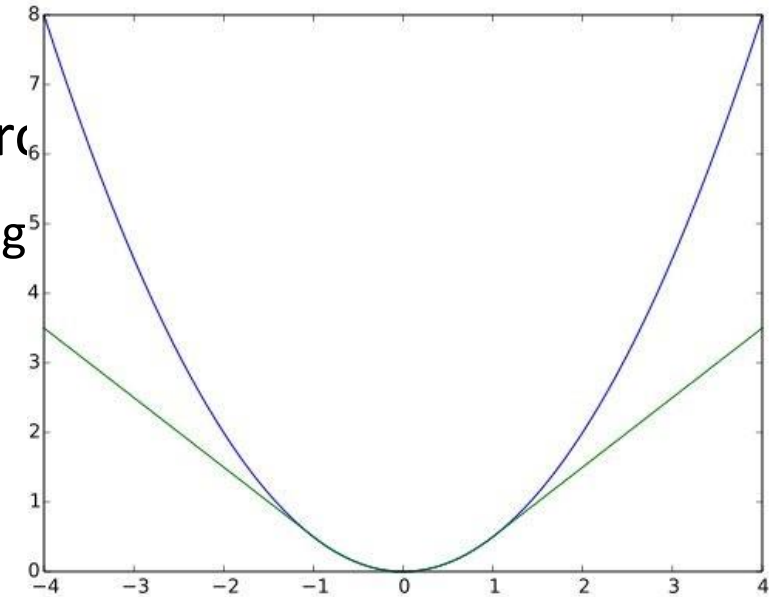
```
1: Initialize  $C, \alpha, \mathcal{D} = \{\}$ , Initialize  $\mathbf{w}, \mathbf{w}^- = \mathbf{w}, t = 0$ 
2: Get initial state  $s_0$ 
3: Loop
4:     Sample action  $a_t$  given  $\epsilon$ -greedy policy for current  $\hat{Q}(s_t, a; \mathbf{w})$ 
5:     Observe reward  $r_t$  and next state  $s_{t+1}$ 
6:     Store transition:  $\mathcal{D} = \mathcal{D} \cup \{(s_t, a_t, r_t, s_{t+1})\}$ 
7:     Sample random minibatch of tuples  $(s_i, a_i, r_i, s_{i+1})$  from  $\mathcal{D}$ 
8:     For  $i$  in minibatch do
9:          $y_i = r_i$  if episode terminated else  $y_i = \gamma \max_{a'} \hat{Q}(s_{i+1}, a'; \mathbf{w}^-)$ 
10:        Update weights:
            
$$\mathbf{w} \leftarrow \mathbf{w} + \alpha [y_i - \hat{Q}(s_i, a_i; \mathbf{w})] \nabla_{\mathbf{w}} \hat{Q}(s_i, a_i; \mathbf{w})$$

11:    End for
12:     $t = t + 1$ 
13:    If  $\text{mod}(t, C) == 0$  then
14:         $\mathbf{w}^- \leftarrow \mathbf{w}$ 
15:    End If
16: End loop
```

Other DQN details

- Uses Huber loss instead of squared loss on Bellman error
 - Huber loss helps stabilize learning; similar to gradient clipping

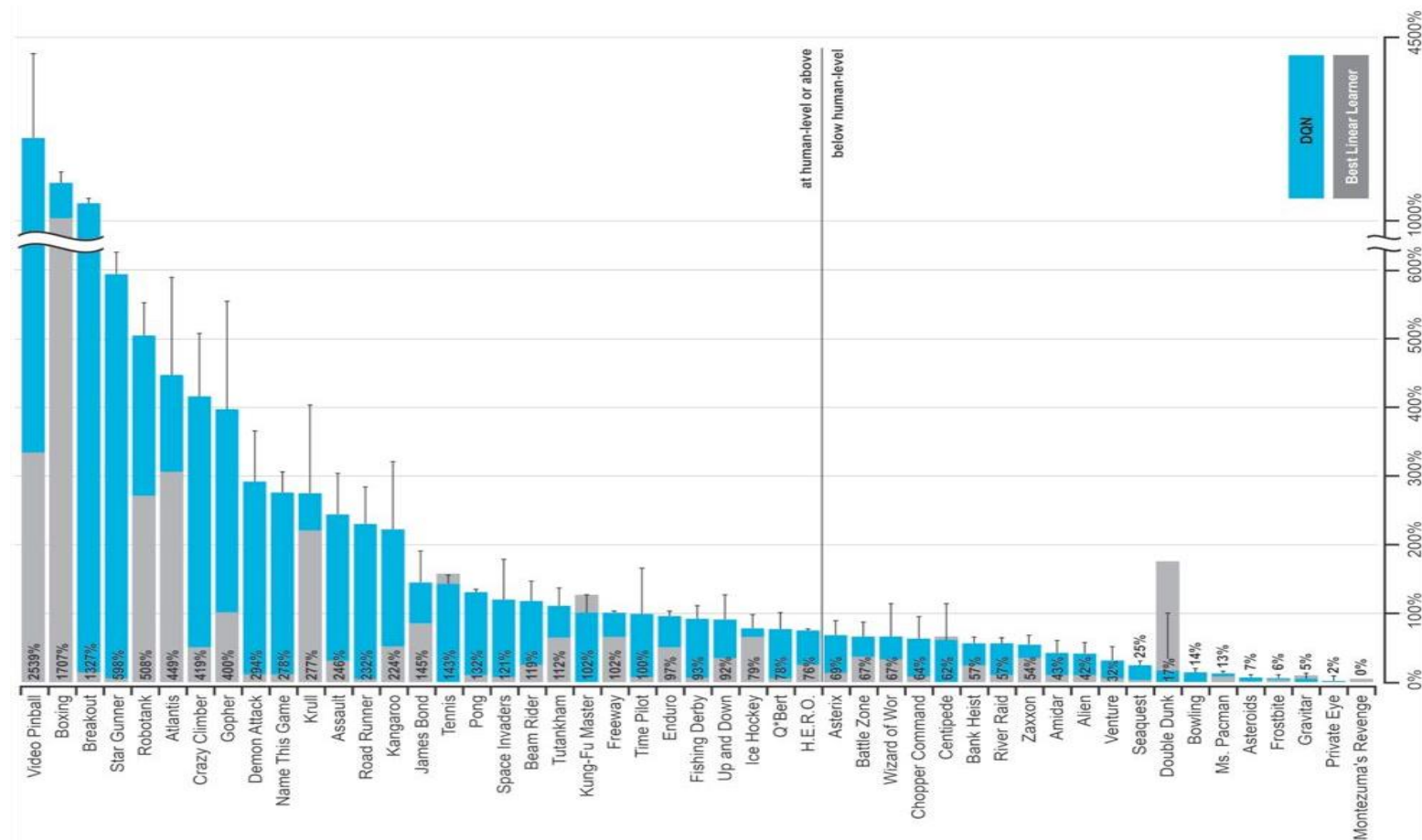
$$L_{\delta}(a) = \begin{cases} \frac{1}{2}a^2 & \text{for } |a| \leq \delta, \\ \delta(|a| - \frac{1}{2}\delta), & \text{otherwise.} \end{cases}$$



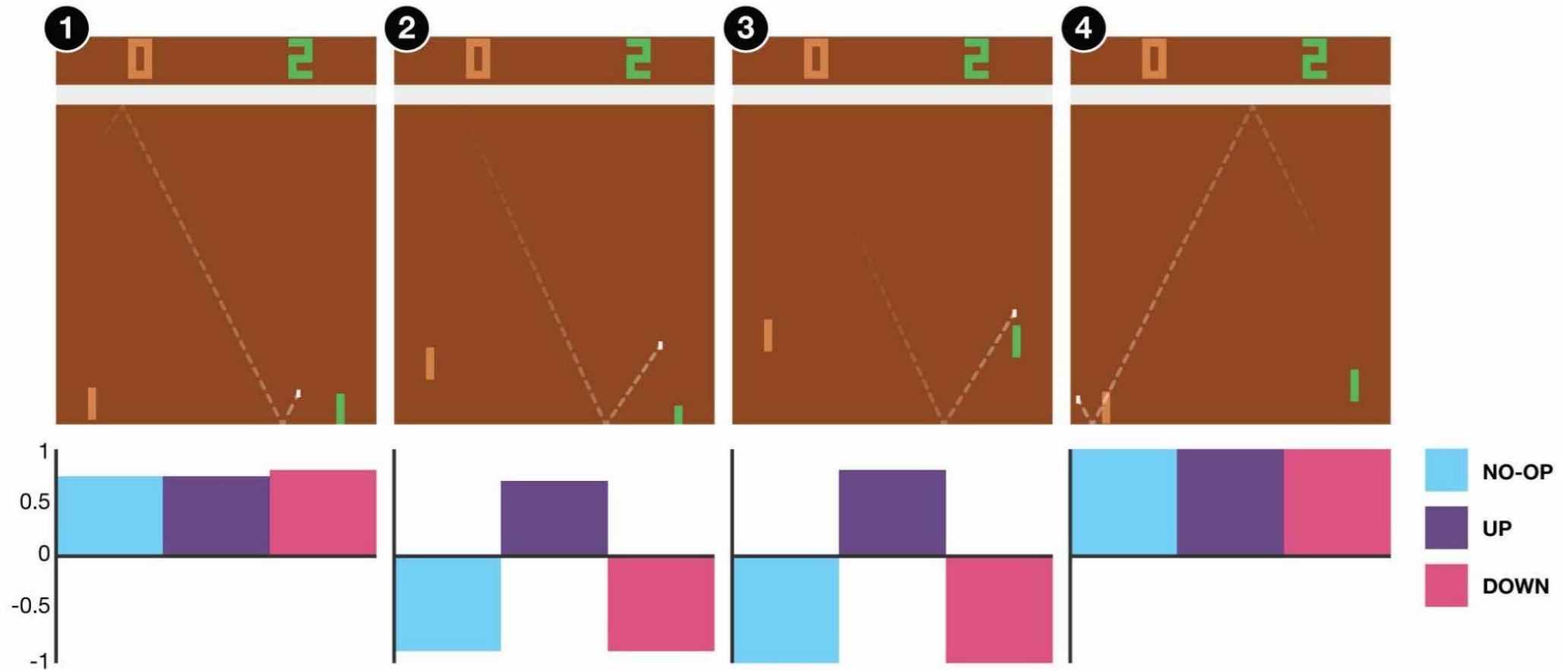
- Uses RMSProp instead of vanilla SGD.
- Anneal the exploration rate:
 - Start ϵ at 1 and anneal it to 0.1 or 0.05 over the first million frames.

ATARI Results

- 49 ATARI 2600 games
- Same algorithm
- Same function approximator
 - 3M free parameters
- Same hyperparameter
- Human level performance on 29 out of 49 games
- Outperforms best linear learner so far



Action Values on Pong



Which Aspects of DQN were Important for Success?

Game	Linear	Deep Network
Breakout	3	3
Enduro	62	29
River Raid	2345	1453
Seaquest	656	275
Space Invaders	301	302

- Note: just using a deep NN actually hurt performance sometimes!

Which Aspects of DQN were Important for Success?

Game	Linear	Deep Network	DQN with target Q
Breakout	3	3	10
Enduro	62	29	141
River Raid	2345	1453	2868
Seaquest	656	275	1003
Space Invaders	301	302	373

- With target Q, DQN begins to outperform linear features; but not much

Which Aspects of DQN were Important for Success?

Game	Linear	Deep Network	DQN with target Q	DQN with replay	DQN with all
Breakout	3	3	10	241	317
Enduro	62	29	141	831	1007
River Raid	2345	1453	2868	4102	7447
Seaquest	656	275	1003	823	2894
Space Invaders	301	302	373	826	1089

- Replay is **hugely** important
- Why? Beyond helping with correlation between samples, what does replaying do?
 - Reusing data – by using experience replay, we are training with a trajectory distribution sampled with many different policies
 - But no one knows the exact answer

Deep RL

- Success in Atari has led to huge excitement in using deep neural networks to do value function approximation in RL
- Some immediate improvements (many others!)
 - Double DQN (Deep Reinforcement Learning with Double Q-Learning, Van Hasselt et al, AAAI 2016)
 - Prioritized Replay (Prioritized Experience Replay, Schaul et al, ICLR 2016)
 - Dueling DQN (best paper ICML 2016) (Dueling Network Architectures for Deep Reinforcement Learning, Wang et al, ICML 2016)

Double DQN

- Recall maximization bias challenge
 - Max of the estimated state-action values can be a biased estimate of the max
- Double Q-learning

Recall: Double Q-Learning

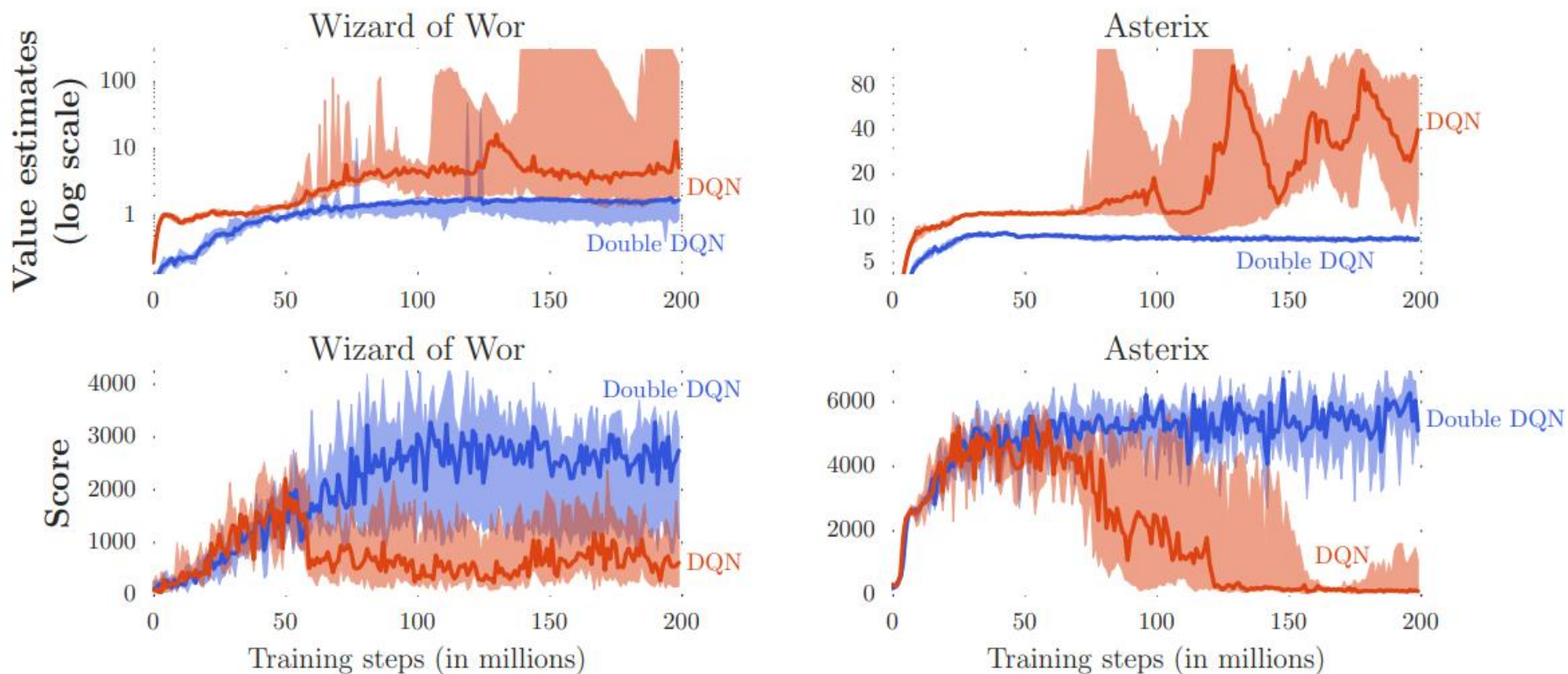
- 1: Initialize $Q_1(s, a)$ and $Q_2(s, a), \forall s \in S, a \in A$
- 2: Initialize $t = 0, s_t = s_0$
- 3: **Loop**
- 4: Select action a_t using ϵ -greedy $\pi(s) = \arg \max_a Q_1(s_t, a) + Q_2(s_t, a)$ *// behavior policy does not matter*
- 5: Observe (r_t, s_{t+1})
- 6: **If** (with 0.5 probability) **then**
- 7: Update Q_1 : $Q_1(s_t, a_t) \leftarrow Q_1(s_t, a_t) + \alpha \left(r_t + \gamma Q_2 \left(s_{t+1}, \arg \max_a Q_1(s_{t+1}, a) \right) - Q_1(s_t, a_t) \right)$
- 8: **Else**
- 9: Update Q_2 : $Q_2(s_t, a_t) \leftarrow Q_2(s_t, a_t) + \alpha \left(r_t + \gamma Q_1 \left(s_{t+1}, \arg \max_a Q_2(s_{t+1}, a) \right) - Q_2(s_t, a_t) \right)$
- 10: **End if**
- 9: $t = t + 1$
- 10: **End loop**

Double DQN

- Extend this idea to DQN
- Current Q-network $\mathbf{w}$ is used to select actions
- Older Q-network $\mathbf{w}^-$ is used to evaluate actions

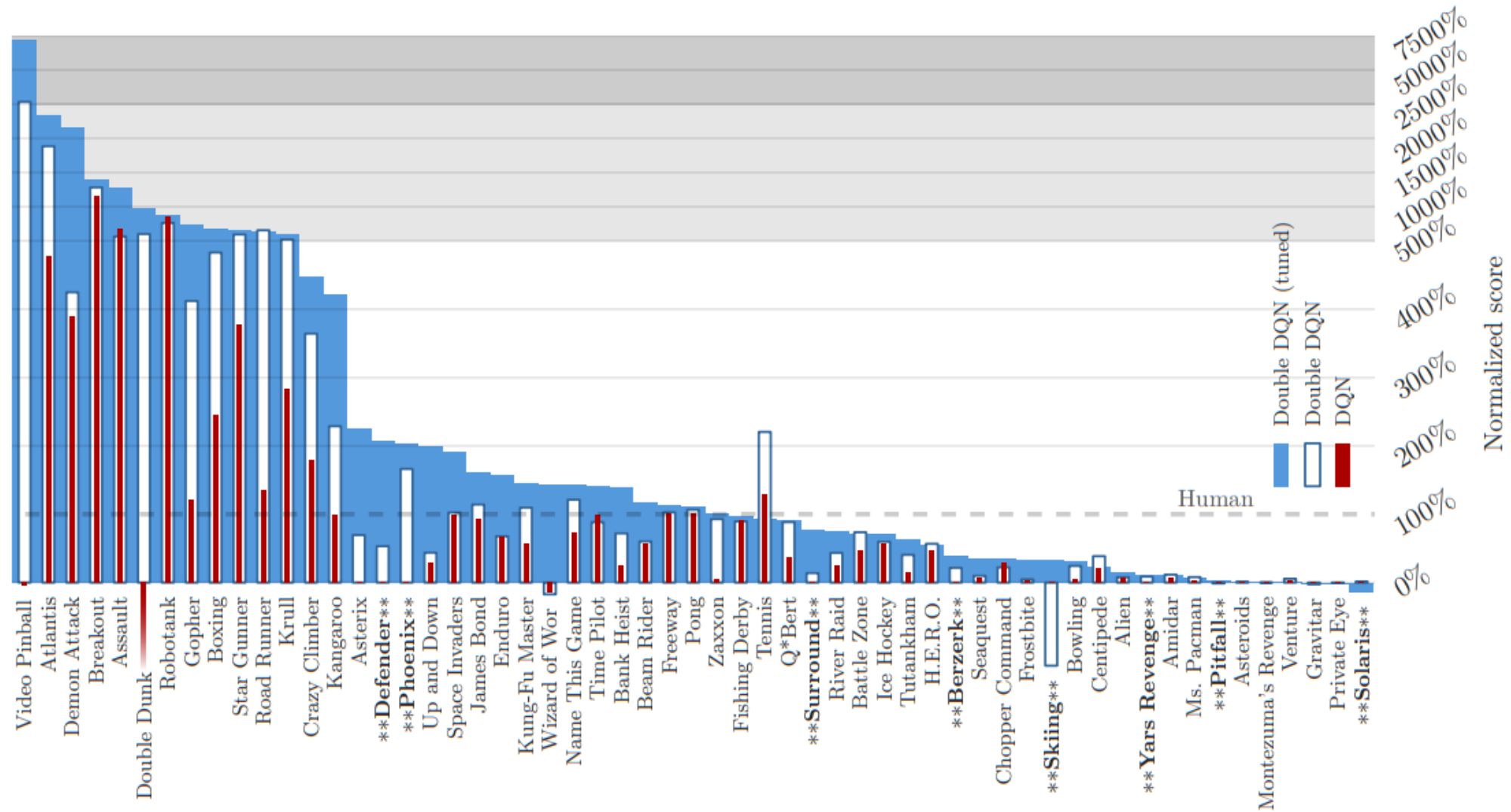
$$\Delta \mathbf{w} = \alpha \left(r + \gamma \overbrace{\hat{Q} \left(s', \underbrace{\arg \max_{a'} \hat{Q}(s', a'; \mathbf{w})}_{\text{Action selection: } \mathbf{w}}; \mathbf{w}^- \right)}^{\text{Action evaluation: } \mathbf{w}^-} - \hat{Q}(s, a; \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{Q}(s, a; \mathbf{w})$$

Double DQN



- Figure shows that DQN overestimates value a lot.

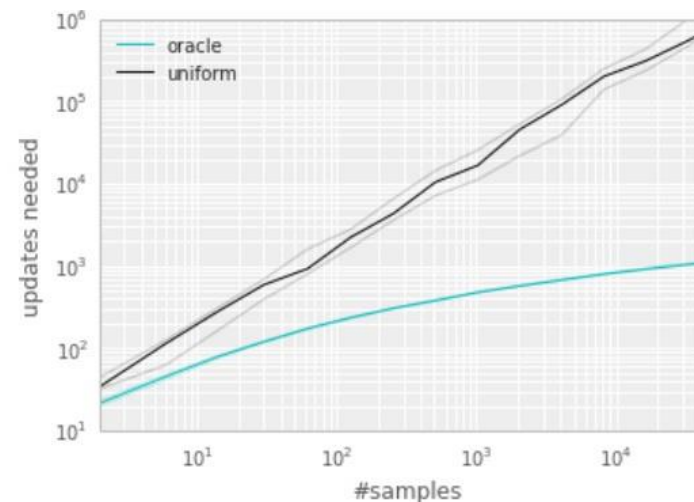
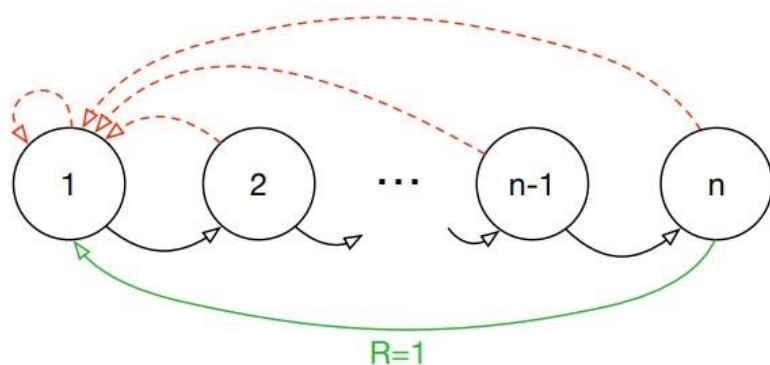
Double DQN Performance



Impact of Replay?

- In tabular TD-learning, **order** of replaying updates could help speed learning
- Repeating some updates seems to better propagate info than others
- Systematic ways to prioritize updates?

Potential Impact of Ordering Episodic Replay Updates



- Schaul, Quan, Antonoglou, Silver ICLR 2016
- Oracle: picks (s, a, r, s') tuple to replay that will minimize global loss
- Exponential improvement in convergence
 - Number of updates needed to converge
- Oracle is not a practical method but illustrates impact of ordering

Prioritized Experience Replay

- Let i be the index of the i^{th} tuple of experience (s_i, a_i, r_i, s_{i+1})
- Sample tuples for update using priority function
- Priority of a tuple i is proportional to DQN error

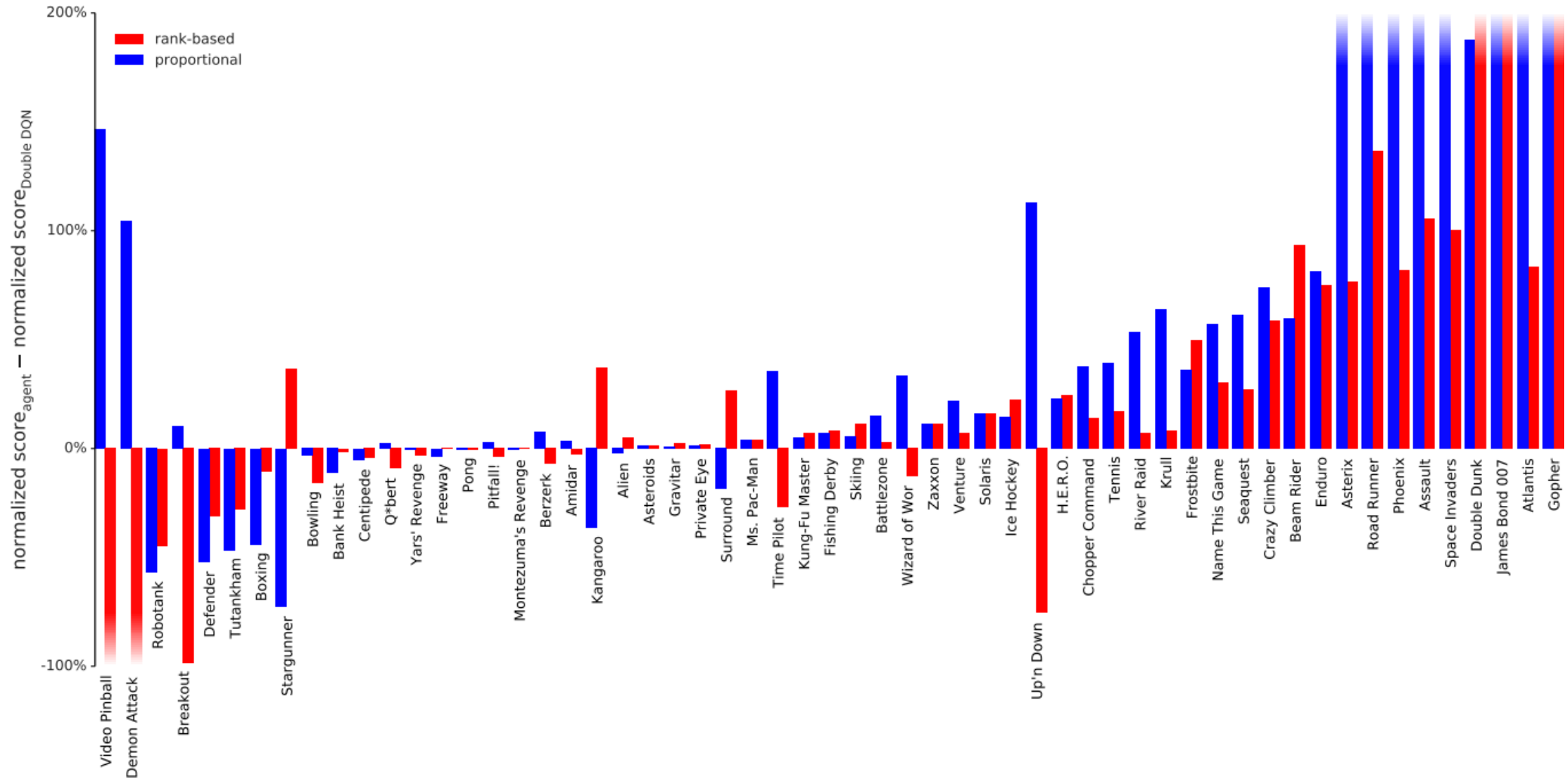
$$p_i = |r + \gamma \max_{a'} Q(s_{i+1}, a'; \mathbf{w}^-) - Q(s_i, a_i; \mathbf{w})|$$

- Update p_i every update. p_i for new tuples is set to maximum value
- One method: proportional (stochastic prioritization with hyperparameter β)

$$P(i) = \frac{p_i^\beta}{\sum_k p_k^\beta}$$

- $\beta = 0$ yields rule of random selection (vanilla DQN)

Performance of Prioritized Replay vs Double DQN

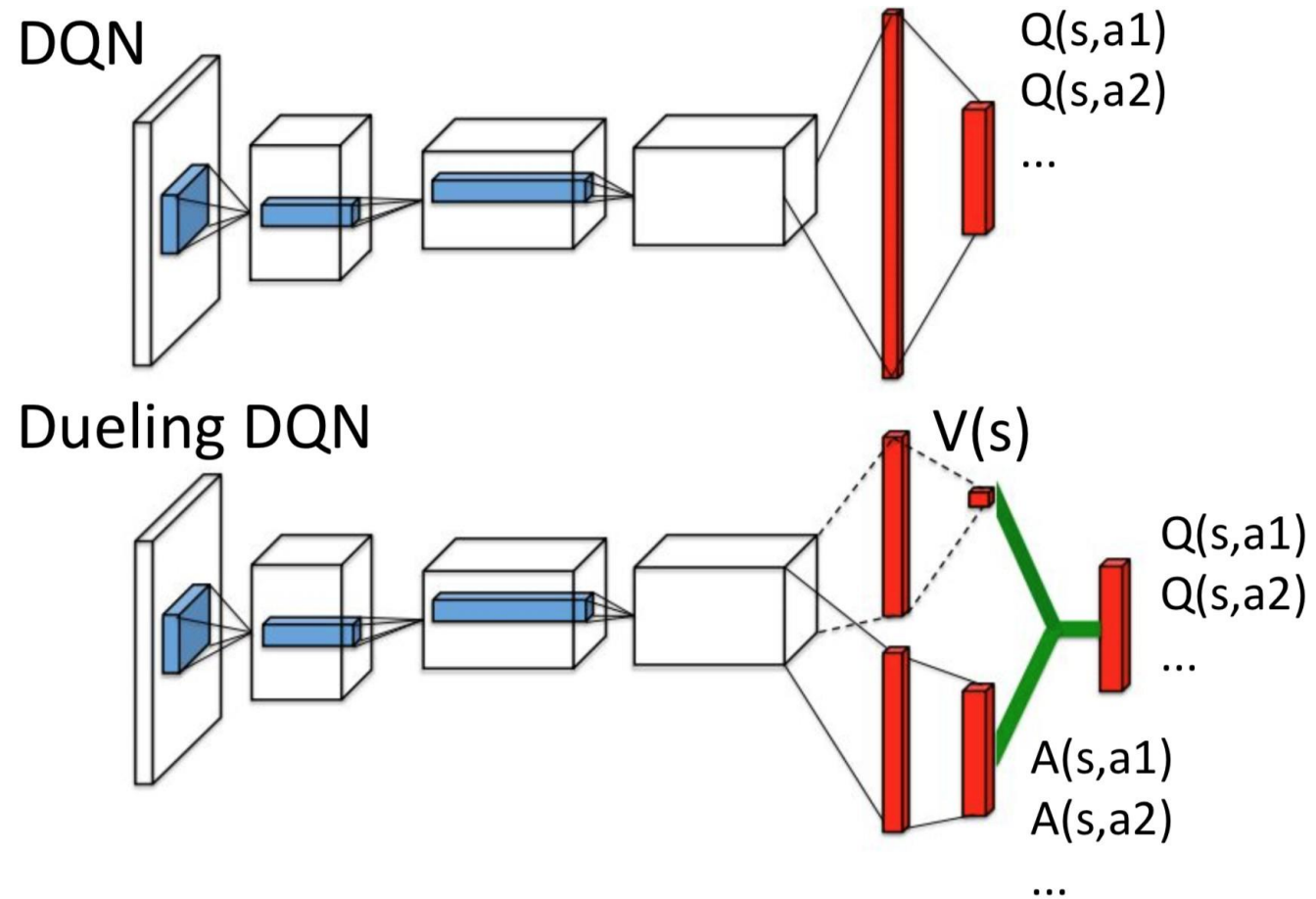


Value & Advantage Function

- Intuition: Features needed to accurately represent value may be different than those needed to specify difference in actions
- E.g.
 - Game score may help accurately predict $V(s)$
 - But not necessarily in indicating relative action values $Q(s, a_1)$ vs $Q(s, a_2)$
- Advantage function (Baird 1993)

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$$

Dueling DQN



Advantage Function and Training

- Advantage function

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$$

- Consider a network that outputs $V(s; \theta, \beta)$ as well as advantage $A(s, a; \theta, \lambda)$ where θ, β and λ are parameters
- To construct Q , we could use $Q(s, a; \theta, \beta, \lambda) = V(s; \theta, \beta) + A(s, a; \theta, \lambda)$
- Do we expect that this architecture will result in learning a good estimate of true V or A ?
- For a given Q function, is there a unique A advantage function and V ?
 - No. For example, we can shift things by a constant, and create infinitely many A and V .

Uniqueness

- How can we make V and A unique?
- **Option 1:** Force $Q(s, a) = V(s)$ for the best action suggested by the advantage:

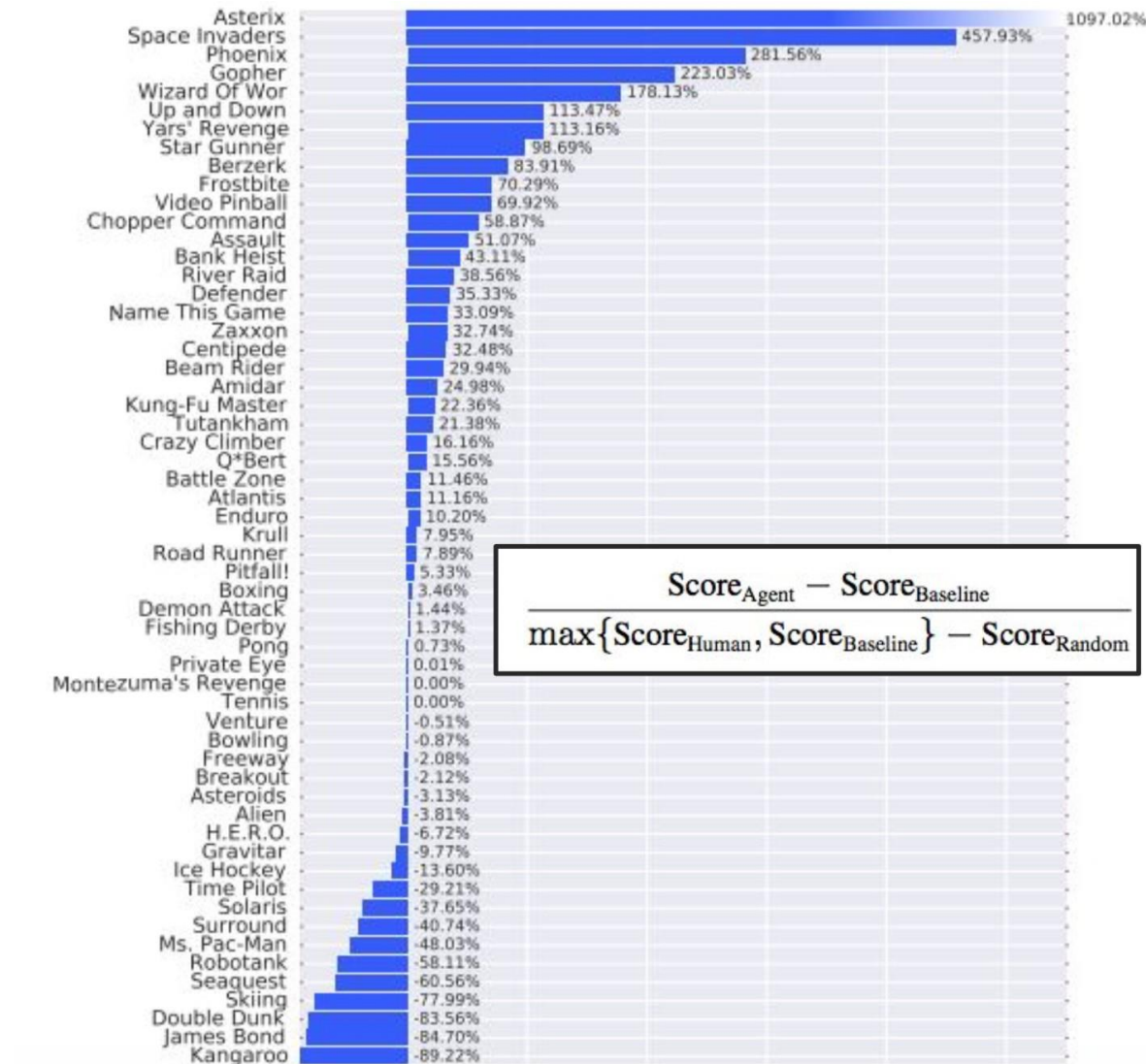
$$\hat{Q}(s, a) = \hat{V}(s) + \left(\hat{A}(s, a) - \max_{a' \in \mathcal{A}} \hat{A}(s, a') \right)$$

- This helps force the V network to approximate V
- **Option 2:** Use mean as baseline (more stable)

$$\hat{Q}(s, a) = \hat{V}(s) + \left(\hat{A}(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a' \in \mathcal{A}} \hat{A}(s, a') \right)$$

- More stable often because averaging over all advantages instead of the advantage of the current max action

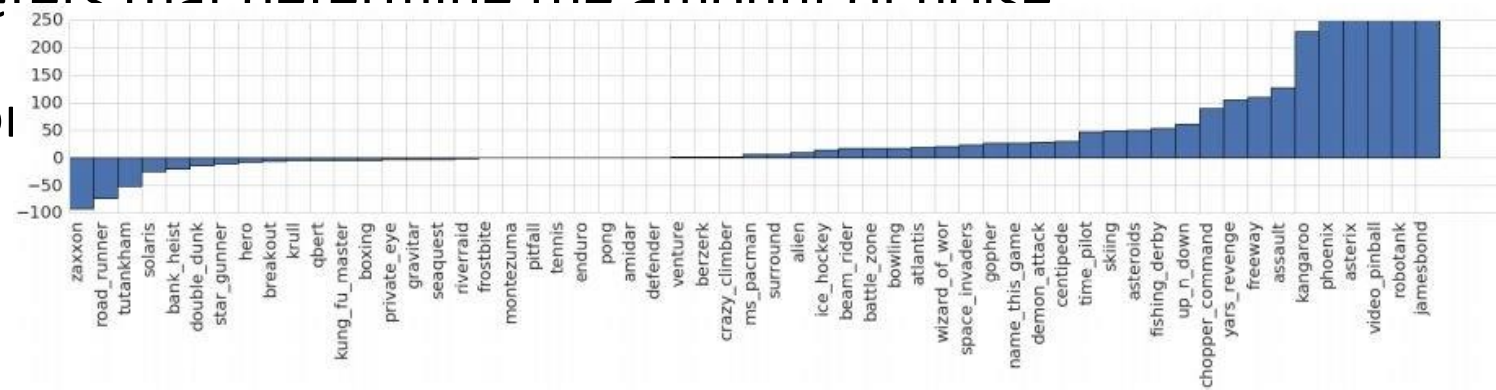
Dueling DQN V.S. Double DQN with Prioritized Replay



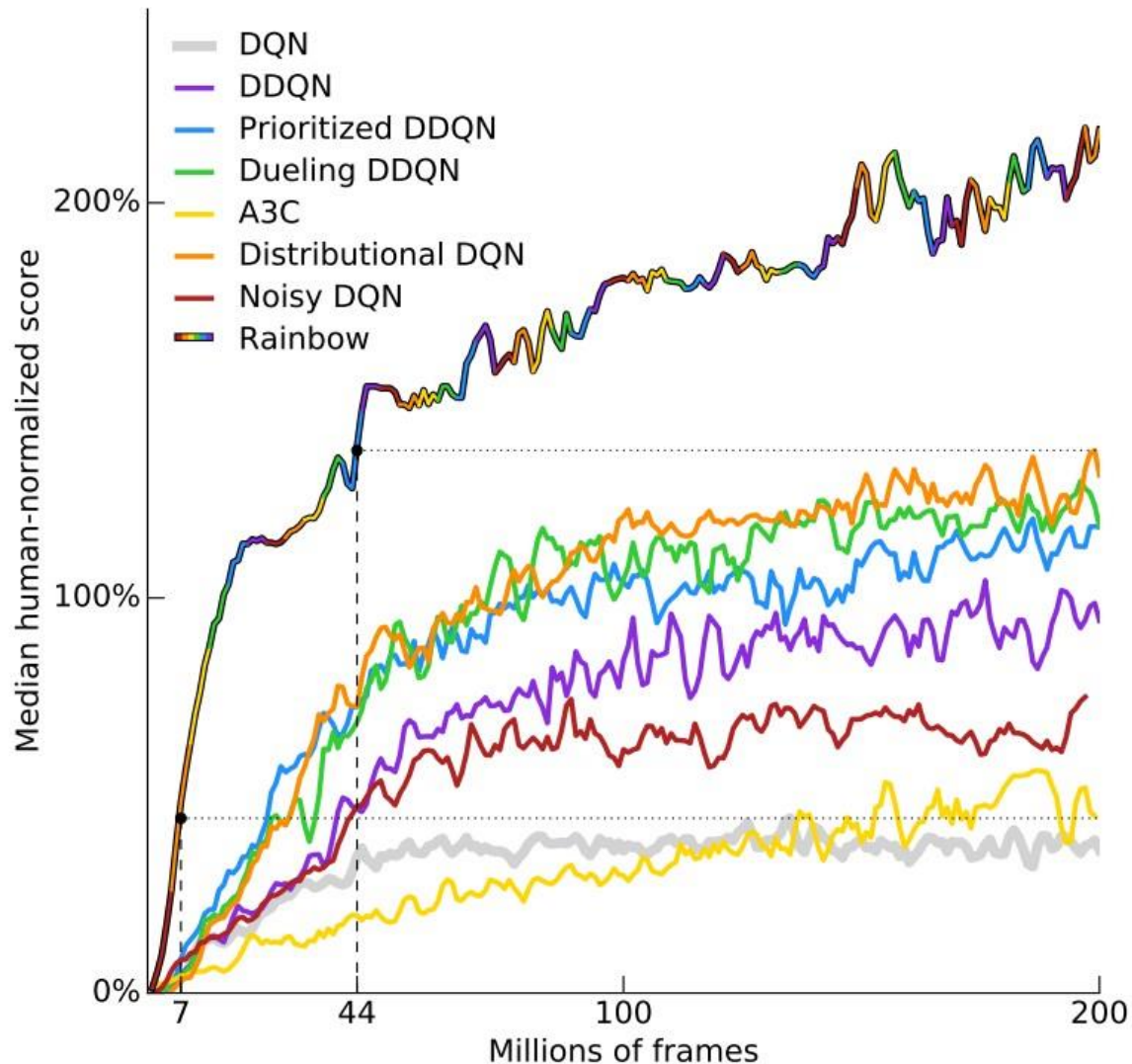
Noisy Nets for Exploration

- Add noise to network parameters for better exploration [Fortunato, Azar, Piot et al. (2017)].
- Standard linear layer: $y = wx + b$
- Noisy linear layer: $y \stackrel{\text{def}}{=} (w + \sigma^w \odot \epsilon^w)x + b + \sigma^b \odot \epsilon^b$
- ϵ^w and ϵ^b contain noise
- σ^w and σ^b are learned parameters that determine the amount of noise

	Baseline		NoisyNet	
	Mean	Median	Mean	Median
DQN	213	47	1210	89
A3C	418	93	1112	121
Dueling	2102	126	1908	154



Value-based Deep RL 2018



- DDQN = Double DQN
- A3C: actor-critic algorithm with asynchronous updates (we may cover this later)
 - We can simply understand it as n-step learning here
- Distributional DQN: learns distribution of discounted return, instead of estimating the mean
- This is very active area of research: what can we do more?

강화학습

LG Aimers



고려대학교 인공지능학과 이병준
Lecture 6: Policy Gradient

Policy-based Reinforcement Learning

- In the last lecture we approximated the value or action-value function using parameters w (changing notations a bit),

$$V_w(s) \approx V^\pi(s)$$

$$Q_w(s, a) \approx Q^\pi(s, a)$$

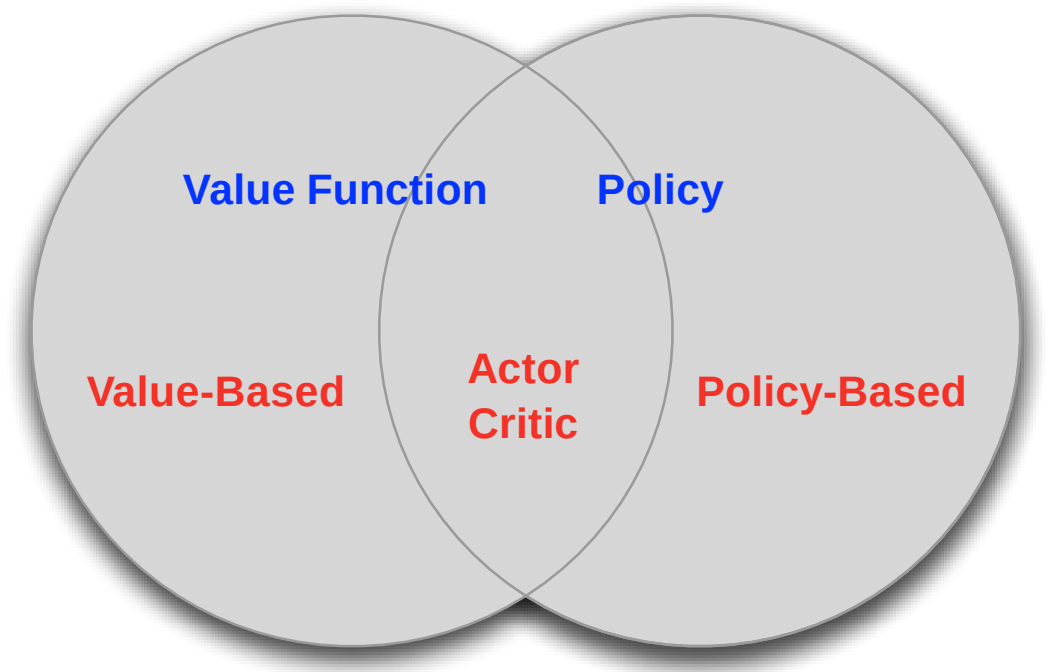
- A policy was generated directly from the value function
 - e.g. using ϵ -greedy
- In this lecture we will directly parametrize the **policy** using parameters θ

$$\pi_\theta(s, a) = P(a|s, \theta)$$

- We will focus again on **model-free** reinforcement learning

Value-Based and Policy-Based RL

- Value Based
 - Learnt Value Function
 - Implicit policy (e.g. ϵ -greedy)
- Policy Based
 - No Value Function
 - Learnt Policy
- Actor-Critic
 - Learnt Value Function
 - Learnt Policy



Advantages of Policy-Based RL

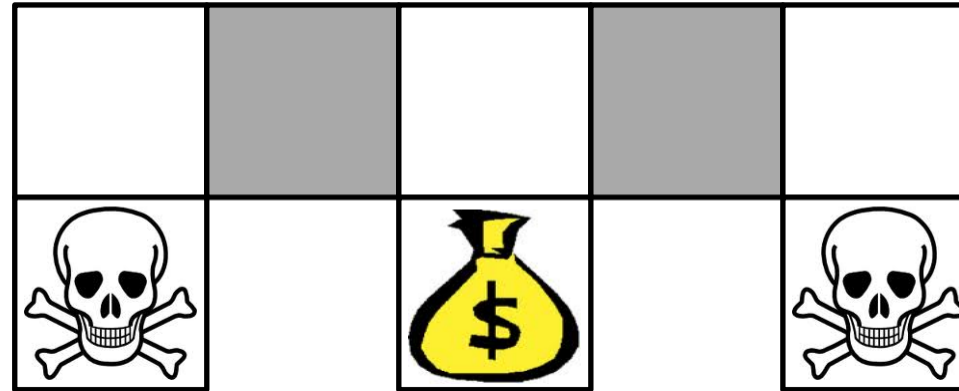
- Advantages:
 - Better convergence properties
 - Effective in high-dimensional or continuous action spaces
 - Can learn stochastic policies – (why do we need stochastic policy?)
- Disadvantages:
 - Typically converge to a local rather than global optimum
 - Evaluating a policy is typically inefficient and high variance

Example: Rock-Paper-Scissors

- Two-player game of rock-paper-scissors
 - Scissors beats paper
 - Rock beats scissors
 - Paper beats rock
- Consider policies for *iterated* rock-paper-scissors
 - A deterministic policy is easily exploited
 - A uniform random policy is optimal (i.e. Nash equilibrium)
- This is not actually a good example (2 player game instead of MDP)
 - But we do not always use RL algorithms on strictly MDP problems!



Example: Aliased Gridworld (1)



- Consider features of the following form (for all N, E, S, W)

$$\phi(s, a) = \mathbb{I}(\text{wall to N, } a = \text{move E})$$

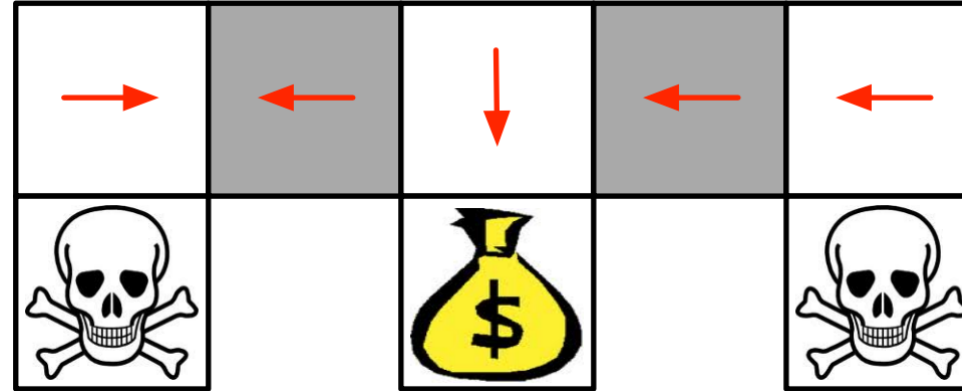
- The agent cannot differentiate the grey states
- Compare value-based RL, using an approximate value function

$$Q_{\theta}(s, a) = f(\phi(s, a), \theta)$$

- To policy-based RL, using a parametrized policy

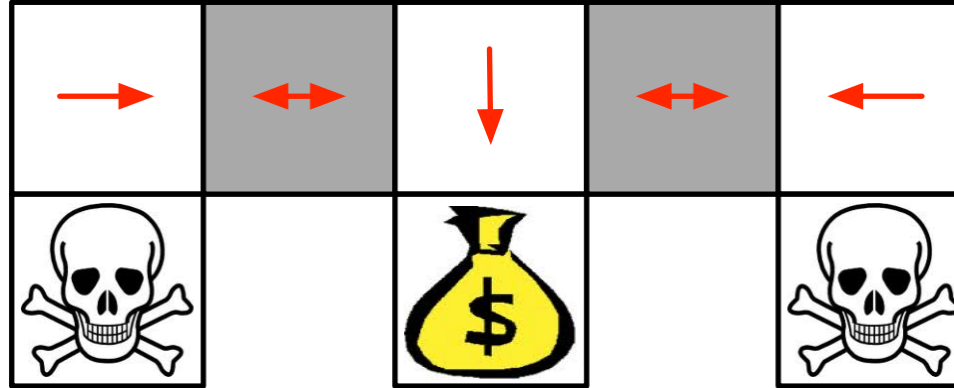
$$\pi_{\theta}(a|s) = g(\phi(s, a), \theta)$$

Example: Aliased Gridworld (2)



- Under aliasing, an optimal **deterministic** policy will either
 - move W in both grey states (shown by red arrows)
 - move E in both grey states
- Either way, it can get stuck and never reach the money
- Value-based RL learns a near-deterministic policy
 - e.g. greedy or ϵ -greedy
- So it will traverse the corridor for a long time

Example: Aliased Gridworld (3)



- An optimal **stochastic** policy will randomly move E or W in grey states
$$\pi_{\theta}(\text{wall to N and S, move E}) = 0.5$$
$$\pi_{\theta}(\text{wall to N and S, move W}) = 0.5$$
- It will reach the goal state in a few steps with high probability
- Policy-based RL can learn the optimal stochastic policy
- Why is it different from tabular MDP case? → adoption of function approximators

Policy Objective Functions

- Goal: given policy $\pi_\theta(a|s)$ with parameters θ , find best θ
- But how do we measure the quality of a policy π_θ ?

- In episodic environments we can use the **start value**

$$J_0(\theta) = V^{\pi_\theta}(s_0)$$

- In continuing environments we can use the **average value**

$$J_{avV}(\theta) = \sum_s d^{\pi_\theta}(s) V^{\pi_\theta}(s)$$

- where $d^{\pi_\theta}(s)$ is **stationary distribution** of Markov chain for π_θ
- Or all the other variants with different expectation on $V^{\pi_\theta}(s)$

Policy Optimization

- Policy based reinforcement learning is an **optimization** problem
- Find θ that maximizes $J(\theta) = V^{\pi_\theta}(s_0)$
- Some approaches do not use gradient
- Greater efficiency often possible using gradient
 - Gradient descent
 - Conjugate gradient
 - Quasi-newton
- We mostly focus on gradient descent, many extensions possible
- And on methods that exploit sequential structure

Policy Gradient

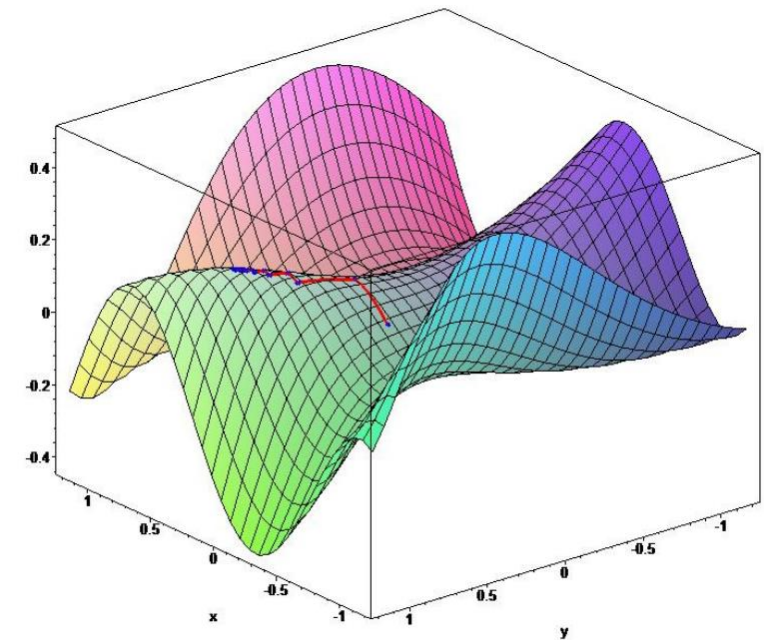
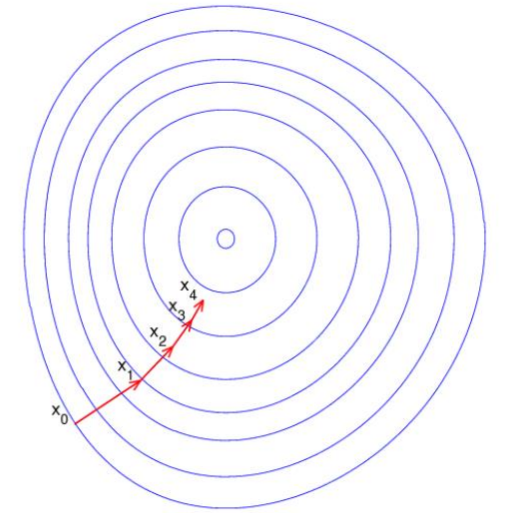
- Let $J(\theta)$ be any policy objective function
- Policy gradient algorithms search for a *local* maximum in $J(\theta)$ by ascending the gradient of the policy, with respect to parameters θ

$$\Delta\theta = \frac{1}{2}\alpha\nabla_{\theta}J(\theta)$$

- Where $\nabla_{\theta}J(\theta)$ is the policy gradient

$$\nabla_{\theta}J(\theta) = \begin{pmatrix} \frac{\partial J(\theta)}{\partial \theta_1} \\ \vdots \\ \frac{\partial J(\theta)}{\partial \theta_n} \end{pmatrix}$$

- and α is step-size parameter



Value of a Parameterized Policy

- Now assume policy π_θ is differentiable whenever it is non-zero and we know the gradient $\nabla_\theta \pi_\theta(a|s)$
- Recall policy value is $V^{\pi_\theta}(s_0) = \mathbb{E}_{\pi_\theta}[\sum_{t=0}^T R(s_t, a_t) | s_0]$ where the expectation is taken over the states & actions visited by π_θ
- We can re-express this in multiple ways
 - $V^{\pi_\theta}(s_0) = \sum_a \pi_\theta(a|s_0) Q^{\pi_\theta}(s_0, a)$
 - $V^{\pi_\theta}(s_0) = \sum_\tau p_\theta(\tau) R(\tau)$
 - Where $\tau = (s_0, a_0, r_0, \dots, s_{T-1}, r_{T-1}, s_T)$ is a state-action trajectory,
 - $p_\theta(\tau)$ is used to denote the probability over trajectories when executing policy π_θ starting in state s_0 ,
 - and $R(\tau) = \sum_{t=0}^T R(s_t, a_t)$ the sum of rewards for a trajectory τ

Likelihood Ratio Policy Gradient

- Using one of the definitions, the Goal is to find the policy parameters θ that:

$$\arg \max_{\theta} V^{\pi_{\theta}}(s_0) = \arg \max_{\theta} \sum_{\tau} p_{\theta}(\tau) R(\tau)$$

- Take the gradient with respect to θ :

$$\begin{aligned} \nabla_{\theta} V^{\pi_{\theta}}(s_0) &= \nabla_{\theta} \sum_{\tau} p_{\theta}(\tau) R(\tau) = \sum_{\tau} \nabla_{\theta} p_{\theta}(\tau) R(\tau) \\ &= \sum_{\tau} \nabla_{\theta} p_{\theta}(\tau) R(\tau) = \sum_{\tau} R(\tau) \underbrace{p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau)}_{\text{likelihood ratio}} \\ &= \mathbb{E}_{p_{\theta}(\tau)} [R(\tau) \nabla_{\theta} \log p_{\theta}(\tau)] \end{aligned}$$

Likelihood Ratio Policy Gradient

- Using one of the definitions, the Goal is to find the policy parameters θ that:

$$\arg \max_{\theta} V^{\pi_{\theta}}(s_0) = \arg \max_{\theta} \sum_{\tau} p_{\theta}(\tau) R(\tau)$$

- Take the gradient with respect to θ :

$$\nabla_{\theta} V^{\pi_{\theta}}(s_0) = \mathbb{E}_{p_{\theta}(\tau)} [R(\tau) \nabla_{\theta} \log p_{\theta}(\tau)]$$

- Approximate with empirical estimate for m sample trajectories under policy π_{θ} :

$$\nabla_{\theta} V^{\pi_{\theta}}(s_0) \approx \hat{g} = \frac{1}{m} \sum_{i=1}^m R(\tau_i) \nabla_{\theta} \log p_{\theta}(\tau_i)$$

Likelihood Ratio Policy Gradient

- Approximate with empirical estimate for m sample trajectories under policy π_θ :

$$\nabla_\theta V^{\pi_\theta}(s_0) \approx \hat{g} = \frac{1}{m} \sum_{i=1}^m R(\tau_i) \nabla_\theta \log p_\theta(\tau_i)$$

$$\nabla_\theta \log p_\theta(\tau_i) = \nabla_\theta \log \left[\underbrace{p(s_0)}_{\text{Initial state distrib.}} \prod_{t=0}^{T-1} \underbrace{\pi_\theta(a_t|s_t)}_{\text{Policy}} \underbrace{P(s_{t+1}|s_t, a_t)}_{\text{Dynamics model}} \right]$$

$$= \nabla_\theta \left[\log p(s_0) + \sum_{t=0}^{T-1} \log \pi_\theta(a_t|s_t) + \log P(s_{t+1}|s_t, a_t) \right]$$

$$= \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t)$$

No dynamics model required!

Likelihood Ratio / Score Function Policy Gradient

- Putting this together
- Goal is to find the policy parameters θ that:

$$\arg \max_{\theta} V^{\pi_{\theta}}(s_0) = \arg \max_{\theta} \sum_{\tau} p_{\theta}(\tau) R(\tau)$$

- Approximate with empirical estimate for m sample trajectories under policy π_{θ} :

$$\begin{aligned} \nabla_{\theta} V^{\pi_{\theta}}(s_0) &\approx \hat{g} = \frac{1}{m} \sum_{i=1}^m R(\tau_i) \nabla_{\theta} \log p_{\theta}(\tau_i) \\ &= \frac{1}{m} \sum_{i=1}^m R(\tau_i) \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \end{aligned}$$

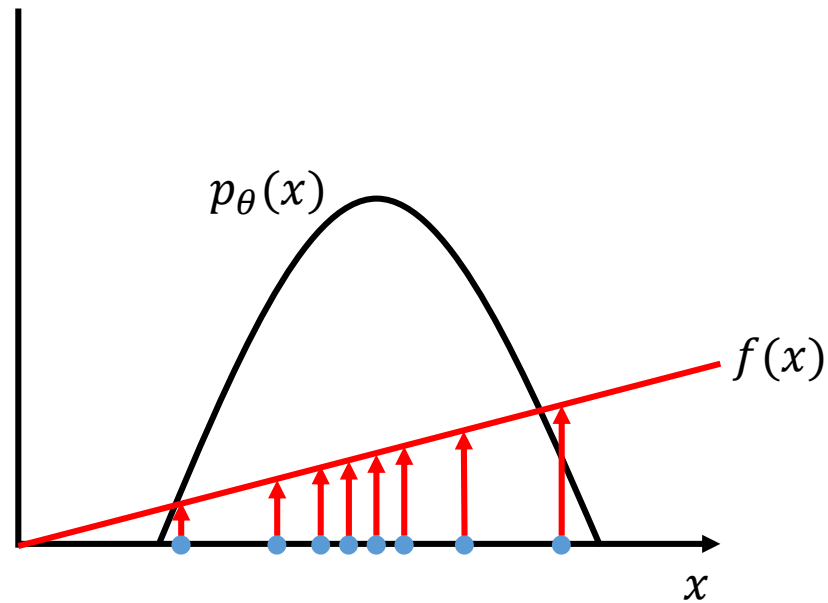
- Do not need to know dynamics model

Score Function Gradient Estimator: Intuition

- Consider generic form of $R(\tau_i)\nabla_{\theta} \log p_{\theta}(\tau_i)$: $\hat{g}_i = f(x_i)\nabla_{\theta} \log p_{\theta}(x_i)$
- $f(x)$ measures how good the sample x is.
- Moving in the direction $\hat{g}_i$ pushes up the log probability of the sample, in proportion to how good it is
- **Valid** even if $f(x)$ is **discontinuous**, and **unknown**, or **sample space** (containing x) is a **discrete set**

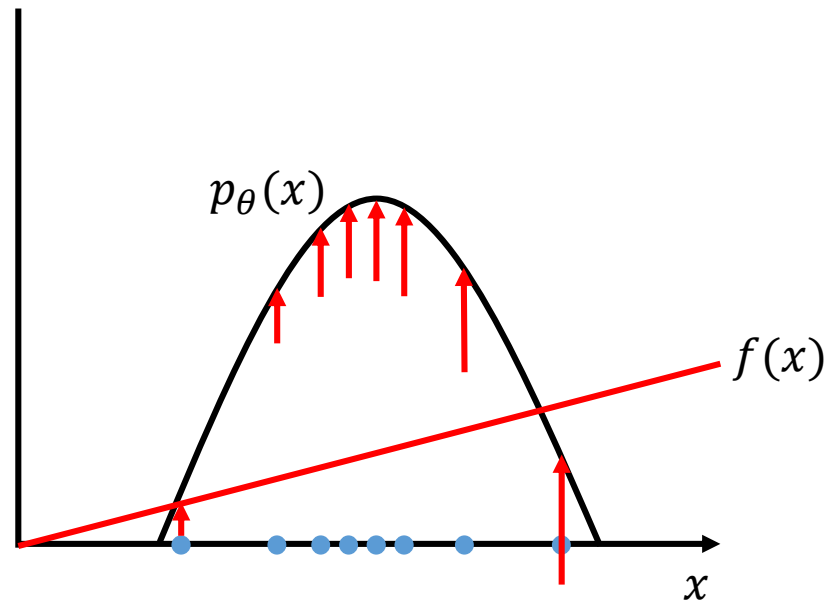
Score Function Gradient Estimator: Intuition

$$\hat{g}_i = f(x_i) \nabla_{\theta} \log p_{\theta}(x_i)$$



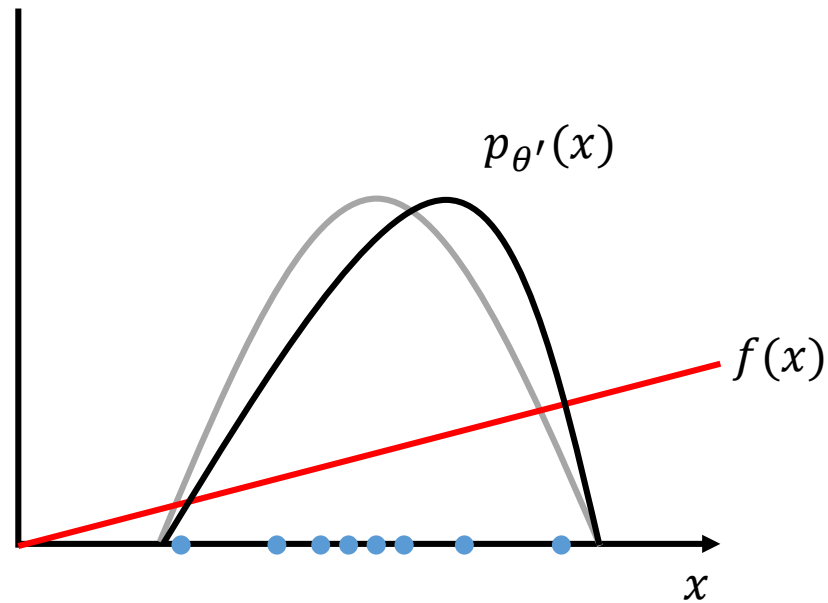
Score Function Gradient Estimator: Intuition

$$\hat{g}_i = f(x_i) \nabla_{\theta} \log p_{\theta}(x_i)$$



Score Function Gradient Estimator: Intuition

$$\hat{g}_i = f(x_i) \nabla_{\theta} \log p_{\theta}(x_i)$$



Likelihood Ratio / Score Function Policy Gradient

$$\nabla_{\theta} V^{\pi_{\theta}}(s_0) \approx \frac{1}{m} \sum_{i=1}^m R(\tau_i) \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

- Unbiased but very noisy; needs multiple trajectories to be sampled to make a single gradient update
- Fixes that make it practical
 - Temporal structure
 - Baseline
 - Alternatives to using Monte Carlo returns $R(\tau_i)$

Policy Gradient: Use Temporal Structure

- Previously:

$$\nabla_{\theta} V^{\pi_{\theta}}(s_0) = \mathbb{E}_{p_{\theta}(\tau)} \left[R(\tau) \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right] = \mathbb{E}_{p_{\theta}(\tau)} \left[\left(\sum_{t=0}^{T-1} \gamma^t r_t \right) \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]$$

- Alternatively, we can split the gradient over timesteps:

$$\nabla_{\theta} V^{\pi_{\theta}}(s_0) = \nabla_{\theta} \mathbb{E}_{\pi_{\theta} | s_0} \left[\sum_{t=0}^{T-1} \gamma^t r_t \right] = \mathbb{E}_{p_{\theta}(\tau)} \left[\sum_{t=0}^{T-1} \gamma^t r_t \sum_{t'=0}^{\textcolor{red}{t}} \nabla_{\theta} \log \pi_{\theta}(a_{t'} | s_{t'}) \right]$$

- This is because r_t does not depend on $\nabla_{\theta} \log \pi_{\theta}(a_{t'} | s_{t'})$ after $t' = t$

Policy Gradient: Use Temporal Structure

- Then,

$$\begin{aligned}\nabla_{\theta} V^{\pi_{\theta}}(s_0) &= \mathbb{E}_{p_{\theta}(\tau)} \left[\sum_{t=0}^{T-1} \gamma^t r_t \sum_{t'=0}^{\textcolor{red}{t}} \nabla_{\theta} \log \pi_{\theta}(a_{t'}|s_{t'}) \right] \\&= \mathbb{E}_{p_{\theta}(\tau)} \left[\begin{array}{l} r_0 \nabla_{\theta} \log \pi_{\theta}(a_0|s_0) + \\ \gamma r_1 \nabla_{\theta} \log \pi_{\theta}(a_0|s_0) + \quad \gamma r_1 \nabla_{\theta} \log \pi_{\theta}(a_1|s_1) + \\ \gamma^2 r_2 \nabla_{\theta} \log \pi_{\theta}(a_0|s_0) + \quad \gamma^2 r_2 \nabla_{\theta} \log \pi_{\theta}(a_1|s_1) + \quad \gamma^2 r_2 \nabla_{\theta} \log \pi_{\theta}(a_2|s_2) + \\ \vdots \end{array} \right] \\&= \mathbb{E}_{p_{\theta}(\tau)} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \gamma^t \sum_{t'=t}^{T-1} \gamma^{t'-t} r_{t'} \right] = \mathbb{E}_{p_{\theta}(\tau)} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t|s_t) \gamma^t G_t \right] \\&\quad \text{(where } G_t \text{ is the return of timestep } t\text{)}\end{aligned}$$

Monte-Carlo Policy Gradient (REINFORCE)

- Leverages likelihood ratio / score function and temporal structure
- Often used in large scale problems (mostly in NLP), due to its simplicity
- Since it can only be applied to episodic tasks, $\gamma = 1$ is usually assumed

1: Initialize policy parameter θ

2: **Loop**

3: Generate an episode $s_0, a_0, r_0, \dots, s_{T-1}, a_{T-1}, r_{T-1}, s_T$, following π_θ

4: **For** t in episode **do**

5: Compute return of timestep t :

$$G \leftarrow \sum_{k=t}^{T-1} \gamma^{k-t} r_k$$

6: Update weights:

$$\theta \leftarrow \theta + \alpha \gamma^t G \nabla \log \pi_\theta(a_t | s_t)$$

7: **End for**

8: **End loop**

Policy Gradient: Introduce Baseline

- Reduce variance by introducing a baseline $b(s)$

$$\nabla_{\theta} V^{\pi_{\theta}}(s_0) = \mathbb{E}_{p_{\theta}(\tau)} \left[\sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \gamma^t \left(\sum_{t'=t}^{T-1} \gamma^{t'-t} r_{t'} - b(s_t) \right) \right]$$

- For any choice of b , gradient estimator is unbiased.
- Near optimal choice is the expected return,

$$b(s_t) \approx \mathbb{E} \left[\sum_{t'=t}^{T-1} \gamma^{t'-t} r_{t'} \mid s_t \right] = V^{\pi}(s_t)$$

- Interpretation: increase log probability of action a_t proportionally to how much returns $\sum_{t'=t}^{T-1} \gamma^{t'-t} r_{t'}$ are better than expected
- This technique is called the **control variates** in the fields outside RL

REINFORCE with Baseline

1: Initialize policy parameter θ , baseline b

2: **Loop**

3: Generate an episode $s_0, a_0, r_0, \dots, s_{T-1}, a_{T-1}, r_{T-1}, s_T$, following π_θ

4: **For** t in episode **do**

5: Compute return of timestep t :

$$G_t \leftarrow \sum_{k=t}^{T-1} \gamma^{k-t} r_k$$

6: Update weights using advantage estimate:

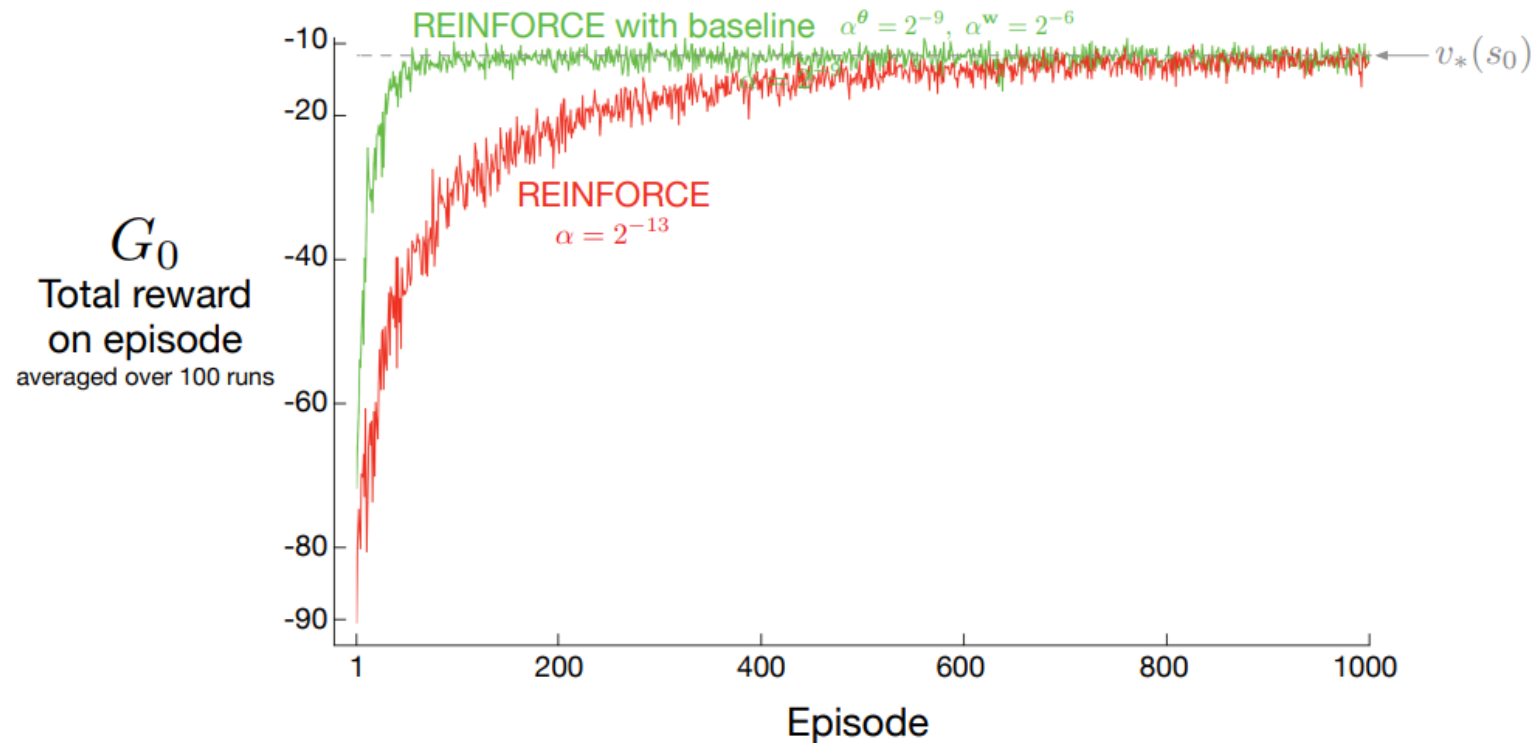
$$\theta \leftarrow \theta + \alpha \gamma^t (G_t - b(s_t)) \nabla \log \pi_\theta(a_t | s_t)$$

7: **End for**

8: Re-fit the baseline, by minimizing $\sum_i \sum_t \|b(s_t) - G_t^{(i)}\|^2$

9: **End loop**

REINFORCE with Baseline: an example



- Significant improvement from baseline!
- Still, it is MC-based algorithm that requires the episodes to be end
- Often baseline is not enough to fully reduce the variance when effective horizon is large

Policy Gradient Theorem

- Recall the REINFORCE gradient:

$$\nabla_{\theta} V^{\pi_{\theta}}(s_0) = \mathbb{E}_{p_{\theta}(\tau)} \left[\sum_{t=0}^{T-1} \gamma^t G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]$$

- We can also write in terms of true action-value function $Q^{\pi_{\theta}}(s, a)$ since $Q^{\pi_{\theta}}(s, a) = \mathbb{E}[G_t]$:

$$\nabla_{\theta} V^{\pi_{\theta}}(s_0) = \mathbb{E}_{p_{\theta}(\tau)} \left[\sum_{t=0}^{T-1} \gamma^t Q^{\pi_{\theta}}(s_t, a_t) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]$$

- We introduce $\mu_{\theta}(s) = \frac{1}{1-\gamma} \sum_{t=0}^{\infty} \gamma^t \Pr(s_t = s | \pi_{\theta})$, the normalized state visitation distribution

$$\nabla_{\theta} V^{\pi_{\theta}}(s_0) = \mathbb{E}_{\mu_{\theta}, \pi_{\theta}} [Q^{\pi_{\theta}}(s, a) \nabla_{\theta} \log \pi_{\theta}(a | s)]$$

- This is called the policy gradient theorem!
- Even if we start from different $J(\theta)$, we can change μ_{θ} respectively and still use the theorem

Choosing the Target

- We now have the policy gradient theorem:

$$\nabla J(\theta) = \mathbb{E}_{\mu_{\theta}, \pi_{\theta}} [Q^{\pi_{\theta}}(s, a) \nabla_{\theta} \log \pi_{\theta}(a|s)]$$

- REINFORCE is using MC target G_t instead of the true target $Q^{\pi_{\theta}}(s, a)$
- Similar to what we have seen in (MC vs TD), it is unbiased but high variance
- We want to reduce variance by introducing bias using bootstrapping & function approximation
 - It is called the **Actor-critic** methods
- Actor-critic methods maintain an explicit representation of **policy (an Actor)** and the **value function (a Critic)**, and update both

Reducing Variance Using a Critic

- Monte-Carlo policy gradient still has high variance
- We use a **critic** to estimate the action-value function,
$$Q_w(s, a) = Q^{\pi_\theta}(s, a)$$
- Actor-critic algorithms maintain two sets of parameters
 - **Critic** Updates action-value function parameters w evaluating the actor
 - **Actor** Updates policy parameters θ , in direction suggested by critic
- Actor-critic algorithms follow an approximate policy gradient

$$\nabla_\theta J(\theta) \approx \mathbb{E}_{\pi_\theta}[Q_w(s, a) \nabla_\theta \log \pi_\theta(a|s)]$$

$$\Delta\theta = \alpha Q_w(s, a) \nabla_\theta \log \pi_\theta(a|s)$$

Estimating the Action-Value Function

- The critic is solving a familiar problem: policy evaluation
- How good is policy π_θ for current parameters θ ?
- This problem was explored in previous lectures, e.g.
 - Monte-Carlo policy evaluation
 - Temporal-Difference learning
 - TD(λ)
- Could also use e.g. least-squares policy evaluation

Action-Value Actor-Critic

- Simple actor-critic algorithm based on action-value critic
- Using some value function approximation $Q_w(s, a)$
 - **Critic** Updates w by semi-gradient TD(0)
 - **Actor** Updates θ by policy gradient

1: Initialize policy parameter θ , baseline b , sample s, a

2: **Loop**

3: Sample reward $r = R(s, a)$, transition $s' \sim p(s'|s, a)$, next action $a' \sim \pi_\theta(a'|s')$

4: $\delta = r + \gamma Q_w(s', a') - Q_w(s, a)$

5: $\theta \leftarrow \theta + \alpha \nabla_\theta \log \pi_\theta(a|s) Q_w(s, a)$

6: $w \leftarrow w + \beta \delta \nabla_w Q_w(s, a)$

7: $a \leftarrow a', s \leftarrow s'$

8: **End loop**

Monotonic Improvements

- The characteristics of algorithm we desire
- Goal: Each step of policy gradient yields an updated policy $\pi_{\theta'}$ whose value is greater than or equal to the prior policy π_{θ} : $V^{\pi_{\theta'}} \geq V^{\pi_{\theta}}$
- Gradient descent approaches update the weights a small step in direction of gradient
- Locally a good approximation, further away less good
- Even if we use natural gradient, it is still a local approximation with different direction

Why are step sizes a big deal in RL?

- Step size is important in any problem involving finding the optima of a function
- Supervised learning: Step too far → next updates will fix it
- Reinforcement learning
 - Step too far → bad policy
 - Next batch: collected under bad policy
 - **Policy** is determining data collection!
 - Essentially controlling exploration and exploitation trade off due to particular policy parameters and the stochasticity of the policy
 - May not be able to recover from a bad choice, collapse in performance!

Policy Performance Difference Lemma

- Goal: find policy parameters that maximize value function

$$J(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right]$$

- Where $s_0 \sim p(s_0)$, $a_t \sim \pi_{\theta}(a_t|s_t)$, $s_{t+1} \sim p(s_{t+1}|s_t, a_t)$
- We would like to know whether the update policy $\pi_{\theta'}$ improves from π_{θ}
 - We can express value of $\pi_{\theta'}$ in terms of advantage over π_{θ}

$$J(\theta') = J(\theta) + \mathbb{E}_{\pi_{\theta'}} \left[\sum_{t=0}^{\infty} \gamma^t A^{\pi_{\theta}}(s_t, a_t) \right]$$

Policy Performance Difference Lemma

- We expressed value of $\pi_{\theta'}$ in terms of advantage over π_{θ}

$$J(\theta') = J(\theta) + \mathbb{E}_{\pi_{\theta'}} \left[\sum_{t=0}^{\infty} \gamma^t A^{\pi_{\theta}}(s_t, a_t) \right] = J(\theta) + \sum_s \mu_{\theta'}(s) \sum_a \pi_{\theta'}(a|s) A^{\pi_{\theta}}(s, a)$$

- Where $\mu_{\theta'}(s)$ is the discounted weighted frequency of state under policy $\pi_{\theta'}$
- We know the advantage $A^{\pi_{\theta}}$ and $\pi_{\theta'}$
- But we can't compute the above because we don't know $\mu_{\theta'}(s)$, the state distribution under the new proposed policy

Local Approximation

$$J(\theta') = J(\theta) + \sum_s \mu_{\theta'}(s) \sum_a \pi_{\theta'}(a|s) A^{\pi_{\theta}}(s, a)$$

- Can we remove the dependency on the discounted visitation frequencies under the new policy?
- Substitute in the discounted visitation frequencies under the current policy to define a new objective function:

$$L_{\theta}(\theta') = J(\theta) + \sum_s \mu_{\theta}(s) \sum_a \pi_{\theta'}(a|s) A^{\pi_{\theta}}(s, a)$$

- This matches to first order:

$$\begin{aligned} \nabla_{\theta'} L_{\theta}(\theta') \Big|_{\theta} &= \nabla_{\theta'} \mathbb{E}_{s \sim \mu_{\theta}, a \sim \pi_{\theta'}} [A^{\pi_{\theta}}(s, a)] \Big|_{\theta} \\ &= \mathbb{E}_{s \sim \mu_{\theta}, a \sim \pi_{\theta'}} [A^{\pi_{\theta}}(s, a) \nabla_{\theta'} \log \pi_{\theta'}(a|s)] \Big|_{\theta} = \nabla_{\theta'} J(\theta') \Big|_{\theta} \end{aligned}$$

- Local approximation to the performance of the policy
- And this is what we can optimize!

Policy Improvement Bound

- Is there a bound on the performance of a new policy obtained by optimizing the surrogate objective? For $L_{\theta}(\theta') = J(\theta) + \sum_s \mu_{\theta}(s) \sum_a \pi_{\theta'}(a|s) A^{\pi_{\theta}}(s, a)$:

Theorem (Policy Improvement Bound)

For arbitrary θ and θ' , with $\epsilon = \max_{s,a} |A^{\pi_{\theta}}(s, a)|$, the following always holds

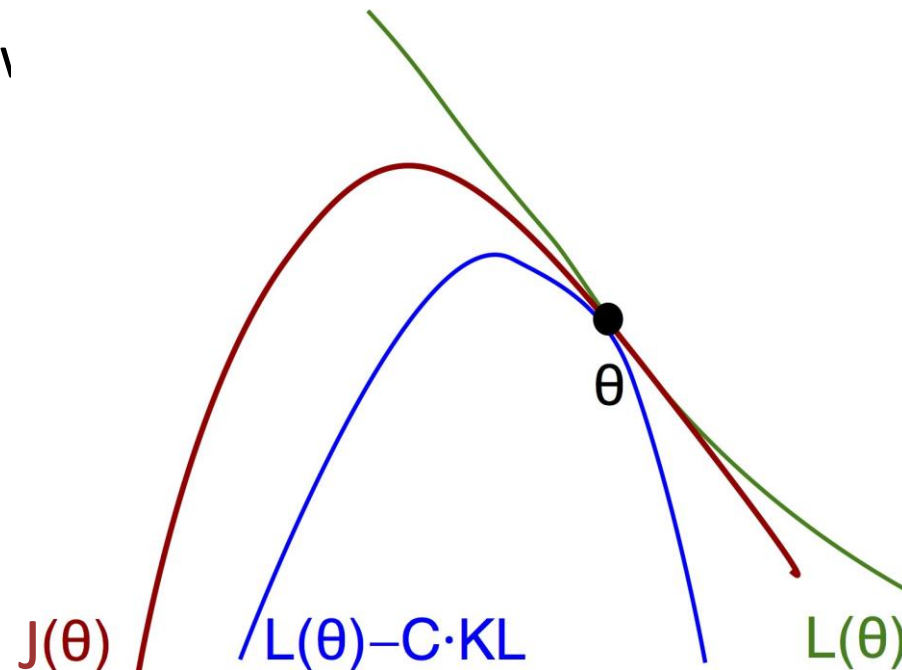
$$J(\theta') \geq L_{\theta}(\theta') - \frac{4\gamma\epsilon}{(1-\gamma)^2} \max_s KL(\pi_{\theta}(\cdot | s) \parallel \pi_{\theta'}(\cdot | s))$$

- See (Schulman, John, et al. "Trust region policy optimization." ICML 2015), (Achiam, Joshua, et al. "Constrained policy optimization." ICML 2017) for proof
 - There are several different versions of theorems
 - However, the idea is to make a **lower bound on policy performance**, wrt. the **policy difference**

Guaranteed Improvement

$$J(\theta') \geq L_{\theta}(\theta') - \frac{4\gamma\epsilon}{(1-\gamma)^2} \max_s KL(\pi_{\theta}(\cdot | s) || \pi_{\theta'}(\cdot | s))$$

- J is the true objective we do not know, L is the surrogate local approximation, $L - C \cdot KL$ is the lower bound



From Theory to Practice: Trust Region

- Goal is to optimize

$$\max_{\theta'} L_{\theta}(\theta') - \frac{4\gamma\epsilon}{(1-\gamma)^2} \max_s KL(\pi_{\theta}(\cdot | s) \| \pi_{\theta'}(\cdot | s)) = L_{\theta}(\theta') - C \cdot D_{KL}^{\max}(\pi_{\theta} \| \pi_{\theta'})$$

- Where C is the penalty coefficient
- In practice, if we used the penalty coefficient recommended by the theory above $C = \frac{4\gamma\epsilon}{(1-\gamma)^2}$, the step sizes would be very small
- New idea: Use a trust region constraint on step sizes. Do this by imposing a constraint on the KL divergence between the new and old policy.

$$\begin{aligned} & \max_{\theta'} L_{\theta}(\theta') \\ & \text{subject to } D_{KL}^{\max}(\pi_{\theta} \| \pi_{\theta'}) \leq \delta \end{aligned}$$

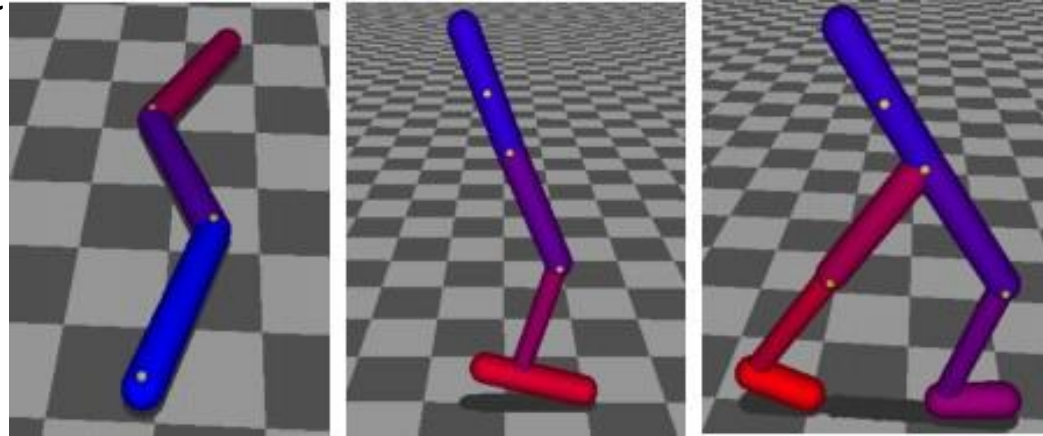
Trust Region Policy Optimization

- Schulman, John, et al. "Trust region policy optimization." ICML 2015

- 1: Initialize policy parameters θ , set trust region hyperparameter δ
- 2: **Loop**
- 3: Run policy π_θ for T timesteps
- 4: Estimate advantage function A^{π_θ} at all timesteps
- 5: Compute policy gradient $g = \nabla_{\theta'} L_\theta(\theta') = \mathbb{E}_{\pi_\theta} \left[\frac{\pi_{\theta'}(a|s)}{\pi_\theta(a|s)} A^{\pi_\theta}(s, a) \right]$
- 6: Use conjugate gradient (with Hessian-vector products) to compute $F^{-1}g$ (this gives the direction)
- 7: Do line search on surrogate loss and KL constraint, to find out maximum step size that meets the constraint δ
- 8: **End loop**

Trust Region Policy Optimization

- Applied to
 - Locomotion controllers in 2D (swimmer, hopper, etc. MuloCo simulator is often used)



- Atari games with pixel input

TRPO Summary

- Policy gradient approach
- Uses surrogate optimization objective
- Automatically constrains the weight update to a trusted region
- Empirically consistently does well
- Very influential: +4000 citations since introduced a few years ago

Drawbacks of TRPO

- Not easy to enforce trust region constraint for complex policy architectures, e.g.
 - Networks that have stochasticity like dropout
 - Parameter sharing between policy and value function
- Conjugate Gradient implementation is complex
- We can't use good first-order optimizers like Adam, RMSProp...
- Due to these reasons, TRPO is not used very often these days

PPO – “Clipped Surrogate Loss”

- PPO:

$$\max_{\theta'} \mathbb{E}_{\pi_{\theta}} \left[\min \left(\frac{\pi_{\theta'}(a|s)}{\pi_{\theta}(a|s)} A^{\pi_{\theta}}(s, a), \quad \text{clip} \left(\frac{\pi_{\theta'}(a|s)}{\pi_{\theta}(a|s)}, 1 - \epsilon, 1 + \epsilon \right) A^{\pi_{\theta}}(s, a) \right) \right]$$

- If there is no update, the policy ratio $\frac{\pi_{\theta'}(a|s)}{\pi_{\theta}(a|s)} = 1$
- Here we are clipping the “expected additional advantage from the policy update” to certain amount
- Such that it is meaningless to update over $\frac{\pi_{\theta'}(a|s)}{\pi_{\theta}(a|s)} > 1 + \epsilon$ (positive advantage) or $\frac{\pi_{\theta'}(a|s)}{\pi_{\theta}(a|s)} < 1 - \epsilon$ (negative advantage)

Applications of PPO: OpenAI Five

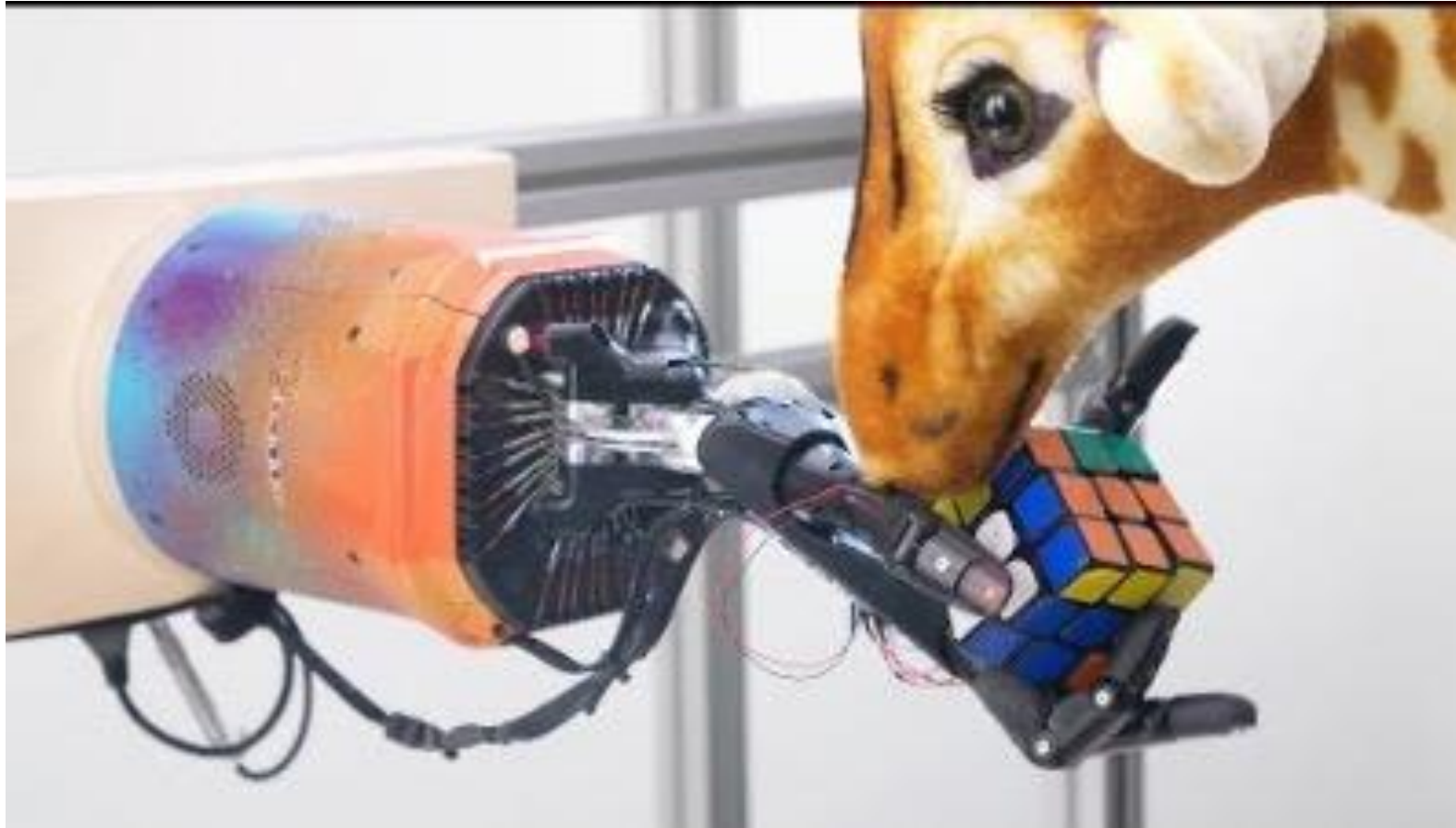
OpenAI Five Defeats Dota 2 World Champions

OpenAI Five is the first AI to beat the world champions in an esports game, having won two back-to-back games versus the world champion Dota 2 team, OG, at Finals this weekend. Both OpenAI Five and DeepMind's AlphaStar had previously beaten good pros privately but lost their live pro matches, making this also the first time an AI has beaten esports pros on livestream.

- OpenAI-5 was trained with PPO



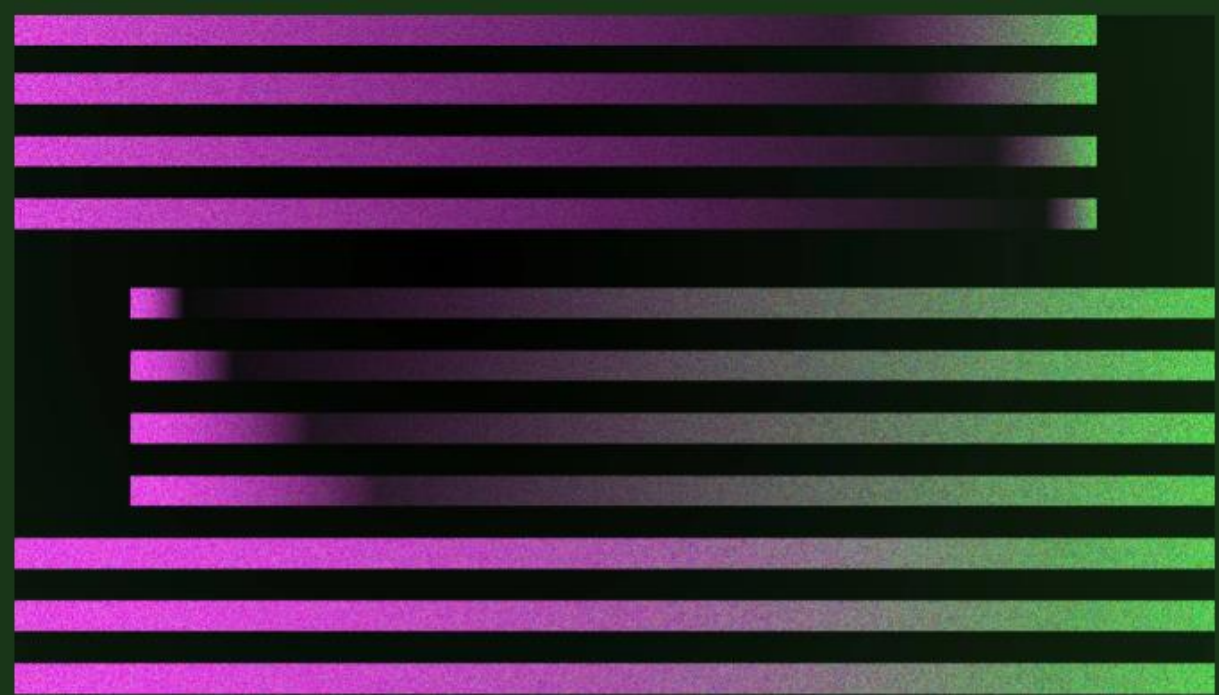
Applications of PPO: OpenAI Rubik's Cube



Applications of PPO: ChatGPT

Introducing ChatGPT

We've trained a model called ChatGPT which interacts in a conversational way. The dialogue format makes it possible for ChatGPT to answer followup questions, admit its mistakes, challenge incorrect premises, and reject inappropriate requests.



- ChatGPT is also trained by PPO!
- John Schulman (author of TRPO/PPO) is co-founder of OpenAI

Policy Gradient Summary

- Extremely popular and useful set of approaches
- Can incorporate prior knowledge by choosing the policy parameterization
- REINFORCE can be a first choice if you are trying to apply RL to real-world large problems
 - Then can try advanced methods – e.g. PPO
- Understand where different estimators can be slotted in (and implications for bias/variance)
- Don't have to be able to derive or remember the specific formulas